

Добрый день!

Хочу отметить ряд моментов.

1. Все задания и бонус выполнил и получил хорошие результаты по метрикам
2. Провел полный анализ данных включая географическое положение объектов
3. Первоначальную очистку провел по пустому `description`
4. Провел поиск нерелевантных записей в `en_txt` с последующим контролем по фото
5. С помощью семантического анализа выбрал типы объектов
6. Провел стратификацию набора не только по Name-description, но и по типу объектов
7. Для токенизации и создания эмбедингов использовал `cointegrated/rubert-tiny2` <https://huggingface.co/cointegrated/rubert-tiny2>
8. В качестве векторной БД использовал `Chroma`
9. Сделал Визуализацию PCA, TSNE, SVD
10. В качестве LLM reader использовал GigaChat (использовал около 90 тыс. токенов)
11. В качестве ре ранжирования использовал `cointegrated/rubert-tiny2` <https://huggingface.co/cointegrated/rubert-tiny2> обернув в объект
12. В качестве промта использовал расширенную версию для насыщения ответа и компенсации анемичности корпуса данных
13. Реализовал Answer Relevancy. Общая оценка *удовлетворительно*
14. Реализовал 2 метрики retrieval-части. Метрика `Context Recall` лежит в области *удовлетворительно*. Метрика `Context Precision` лежит в области *хорошо*

Образы:

- Векторная БД `Chroma` <https://disk.yandex.ru/d/FZt9lOA7Od5JYw> (40M)
- Датасет документов с метаинформацией и метриками (pkl) <https://disk.yandex.ru/d/5TaD7iXsm5z2pA> (4M)

Удачи! Александр. @AlexKChin

Домашнее задание. RAG.

В этом домашнем задании вам предстоит создать туристический RAG, провести EDA и проявить креатив! :)

Вам дан набор данных, содержащий информацию о памятниках, архитектуре и других значимых местах. В данных есть:

- название достопримечательности;
- идентификатор на WikiData;
- город расположения;
- координаты долготы и широты;
- описание (извлеченное из WikiData);
- изображение в формате base64;

- сгенерированное описание изображения (генерация с помощью модели BLIP).

Некоторые из памятников могут встречаться несколько раз, но с различными изображениями, из которых не все могут быть качественными. Для компенсации этого даны текстовые описания.

Часть данных взята с Национальной технологической олимпиады студентов по CV. Базовое решение предполагает использование мультимодального ru-CLIP. Но в данном домашнем задании предлагается решить эту задачу творчески, используя NLP подход.

Установка и импорт библиотек

```
#!/pip install -q gdown torch transformers transformers accelerate
bitsandbytes langchain sentence-transformers faiss-gpu openpyxl
datasets langchain-community ragatouille umap-learn

# from langchain.docstore.document import Document as
LangchainDocument
# from langchain.vectorstores import FAISS
# from langchain_community.embeddings import HuggingFaceEmbeddings
# from langchain_community.vectorstores.utils import DistanceStrategy
# from ragatouille import RAGPretrainedModel
# from sentence_transformers import SentenceTransformer

#-----
# from langchain.text_splitter import ... # выберите другую стратегию
разбивки на чанки

# pd.set_option("display.max_colwidth", None)
```

Загрузка данных (4 балла)

```
from networkx.algorithms.traversal import dfs_edges

url = 'https://drive.google.com/uc?id=1P1BsvI2jPN3fEqjc2YZxmQ-
MTs22WVUk'
output = r'C:\prj\data\points_of_interest\file.csv'

# gdown.download(url, output, quiet=False)

import pandas as pd

df = pd.read_csv(output)

df.head()
```

Unnamed: 0	Name	WikiData	City	Lon
Lat \				
0	0	Динамо	Q37996725	Екатеринбург
			60.600349	56.845398

1	1	Динамо	Q37996725	Екатеринбург	60.600349	56.845398
2	2	Динамо	Q37996725	Екатеринбург	60.600349	56.845398
3	3	Динамо	Q37996725	Екатеринбург	60.600349	56.845398
4	4	Динамо	Q37996725	Екатеринбург	60.600349	56.845398

```

description \
0 спорткомплекс в Екатеринбурге
1 спорткомплекс в Екатеринбурге
2 спорткомплекс в Екатеринбурге
3 спорткомплекс в Екатеринбурге
4 спорткомплекс в Екатеринбурге

```

```

image \
0 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
1 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
2 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
3 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
4 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...

```

```

en_txt
0 there are two people that are standing on a tr...
1 arafed building with a blue and white exterior...
2 there is a blue and white building with a cloc...
3 there is a blue and white building with a cloc...
4 there is a boat that is in the water near a bu...

```

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14634 entries, 0 to 14633
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      14634 non-null  int64
1   Name            14634 non-null  object
2   WikiData        12078 non-null  object
3   City            14634 non-null  object
4   Lon             14634 non-null  float64
5   Lat             14634 non-null  float64
6   description     12078 non-null  object
7   image           14634 non-null  object
8   en_txt          14634 non-null  object
dtypes: float64(2), int64(1), object(6)
memory usage: 1.0+ MB

```

```
df.columns
```

```
Index(['Unnamed: 0', 'Name', 'WikiData', 'City', 'Lon', 'Lat',  
      'description',  
      'image', 'en_txt'],  
      dtype='object')
```

```
df.rename(columns={'Unnamed: 0': 'Num'}, inplace=True)
```

Для декодирования изображений, закодированных в формате base64, можно использовать следующий код:

```
import base64  
import requests  
import matplotlib.pyplot as plt  
from PIL import Image  
import PIL  
from io import BytesIO  
  
def get_image(data):  
    image = Image.open(BytesIO(base64.b64decode(data))).convert('RGB')  
    return image  
  
get_image(df.sample(1)['image'].tolist()[0])
```



В данных есть выбросы — записи, не относящихся к туристическим или историческим объектам. Для очистки данных можно, например, использовать алгоритм TF-IDF: создаем

корпус текстов из описаний, применяем TF-IDF, извлекаем наиболее редко или часто встречающиеся слова. Они могут указывать на мусорные данные, такие как мемы, баннеры или личные фото людей. После выделения подозрительных паттернов, можно провести ручную проверку и фильтрацию данных.

```
CITIES = df.City.unique()
CITIES
array(['Екатеринбург', 'Нижний Новгород', 'Владимир', 'Ярославль'],
      dtype=object)
```

Первоначальная очистка

```
df['description'] = df['description'].fillna('')
df['en_txt'] = df['en_txt'].fillna('')
```

Очистим корпус от лишних пробелов

```
df['description'] = df['description'].str.strip()
df['en_txt'] = df['en_txt'].str.strip()
```

Считаем количество пустых записей с описанием

```
(df['description'] == '').sum()
2556
```

Описание фотографий не пустое

```
(df['en_txt'] == '').sum()
0
print("Найдено записи с пустым описанием (%): ",
      round((df['description'] == '').sum() / df.shape[0] * 100, 1))
Найдено записи с пустым описанием (%): 17.5
```

Посмотрим какие параметры пустые при пустом описании

```
df[df['description'] == ''].nunique()
Num          2500
Name          71
WikiData       0
City           2
Lon           74
Lat           73
description     1
image        2404
```

```
en_txt      2230
dtype: int64
```

Name, City, WikiData разрежены при пустом описании

Предполагаем что это дополнительные фотографии, ценность которых сомнительна

Рассмотрим конкретные записи при пустом desc

```
df[df['description'] == ''][
    ['Name', 'WikiData', 'description', 'en_txt']].sample(10)
```

	description \	Name	WikiData
9388		Мальчик с рогаткой	NaN
9218	Оборонительный земляной вал древнего Владимира		NaN
11308		Дом-музей Максима Богдановича	NaN
9393		Мальчик с рогаткой	NaN
9718		Памятник Гоголю	NaN
9783		Макет ракеты-носителя Союз-ТМ	NaN
9648		Великая Княгиня Ольга	NaN
8131		Кресень	NaN
9109		Дева Мария	NaN
9851		Чернобыль - трагедия XX века	NaN

	en_txt
9388	two boys are sitting on a fence with a rope
9218	painting of a fence and a gate in a desert area
11308	there is a small wooden house with a metal roof
9393	arafed boy holding a tennis racket in a field
9718	there is a statue of a man sitting on a pedestal
9783	a model of a rocket with a red and white strip...
9648	painting of a woman in a white robe and red cape
8131	a close up of a knife on a wooden table
9109	a painting of a woman holding a child in her arms
9851	there are many military vehicles and trucks on...

Большинство записей имеют не конкретное название

Посмотрим примеры фото по найденным индексам

```
get_image(df.at[9346, 'image'])
```



```
get_image(df.at[9800, 'image'])
```


РАКЕТА-НОСИТЕЛЬ «СОЮЗ-ФГ»



Стартовая масса
~313 т



Компоненты топлива:
I-III ступени - кислород-керосин



Основной двигатель:
I СТУПЕНЬ - 4 x РД-107А,
II СТУПЕНЬ - РД-108А,
III СТУПЕНЬ - РД-0110



Используемые разрывные блоки:
«ФРЕЗЛ»
При запуске пилотируемых и грузовых
кораблей разрывной блок не используется



Космодром:
БАЙКОНУР



Масса полезного груза, т:
- с пилотируемым кораблем типа «Союз» - 7,2
- с грузовым кораблем типа «Прогресс» - 7,4



```
get_image(df.at[9584, 'image'])
```



Фото тоже имеет содержание которое слабо относится к достопримечательностям

Принимаем решение удалить записи с пустым description и WikiData

```
df = df[df['description'] != '']
```

Статистика по датасету

```
df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 12078 entries, 0 to 14633
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Num              12078 non-null  int64
1   Name             12078 non-null  object
2   WikiData         12078 non-null  object
3   City             12078 non-null  object
4   Lon              12078 non-null  float64
5   Lat              12078 non-null  float64
6   description      12078 non-null  object
7   image            12078 non-null  object
8   en_txt           12078 non-null  object
dtypes: float64(2), int64(1), object(6)
memory usage: 943.6+ KB
```

EDA разведочный анализ данных

Поле Num

```
df.Num.describe()

count      12078.000000
mean       1810.912651
std        1169.944608
min         0.000000
25%        775.250000
50%       1722.500000
75%       2821.000000
max       4371.000000
Name: Num, dtype: float64
```

```
df.Num.unique().shape[0]
```

```
-----
AttributeError
last)
```

```
Traceback (most recent call
```

```

~\AppData\Local\Temp\ipykernel_35228\1499976500.py in ?()
----> 1 df.Num.unique().shape[0]

C:\prj\DLS\dls_part_2_nlp\venv\Lib\site-packages\pandas\core\
generic.py in ?(self, name)
    6317         and name not in self._accessors
    6318         and
self._info_axis._can_hold_identifiers_and_holds_name(name)
    6319     ):
    6320         return self[name]
-> 6321     return object.__getattribute__(self, name)

AttributeError: 'DataFrame' object has no attribute 'Num'

```

На 12 тыс записей имеем всего 4 тыс уникальных значений

Построим распределение по дубликатам

```

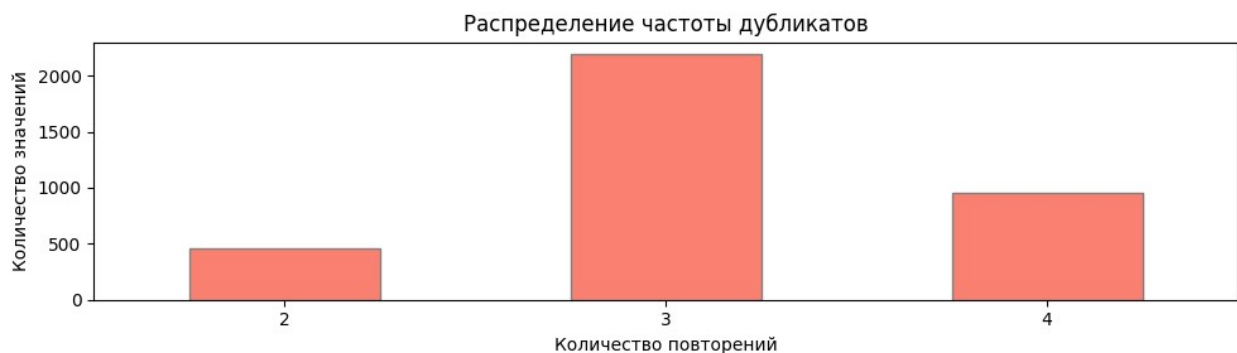
def frequency_distribution(s: pd.Series):
    value_counts = s.value_counts()
    duplicate_counts = value_counts[value_counts > 1]
    plt.figure(figsize=(10, 3))

    frequency_distribution =
duplicate_counts.value_counts().sort_index()
    frequency_distribution.plot(kind='bar', color='salmon',
edgecolor='grey')
    plt.title('Распределение частоты дубликатов')
    plt.xlabel('Количество повторений')
    plt.ylabel('Количество значений')
    plt.xticks(rotation=0)

    plt.tight_layout()
    plt.show()

frequency_distribution(df['Num'])

```



```
for city in CITIES:
    print(city, df[df.City == city]['Num'].duplicated().sum())
```

```
Екатеринбург 0
Нижний Новгород 0
Владимир 0
Ярославль 0
```

Получается что Num уникален внутри города.

Поле Name

```
df['Name'].unique().shape[0]
```

```
317
```

```
df['Name'].unique()
```

```
array(['Динамо', '№32 Дом обороны', 'Дом обороны',
       '№28 Здание городской электростанции «Луч»',
       '№27 Дом Г.Н. Скрябина', 'Литературная жизнь Урала XX века',
       'Дом-музей Бажова', 'Театр кукол', 'Харитоновский сад',
       'Храм в честь Вознесения Господня', 'Суворовское училище',
       'Свято-Троицкий Кафедральный собор',
       'Дом-музей Д. Н. Мамина-Сибиряка',
       'Художественный Музей Эрнста Неизвестного', 'Серго
Орджоникидзе',
       'Н. И. Кузнецову', 'Успенский собор на ВИЗе', 'Царский мост',
       'Дом печати (Екатеринбург)', 'Музей изобразительных искусств',
       'Алекسانдро-Невский собор', 'Здание бывшей гостиницы "Мадрид"',
       'Екатеринбургский музей изобразительных искусств',
       'Уральский государственный медицинский университет',
       'Центральный стадион', 'Уральская государственная
консерватория',
       '№18 Фотографический музей «Дом Метенкова»',
       'Преображенская церковь на Уктусе',
       'Уральский геологический музей', 'Дом Метенкова', 'Гимназия №
9',
       'Второй Дом Советов', 'Водонапорная башня', 'Театр оперы и
балета',
       'Малышеву', 'Памятник Я.М. Свердлову', 'Комсомолу Урала',
       'Ленин',
       'Шарташские каменные палатки', '№34 Дом контор',
       'Алекسانдро-Невская часовня', '№19 Музей истории
Екатеринбурга',
       'Городок Чекистов', 'Иоанно-Предтеченский Кафедральный собор',
       'Дом контор',
       'Уральский государственный колледж имени И.И. Ползунова',
       'Площадь 1905 года', 'Музей истории Екатеринбурга',
       'Музей кукол и детской книги «Страна чудес»',
```

'Музей истории, науки и техники Свердловской железной дороги',
 'Свердловское художественное училище им. И. Д. Шадра',
 '№12 Дом Севастьянова',
 '№17 Свердловская государственная академическая филармония',
 '№16 Усадьба Расторгуева-Харитонов',
 'Усадьба Расторгуевых – Харитоновых',
 'Уральский политехнический колледж',
 'Музей плодового садоводства Среднего Урала', 'Филармония',
 'Дом-музей Ф.М. Решетникова', 'Каменный мост', '№15 Храм-на-Крови',
 'Храм на Крови', 'Белая башня', 'Добровольцам-танкистам',
 'Ново-Тихвинский монастырь', '№29 Дом В.И. Чувильдина',
 'Свердловский областной краеведческий музей',
 'Театр музыкальной комедии', 'Дом Актёра',
 'Доходный дом купца Чувильдина', 'улица Февральской Революции',
 'улица Чернышевского', 'улица Добролюбова',
 'Мемориал памяти Уральских коммунаров', 'Екатеринбургский завод',
 'Уральский турбинный завод', 'Усадьба Железнова',
 'Усадьба ротмистра Переяславцева',
 'Комплекс штаба военного округа',
 'Уральский государственный университет имени А. М. Горького',
 'Уральский государственный технический университет',
 'Площадь 1-й Пятилетки', 'Усадьба Тарасова', 'Дом Артистов',
 'Дом Э. Ф. Филитц',
 'Здание пансиона и церкви Святой Магдалины при первой женской гимназии',
 'Дом Бревновых', 'Дом Маева', 'Дом Промышленности',
 'Дом Степанова', 'Крестовоздвиженская церковь (Екатеринбург)',
 'Музей «Литературная жизнь Урала XIX века»',
 'Свято-Никольский храм (Екатеринбург)',
 'Центральная городская библиотека им. А. И. Герцена',
 'Дом Гайдара', 'Дом дворянки Е. О. Селивановой',
 'Дом связи (Екатеринбург)', 'Дом В. И. Смирнова',
 'Памятник Ленину', 'Северная башня', 'Никольская башня',
 'Дом архитектора',
 'Нижегородский государственный академический театр драмы им. М. Горького',
 'Нижегородский кремль', 'Нижегородская соборная мечеть',
 'Домик Каширина', 'Автозаводский парк культуры и отдыха',
 'Собор Александра Невского', 'Собор Архангела Михаила',
 'Церковь в честь Рождества Богородицы',
 'Спасо-преображенская церковь', 'Дворец вице-губернатора',
 'Парк имени Кулибина', 'Церковь Воскресения Христова',
 'Храм Сергия Радонежского', 'Храм святителей Московских',
 'Скоба',
 'Нижегородский Кремль', 'Дмитриевская башня', 'Лицей № 82',
 'Часовая башня', 'Горьковчане фронту',
 'Нижегородский государственный художественный музей',

'Рождественская церковь', 'Дмитровская башня',
 'Музей-квартира А.М. Горького', 'Благовещенский монастырь',
 'Благовещенский собор', 'Нижегородское речное училище',
 'Парк имени А.С. Пушкина', 'Тайницкая башня',
 'Монумент героям Волжской военной флотилии', 'Гарнизонная
 казарма',
 'Памятник жертвам и мученикам революции 1905 года',
 'Домик Петра I', 'Государственный центральный банк',
 'Экспозиция «Подвиг народного единства»', 'Усадьба
 Рукавишниковых',
 'Обелиск Минину и Пожарскому',
 'Церковь Знамения Божией Матери и святых Жен-Мироносиц',
 'Ивановская башня', 'Нижегородский острог', 'Сормовский парк',
 'Блиновский пассаж', 'Шахматный дом (Игорный дом А.И.
 Троицкого)',
 'Рекорд',
 'Нижегородская государственная консерватория им. М.И. Глинки',
 'Церковь Рождества Иоана Предтечи', 'Коромыслова башня',
 'Усадьба Добролюбовых', 'Борисоглебская башня',
 'Дом-музей Добролюбова', 'Дом Сироткина',
 'Спасский староармарочный кафедральный собор',
 'Георгиевская башня', 'Локомотив', 'Крестовоздвиженский
 монастырь',
 'Доходный дом купчихи П. Е. Кубаревой', 'Чкаловская лестница',
 'Администрация Нижнего Новгорода', 'Дом Советов',
 'Пороховая башня', 'Чкаловская лестница',
 'Церковь во имя Всемилостивейшего Спаса',
 'Храм Живоначальной Троицы',
 'Церковь в честь Преображения Господня',
 'Музей науки «Нижегородская радиолaborатория»',
 'Крестовоздвиженский собор', 'Губернаторский сад', 'М.
 Горькому',
 'Кладовая башня', 'Спасо-преображенский собор',
 'Церковь во имя Всемирлостивейшего Спаса',
 'Церковь во имя Святого Пророка Божия Илии', 'Зачатьевская
 башня',
 'Здание Удельной конторы', 'Печёрский Вознесенский монастырь',
 'Церковь Александра Невского в Сормово', 'Усадьба Вяхиревых',
 'Дом Ермолаевых – П. Парфёнова', 'Доходный дом Б. Н. Юсупова',
 'Торговый флигель О. Н. Каменевой', 'Дом Е. Трушенинниковой',
 'Дом Виноградовых', 'Дом М. В. Медведева',
 'Здание Волжско-Камского банка', 'Дом И. Н. Соболева',
 'Дом А. И. Фролова', 'Казанская церковь (Кстово)',
 'Усадьба Е. И. Богоявленской', 'Усадьба Киршбаумов',
 'Дом А. К. Фомина', 'Дом культуры ГУВД (Нижний Новгород)',
 'Церковь Никиты Великомученика', 'Авиамеханический колледж',
 'Собор Успения Пресвятой Богородицы', 'Троицкая церковь',
 'Дом-музей Столетовых', 'Исторический музей',
 'Князь-Владимирская церковь',

'Владимирский академический областной драматический театр',
 'Здание Присутственных мест. "Палаты"',
 'Бывшая водонапорная башня',
 'Костел Святого Розария Пресвятой Девы Марии Римско-
 Католической Церкви',
 'Дмитриевский собор',
 'Богородице-Рождественский мужской монастырь',
 'Успенский Княгинин монастырь', 'Золотые ворота',
 'Владимирский централ',
 'Владимирский государственный гуманитарный университет',
 'Владимирское духовное училище',
 'Белокаменные памятники Владимира и Суздаля',
 'Владимирский планетарий', 'Храм Казанской иконы Божией
 Матери',
 'Здание городской думы', 'Музей ложки',
 'Князь-Владимирское кладбище', 'Владимирская митрополия',
 'Владими́ро-Сузда́льский музей-заповедник (центральный офис)',
 'Владимирская духовная семинария', 'église arménienne de
 Vladimir',
 'Памятник природы "Лесной парк Дружба"',
 'Cerkiew Narodzenia Pańskiego w Jarosławiu',
 'Cerkiew Włodzimierskiej Ikony Matki Bożej w Jarosławiu',
 'Cerkiew św. Dymitra Sołtuńskiego w Jarosławiu',
 'Автомеханический техникум', 'Беседка на Волжской набережной',
 'Богоявленская башня', 'Больница имени Н. В. Соловьёва',
 'Бутусовский посёлок', 'Вечный огонь (Ярославль)',
 'Владимирская церковь', 'Власьевская (Знаменская) башня',
 'Власьевский сквер', 'Воинское мемориальное кладбище',
 'Волжская башня', 'Г.М. Димитрову', 'Городам побратимам',
 'Гостиный двор (Ярославль)', 'Губернаторский дом (Ярославль)',
 'Демидовский сад', 'Демидовский столп (Ярославль)',
 'Дом Болконского', 'Дом муз', 'Знаменская церковь',
 'Исторический центр Ярославля', 'Казанский женский монастырь',
 'Кирило-Афанасиевский мужской монастырь',
 'Колокольня Никитского прихода', 'Крестоборогодская церковь',
 'Леонтьевская церковь', 'Леонтьевское кладбище', 'М.А.
 Богданович',
 'М.В. Фрунзе', 'Мемориальный дом-музей М. Богдановича',
 'Михайловские курганы', 'Мост Волжские Ворота',
 'Музей Боевой Славы', 'Музей древнерусского искусства',
 'Музей зарубежного искусства', 'Музей истории города
 Ярославля',
 'Музыка и время', 'Н.А. Некрасову', 'Николая Чудотворца',
 'Николо-Сковородский монастырь', 'Николо-Тропинская церковь',
 'Памятник 1000-летию Ярославля',
 'Памятник жертвам белогвардейского мятежа',
 'Памятник Леониду Николаевичу Трефолеву',
 'Парк 1000-летия Ярославля', 'Парк на Стрелке',
 'Петровский монастырь (Ярославль)', 'Петропавловский парк',


```

        'Приход Воздвижения Святого Креста (Ярославль)',
        'Российский государственный академический театр драмы имени
Федора Волкова',
        'Спасо-Афанасиевский Ярославский монастырь',
        'Спасо-Преображенский монастырь', 'Спасо-Преображенский собор',
        'Сретенский храм',
        'Старообрядческий храм Иоанна\ха0Златоуста в Коровниках',
        'Тимерёвский археологический комплекс',
        'Толгский Ярославский в честь Введения во храм Пресвятой
Богородицы монастырь',
        'Успенский собор', 'Ф. Г. Волкову', 'Фёдор Иванович Толбухин',
        'Федоровский собор', 'Храм Архангела Михаила',
        'Храм Великомученика Никиты', 'Храм Зосимы и Савватия',
        'Храм Петра и Павла', 'храм Ярославской иконы Божией Матери',
        'Церковь Благовещения', 'Церковь Богоявления Господня',
        'Церковь Вознесения Господня', 'Церковь Ильи Пророка',
        'церковь Иоанна Предтечи в Толчкове', 'церковь Михаила
Архангела',
        'Церковь Николая Надеина', 'церковь Николая Чудотворца',
        'Церковь Николы в Меленках', 'церковь Николы Рублёного',
        'Церковь Рождества Христова',
        'Церковь Софии Премудрости Божией в Савино',
        'Церковь Спаса на городу', 'Цирк', 'Часовня Александра
Невского',
        'часовня Казанской Богоматери', 'Школа № 33 имени Карла
Маркса',
        'Ярославская соборная мечеть',
        'Ярославский государственный театр юного зрителя им. В. С.
Розова',
        'Ярославский дворец молодёжи', 'Ярославский зоопарк',
        'Ярославский камерный театр', 'Ярославский музей-заповедник',
        'Ярославский филиал Военно-космической академии имени А. Ф.
Можайского',
        'Ярославский художественный музей (Губернаторский дом)',
        'Ярославское художественное училище'], dtype=object)

```

Типы объектов по содержимому Name

Выделим внутренние типы объектов

Сделаем анализ поля Name с помощью LLM:

Промпт: Выдели все объекты в городе наиболее встречаемые. Например, дом, церковь, башня и т.д.

Выведи в виде списка: ['дома', 'соборы', ...]

```

type_freq = ['дом', 'церковь', 'музей', 'памятник', 'собор', 'башня',
'театр',
            'улица', 'парк', 'монастырь', 'усадьба', 'университет',

```

```
'мост',  
        'храм']
```

Также выделим редкие объекты но выжные для города

```
type_rare = ['сад', 'сквер', 'гостиница', 'стадион', 'зоопарк',  
            'цирк',  
            'планетарий']  
  
import matplotlib.pyplot as plt  
import re  
  
data = df['Name'].unique()  
  
# Функция для поиска вхождений  
def count_types_in_data(data, types_list):  
    counts = {type_name: 0 for type_name in types_list}  
    counts["прочее"] = 0  
  
    for item in data:  
        item_lower = item.lower()  
        for type_name in types_list:  
            # Ищем точные вхождения слов  
            if re.search(r'\b' + re.escape(type_name) + r'\b',  
item_lower):  
                counts[type_name] += 1  
            else:  
                counts["прочее"] += 1  
  
    return counts  
  
# Подсчитываем вхождения  
type_counts = count_types_in_data(data, type_freq)  
etc_count = type_counts.pop('прочее')  
  
# Сортируем по убыванию частоты  
sorted_types = sorted(type_counts.items(), key=lambda x: x[1],  
reverse=True)  
types, counts = zip(*sorted_types)  
  
# Строим график  
plt.figure(figsize=(6, 4))  
bars = plt.bar(types, counts, color='skyblue', edgecolor='navy',  
alpha=0.7)  
  
# Добавляем значения на столбцы  
for bar, count in zip(bars, counts):  
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() +
```

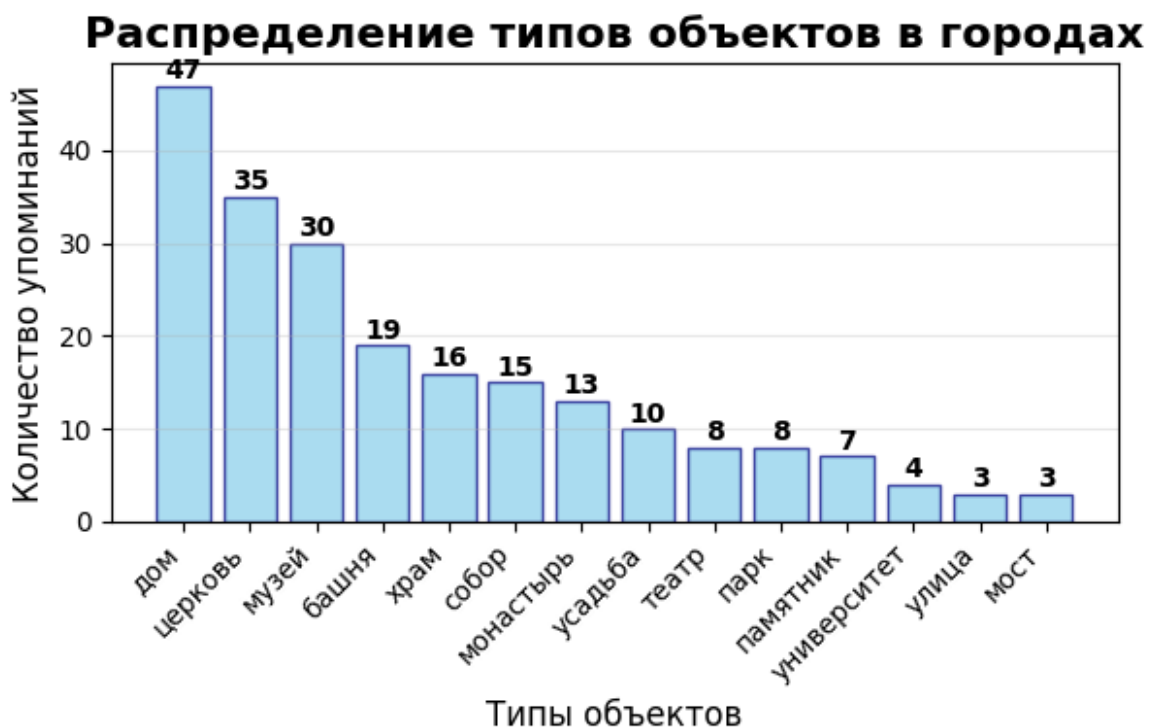
```

0.1,
    str(count), ha='center', va='bottom', fontweight='bold')

plt.title('Распределение типов объектов в городах', fontsize=16,
          fontweight='bold')
plt.xlabel('Типы объектов', fontsize=12)
plt.ylabel('Количество упоминаний', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()

plt.show()

```



Считаем эти объекты идентифицированные по типам, и это один из критериев чистоты

Поле WikiData

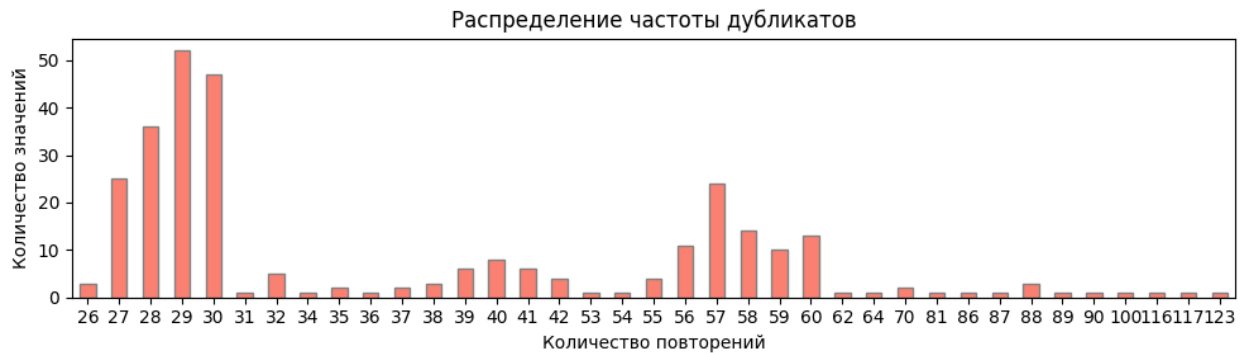
```

df.WikiData.unique().shape[0]

296

frequency_distribution(df['WikiData'])

```

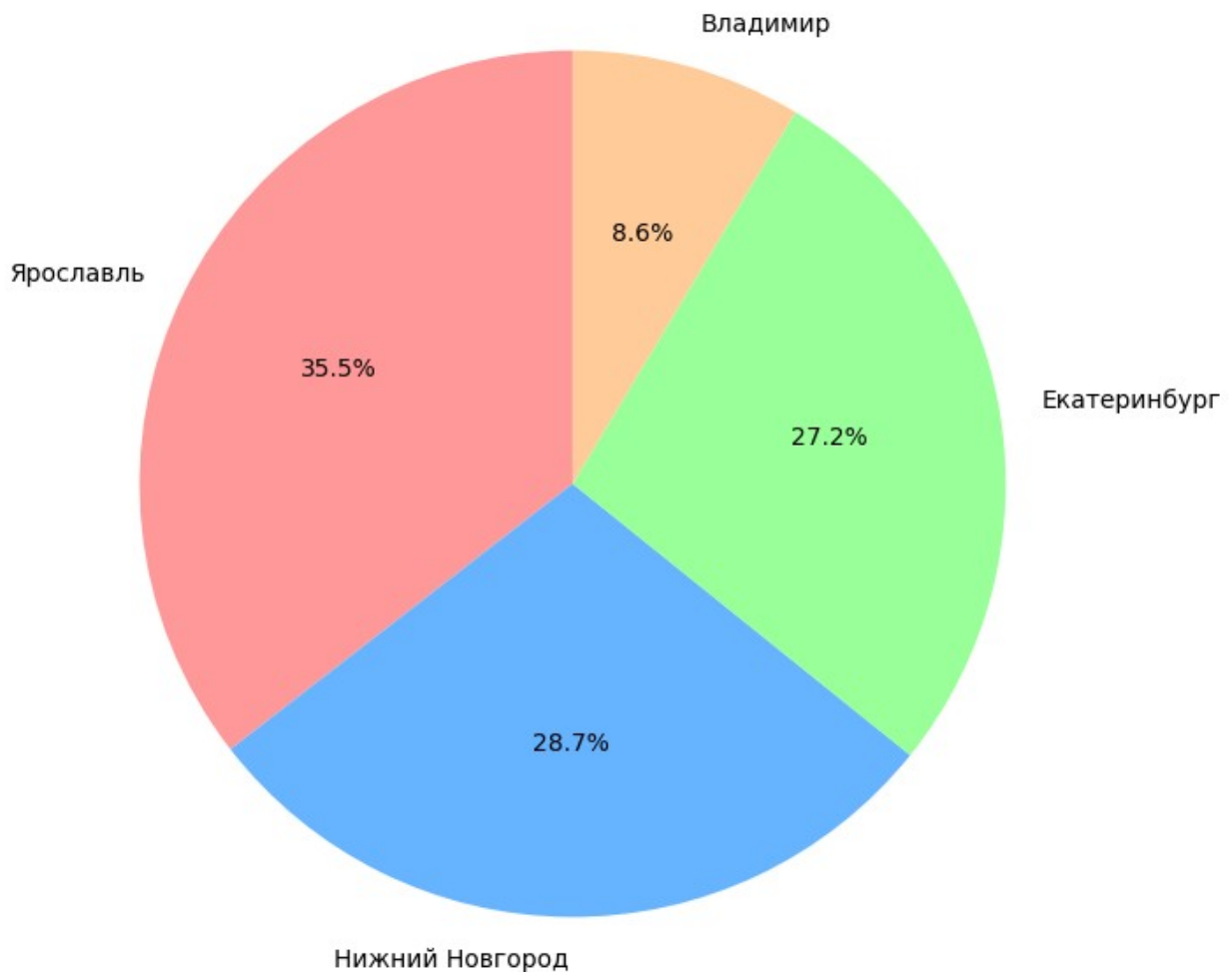


Индекс довольно много продублирован. Считаем что чистых данных (объектов) около 300

Поле City

```
df.City.unique().shape[0]
4
df.City.unique()
array(['Екатеринбург', 'Нижний Новгород', 'Владимир', 'Ярославль'],
      dtype=object)
plt.figure(figsize=(8, 8))
city_counts = df['City'].value_counts()
colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99']
plt.pie(city_counts.values, labels=city_counts.index, autopct='%1.1f%%',
        colors=colors, startangle=90)
plt.title('Распределение записей по городам', fontsize=14,
fontweight='bold')
plt.show()
```

Распределение записей по городам



Всего 4 города с перекосом распределения

Поле description

```
df.description.unique().shape[0]
```

245

Всего по всему датасету 245 уникальных описаний

Это и есть конкретные достопримечательности во всех городах

```
df.description.unique()
```

```
array(['спорткомплекс в Екатеринбурге', 'здание в Екатеринбурге',  
      'здание и достопримечательность в Екатеринбурге',
```

'Дом Е.Д. Скрябиной', 'Музей',
'Мемориальный дом-музей П. П. Бажова', 'кинотеатр в
Екатеринбурге',
'парк в Екатеринбурге', 'Храм Вознесения Господня
(Екатеринбург)',
'военно-учебное заведение в Екатеринбурге',
'Троицкий собор (Екатеринбург)',
'Литературно-мемориальный дом-музей Д. Н. Мамина-Сибиряка',
'музей в Екатеринбурге', 'Памятник в Екатеринбурге',
'Успенский собор (Екатеринбург)',
'мост через Исеть в Екатеринбурге',
'православный собор в Екатеринбурге', 'гостиница в
Екатеринбурге',
'высшее учебное заведение в Екатеринбурге',
'стадион в Екатеринбурге', 'Преображенская церковь',
'Второй Дом Советов (Екатеринбург)', 'сооружение в
Екатеринбурге',
'оперный театр в Екатеринбурге', 'памятник в Екатеринбурге',
'геологический и археологический памятник природы в
Екатеринбурге',
'Дом контор', 'часовня в Екатеринбурге',
'комплекс зданий в стиле конструктивизма',
'церковь в Екатеринбурге',
'профессиональное учебное заведение Екатеринбурга',
'главная площадь Екатеринбурга',
'Музей кукол и детской книги «Страна чудес»',
'здание в Железнодорожном районе Екатеринбурга',
'Екатеринбургское художественное училище им. И. Д. Шадра',
'концертный зал в Екатеринбурге',
'учебное заведение в Екатеринбурге',
'Музей истории плодового садоводства Среднего Урала',
'Литературно-мемориальный дом-музей Ф. М. Решетникова',
'православный храм в Екатеринбурге', 'Ново-Тихвинский
монастырь',
'Дом купца Чувильдина',
'Свердловский областной краеведческий музей',
'музыкальный театр в Екатеринбурге', 'Дом актёра
(Екатеринбург)',
'улица в Екатеринбурге', 'Мемориальный комплекс в
Екатеринбурге',
'металлургический завод (XVIII век)',
'предприятие в Екатеринбурге', 'Усадьба Железнова',
'Усадьба ротмистра Переяславцева',
'Комплекс штаба военного округа', 'университет в
Екатеринбурге',
'бывшее название университета в Екатеринбурге',
'городская площадь в Екатеринбурге', 'Усадьба Тарасова',
'Здание пансиона и церкви Святой Магдалины при первой женской
гимназии',

'Дом Бревновых',
 'деревянный особняк начала XX века на Вознесенской горке в
 Екатеринбурге',
 'историческое здание в Екатеринбурге',
 'Крестовоздвиженская церковь (Екатеринбург)',
 'Литературный музей в Екатеринбурге', 'Библиотека в
 Екатеринбурге',
 'Дом Гайдара', 'Дом дворянки Е. О. Селивановой',
 'Дом В. И. Смирнова',
 'Памятник В. И. Ленину на площади Ленина (Нижний Новгород)',
 'круглая башня Нижегородского кремля',
 'проездная башня Нижегородского кремля',
 'Жилой дом горьковского архитектора А. А. Яковлева',
 'Старейший театр Нижнего Новгорода',
 'Крепость в центре Нижнего Новгорода',
 'Нижегородская соборная мечеть',
 'музей в Нижнем Новгороде (Россия)', 'Автозаводский парк',
 'православный собор в Нижнем Новгороде', 'православный храм',
 'Церковь Рождества Богородицы в селе Гнилицы',
 'Спасо-Преображенская церковь (Карповка)',
 'административное здание в Нижнем Новгороде, Россия',
 'Парк Кулибина', 'Воскресенская церковь (Нижний Новгород)',
 'Церковь Сергия Радонежского',
 'Храм Святителей Московских (Нижний Новгород)',
 'Торговый дом Скоба',
 'историческое здание в Нижнем Новгороде, Россия',
 'проездная башня Нижегородского кремля, один из символов
 Нижнего Новгорода',
 'Лицей №82 "Школа баррикад"',
 'Мемориал «Горьковчане – фронту»',
 'один из старейших музеев России, крупнейший музей
 изобразительного искусства в Нижегородской области.',
 'Церковь в Нижнем Новгороде',
 'Музей-квартира А. М. Горького (Нижний Новгород)',
 'монастырь в Нижнем Новгороде',
 'чебное заведение среднего профессионального образования в
 Нижнем Новгороде, готовящее специалистов водного транспорта.С 2005
 года – филиал Волжской государственной академии водного транспорта,
 которая сейчас называется ВГУВТ.',
 'парк культуры и отдыха в Нижнем Новгороде.',
 'Монумент героям Волжской военной флотилии (Нижний Новгород)',
 'здание в Нижегородском кремле, Россия',
 'Памятник героям и жертвам революции 1905 года',
 'Домик Петра I (Нижний Новгород)',
 'Государственный банк (Нижний Новгород)',
 'Усадьба С. М. Рукавишникова',
 'Нижегородский государственный академический театр оперы и
 балета имени А. С. Пушкина',
 'памятник в Нижнем Новгороде, Россия',

'Церковь Жен-Мироносиц (Нижний Новгород)', 'Нижегородский острог',
'Сормовский парк', 'Блиновский пассаж',
'Памятник градостроительства и архитектуры',
'Общежитие института инженеров водного транспорта с кинотеатром «Рекорд»',
'Нижегородская государственная консерватория имени М. И. Глинки',
'Церковь Рождества Иоанна Предтечи на Торгу',
'единственный в мире музей литературного критика и писателя Николая Александровича Добролюбова, расположенный в Нижнем Новгороде.',
'памятник архитектуры Нижнего Новгорода, корпус Художественного музея',
'Собор на Нижегородской ярмарке', 'башня Нижегородского кремля',
'многоцелевой стадион в Нижнем Новгороде, Россия',
'Крестовоздвиженский монастырь (Нижний Новгород)',
'Дом П. Е. Кубаревой',
'лестница на Волжском откосе в Нижнем Новгороде',
'Храм Всемилостивого Спаса (Нижний Новгород)',
'соборный храм в Нижегородском районе Нижнего Новгорода',
'Спасо-Преображенская церковь (Старые Печёры)',
'Нижегородская радиолaborатория',
'парк в Нижегородском кремле, Россия', 'памятник в России',
'православный храм в Нижнем Новгороде',
'церковь в Нижнем Новгороде', 'Здание Удельной конторы',
'Церковь Александра Невского в Сормово', 'Усадьба Вяхиревых',
'Дом Ермолаевых – П. Парфёнова', 'Доходный дом Б. Н. Юсупова',
'Торговый флигель О. Н. Каменевой', 'Дом Е. Трушенинниковой',
'Дом Виноградовых', 'Дом М. В. Медведева',
'Здание Волжско-Камского банка', 'Дом И. Н. Соболева',
'Дом А. И. Фролова',
'православный храм в г. Кстово, Нижегородская область',
'Усадьба Е. И. Богоявленской', 'Усадьба Киршбаумов',
'памятник',
'Дом культуры ГУВД (Нижний Новгород)', 'Никитская церковь',
'Владимирский авиамеханический колледж',
'православный храм во Владимире, памятник белокаменного зодчества домонгольской Руси',
'Троицкая церковь',
'Мемориальный музей в городе Владимире, посвящённый жизни и деятельности выдающегося учёного-физика А. Г. Столетова и генерала от инфантерии Н. Г. Столетова, сыгравшего видную роль в освобождении Болгарии от турецкого ига',
'Музей во Владимире, Россия',
'православный кладбищенский храм во Владимире',
'театр города Владимира', 'Здание присутственных мест',
'достопримечательность во Владимире',

'католический храм в городе Владимир',
'Православный храм во Владимире',
'Богородице-Рождественский монастырь (Владимир)',
'Успенский Княгинин монастырь', 'памятник XII века во Владимире',
'Тюрьма в городе Владимир',
'Владимирский государственный гуманитарный университет',
'Владимирское духовное училище',
'Белокаменные памятники Владимира и Суздаля',
'Владимирский планетарий', 'Храм Казанской иконы Божией Матери',
'Здание городской думы', 'Музей ложки', 'некрополь во Владимире',
'Владимирский кремль', 'Владими́ро-Сузда́льский музей-заповедник',
'Владимирская духовная семинария',
'Армянская церковь в г. Владимир России', 'Лесной парк «Дружба»',
'Православный храм в Ярославле',
'Архитектурный ансамбль в Ярославле',
'Церковь великомученика Димитрия Солунского (Смоленская) (Ярославль)',
'образовательное учреждение в Ярославле',
'Беседка на Волжской набережной',
'ГАОУЗ ЯО Клиническая больница скорой медицинской помощи имени Н. В. Соловьёва',
'Бутусовский посёлок', 'Вечный огонь (Ярославль)',
'Владимирская церковь на Божедомке',
'Крепостная башня в Ярославле', 'Городской сад в Ярославле',
'Воинское мемориальное кладбище (Ярославль)', 'Башня в Ярославле',
'Памятник Георгию Димитрову (Ярославль)',
'Памятный знак городам-побратимам (Ярославль)',
'Торговые ряды в Ярославле', 'Губернаторский дом (Ярославль)',
'Демидовский сад', 'Демидовский столп (Ярославль)',
'Дом Болконского', 'Дом муз', 'Исторический центр Ярославля',
'Казанский монастырь (Ярославль)', 'Монастырь в Ярославле',
'приход в Ярославле', 'Крестобогородская церковь',
'кладбище в Ярославле',
'белорусский поэт, публицист, литературовед, переводчик.',
'Памятник Михаилу Фрунзе (Ярославль)',
'Мемориальный дом-музей М. Богдановича', 'Михайловские курганы',
'мост в Ярославле', 'Музей в Ярославле',
'Митрополичьи палаты (Ярославль)',
'художественный музей в Ярославле',
'Музей истории города Ярославля', 'Частный музей',
'Памятник в Ярославле', 'храм в Ярославле',
'Николо-Сковородский монастырь', 'Церковь в Ярославле',

```

'Памятник жертвам белогвардейского мятежа',
'Памятник Леониду Трефолеву (Ярославль)', 'Парк в Ярославле',
'Петровский монастырь (Ярославль)', 'Петропавловский парк',
'Приход Воздвижения Святого Креста (Ярославль)',
'Российский академический театр драмы им. Ф. Волкова',
'монастырь в Ярославле', 'Храм в Ярославле',
'Ярославский государственный театр кукол',
'Тимерёвский археологический комплекс',
'Женский православный монастырь Ярославской епархии',
'кафедральный собор в Ярославле',
'Памятник Фёдору Толбухину (Ярославль)',
'Церковь Михаила Архангела',
'Храм в честь Ярославской иконы Божией Матери',
'Благовещенско-Вознесенский приход',
'Вознесенско-Сретенский приход', 'Церковь Рождества Христова',
'Ярославский государственный цирк', 'часовня в Ярославле',
'Часовня Казанской Богоматери (Ярославль)',
'Школа № 33 имени Карла Маркса', 'Ярославская соборная мечеть',
'Театр в Ярославле', 'Ярославский дворец молодёжи',
'Ярославский зоопарк', 'Ярославский камерный театр',
'Ярославский музей-заповедник', 'высшее военное училище в
России',
'Ярославское художественное училище'], dtype=object)

```

Построим распределение типов достопримечательностей по всему датасету

```

import re
import matplotlib.pyplot as plt

data = df['description'].unique()

# Функция для поиска вхождений
def count_types_in_data(data, types_list):
    counts = {type_name: 0 for type_name in types_list}

    for item in data:
        item_lower = item.lower()
        for type_name in types_list:
            # Ищем точные вхождения слов
            if re.search(r'\b' + re.escape(type_name) + r'\b',
item_lower):
                counts[type_name] += 1

    return counts

# Подсчитываем вхождения
type_counts = count_types_in_data(data, type_freq)

```

```

# Сортируем по убыванию частоты
sorted_types = sorted(type_counts.items(), key=lambda x: x[1],
reverse=True)
types, counts = zip(*sorted_types)

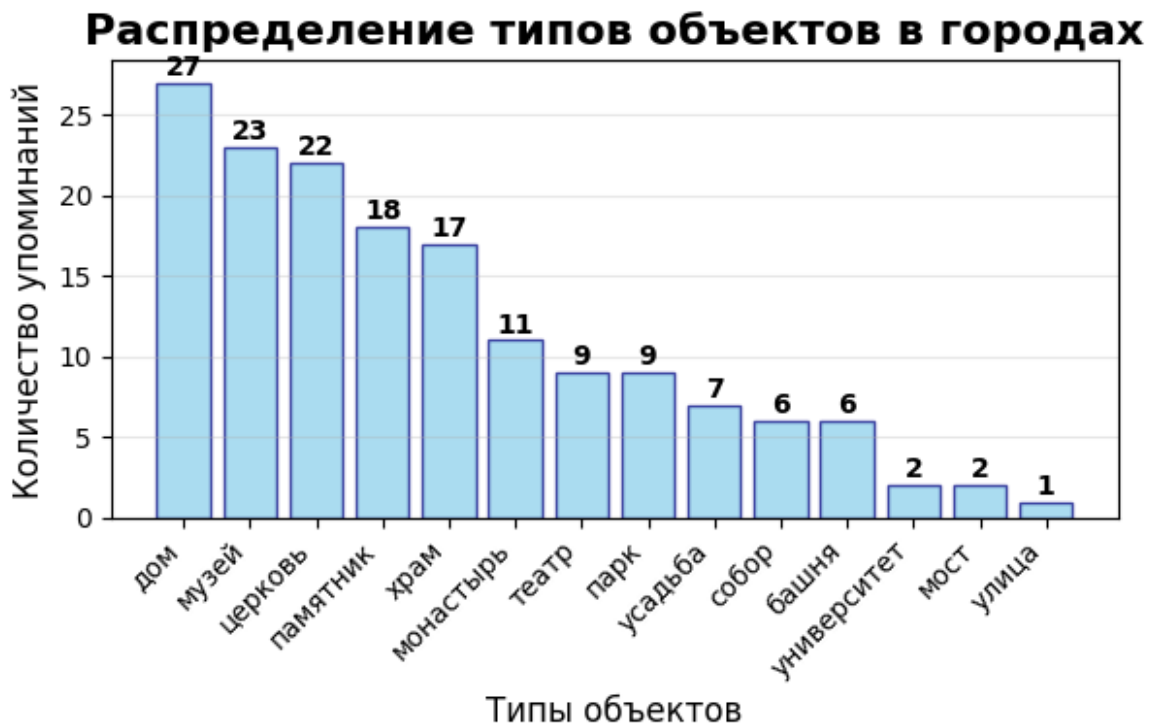
# Строим график
plt.figure(figsize=(6, 4))
bars = plt.bar(types, counts, color='skyblue', edgecolor='navy',
alpha=0.7)

# Добавляем значения на столбцы
for bar, count in zip(bars, counts):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() +
0.1,
            str(count), ha='center', va='bottom', fontweight='bold')

plt.title('Распределение типов объектов в городах', fontsize=16,
        fontweight='bold')
plt.xlabel('Типы объектов', fontsize=12)
plt.ylabel('Количество упоминаний', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()

plt.show()

```



Смысловой анализ описания привязанного к городу

Выведем все описания привязанные к городу

```
def build_dichotomy_scheme_detailed(df):
    scheme = {}

    for city in df['City'].unique():
        city_data = df[df['City'] == city]
        descriptions = city_data['description'].unique()
        scheme[city] = {}

        for desc in descriptions:
            elements = city_data[city_data['description'] == desc][
                'Name'].tolist()
            scheme[city][desc] = len(elements)

    return scheme

def print_detailed_scheme(scheme):
    for city in scheme:
        print(f"\n### **{city}**")
        print("` ` ` `")
        descriptions = list(scheme[city].keys())
        for i, desc in enumerate(descriptions):
            num = scheme[city][desc]
            if i == len(descriptions) - 1:
                print(f"└─ {desc} {num}")
            else:
                print(f"└─ {desc} {num}")
        print("` ` ` `")

# Использование
scheme = build_dichotomy_scheme_detailed(df)
print_detailed_scheme(scheme)
```

```
### **Екатеринбург**
```

```
` ` ` `
```

```
└─ спорткомплекс в Екатеринбурге 41
└─ здание в Екатеринбурге 287
└─ здание и достопримечательность в Екатеринбурге 55
└─ Дом Е.Д. Скрябиной 28
└─ Музей 27
└─ Мемориальный дом-музей П. П. Бажова 29
└─ кинотеатр в Екатеринбурге 40
└─ парк в Екатеринбурге 31
└─ Храм Вознесения Господня (Екатеринбург) 57
└─ военно-учебное заведение в Екатеринбурге 32
```

- Троицкий собор (Екатеринбург) 30
- Литературно-мемориальный дом-музей Д. Н. Мамина-Сибиряка 29
- музей в Екатеринбурге 285
- Памятник в Екатеринбурге 116
- Успенский собор (Екатеринбург) 29
- мост через Исеть в Екатеринбурге 69
- православный собор в Екатеринбурге 27
- гостиница в Екатеринбурге 29
- высшее учебное заведение в Екатеринбурге 58
- стадион в Екатеринбурге 40
- Преображенская церковь 57
- Второй Дом Советов (Екатеринбург) 30
- сооружение в Екатеринбурге 78
- оперный театр в Екатеринбурге 40
- памятник в Екатеринбурге 204
- геологический и археологический памятник природы в Екатеринбурге 28
- Дом контор 57
- часовня в Екатеринбурге 29
- комплекс зданий в стиле конструктивизма 34
- церковь в Екатеринбурге 28
- профессиональное учебное заведение Екатеринбурга 29
- главная площадь Екатеринбурга 41
- Музей кукол и детской книги «Страна чудес» 30
- здание в Железнодорожном районе Екатеринбурга 29
- Екатеринбургское художественное училище им. И. Д. Шадра 29
- концертный зал в Екатеринбурге 70
- учебное заведение в Екатеринбурге 28
- Музей истории плодового садоводства Среднего Урала 30
- Литературно-мемориальный дом-музей Ф. М. Решетникова 30
- православный храм в Екатеринбурге 100
- Ново-Тихвинский монастырь 28
- Дом купца Чувильдина 60
- Свердловский областной краеведческий музей 30
- музыкальный театр в Екатеринбурге 38
- Дом актёра (Екатеринбург) 30
- улица в Екатеринбурге 179
- Мемориальный комплекс в Екатеринбурге 29
- металлургический завод (XVIII век) 30
- предприятие в Екатеринбурге 29
- Усадьба Железнова 30
- Усадьба ротмистра Переяславцева 28
- Комплекс штаба военного округа 28
- университет в Екатеринбурге 28
- бывшее название университета в Екатеринбурге 28
- городская площадь в Екатеринбурге 29
- Усадьба Тарасова 60
- Здание пансиона и церкви Святой Магдалины при первой женской гимназии 29

- Дом Бревновых 27
- деревянный особняк начала XX века на Вознесенской горке в
Екатеринбурге 30
- историческое здание в Екатеринбурге 30
- Крестовоздвиженская церковь (Екатеринбург) 29
- Литературный музей в Екатеринбурге 28
- Библиотека в Екатеринбурге 27
- Дом Гайдара 29
- Дом дворянки Е. О. Селивановой 29

Нижний Новгород

- Дом В. И. Смирнова 29
- Памятник В. И. Ленину на площади Ленина (Нижний Новгород) 29
- круглая башня Нижегородского кремля 288
- проездная башня Нижегородского кремля 145
- Жилой дом горьковского архитектора А. А. Яковлева 27
- Старейший театр Нижнего Новгорода 29
- Крепость в центре Нижнего Новгорода 56
- Нижегородская соборная мечеть 29
- музей в Нижнем Новгороде (Россия) 28
- Автозаводский парк 29
- православный собор в Нижнем Новгороде 37
- православный храм 32
- Церковь Рождества Богородицы в селе Гнилицы 57
- Спасо-Преображенская церковь (Карповка) 29
- административное здание в Нижнем Новгороде, Россия 114
- Парк Кулибина 30
- Воскресенская церковь (Нижний Новгород) 29
- Церковь Сергия Радонежского 58
- Храм Святителей Московских (Нижний Новгород) 30
- Торговый дом Скоба 30
- историческое здание в Нижнем Новгороде, Россия 28
- проездная башня Нижегородского кремля, один из символов Нижнего
Новгорода 64
- Лицей № 82 "Школа баррикад" 28
- Мемориал «Горьковчане — фронту» 27
- один из старейших музеев России, крупнейший музей изобразительного
искусства в Нижегородской области. 29
- Церковь в Нижнем Новгороде 39
- Музей-квартира А. М. Горького (Нижний Новгород) 60
- монастырь в Нижнем Новгороде 128
- учебное заведение среднего профессионального образования в Нижнем
Новгороде, готовящее специалистов водного транспорта. С 2005 года —
филиал Волжской государственной академии водного транспорта, которая
сейчас называется ВГУВТ. 30
- парк культуры и отдыха в Нижнем Новгороде. 29
- Монумент героям Волжской военной флотилии (Нижний Новгород) 30

- здание в Нижегородском кремле, Россия 30
- Памятник героям и жертвам революции 1905 года 29
- Домик Петра I (Нижний Новгород) 40
- Государственный банк (Нижний Новгород) 27
- Усадьба С. М. Рукавишникова 39
- Нижегородский государственный академический театр оперы и балета имени А. С. Пушкина 38
- памятник в Нижнем Новгороде, Россия 29
- Церковь Жен-Мироносиц (Нижний Новгород) 59
- Нижегородский острог 28
- Сормовский парк 40
- Блиновский пассаж 29
- Памятник градостроительства и архитектуры 28
- Общежитие института инженеров водного транспорта с кинотеатром «Рекорд» 27
- Нижегородская государственная консерватория имени М. И. Глинки 27
- Церковь Рождества Иоанна Предтечи на Торгу 29
- единственный в мире музей литературного критика и писателя Николая Александровича Добролюбова, расположенный в Нижнем Новгороде. 58
- памятник архитектуры Нижнего Новгорода, корпус Художественного музея 32
- Собор на Нижегородской ярмарке 27
- башня Нижегородского кремля 58
- многоцелевой стадион в Нижнем Новгороде, Россия 32
- Крестовоздвиженский монастырь (Нижний Новгород) 56
- Дом П. Е. Кубаревой 30
- лестница на Волжском откосе в Нижнем Новгороде 123
- Храм Всемилостивого Спаса (Нижний Новгород) 117
- соборный храм в Нижегородском районе Нижнего Новгорода 60
- Спасо-Преображенская церковь (Старые Печёры) 29
- Нижегородская радиолaborатория 29
- парк в Нижегородском кремле, Россия 30
- памятник в России 29
- православный храм в Нижнем Новгороде 30
- церковь в Нижнем Новгороде 58
- Здание Удельной конторы 28
- Церковь Александра Невского в Сормово 59
- Усадьба Вяхиревых 28
- Дом Ермолаевых – П. Парфёнова 30
- Доходный дом Б. Н. Юсупова 29
- Торговый флигель О. Н. Каменевой 30
- Дом Е. Трушенинниковой 28
- Дом Виноградовых 28
- Дом М. В. Медведева 29
- Здание Волжско-Камского банка 58
- Дом И. Н. Соболева 29
- Дом А. И. Фролова 27
- православный храм в г. Кстово, Нижегородская область 30
- Усадьба Е. И. Богоявленской 28

- |— Усадьба Киршбаумов 29
- |— памятник 28
- |— Дом культуры ГУВД (Нижний Новгород) 27
- \\

Владимир

- |— Никитская церковь 57
- |— Владимирский авиамеханический колледж 30
- |— православный храм во Владимире, памятник белокаменного зодчества домонгольской Руси 58
- |— Троицкая церковь 41
- |— Мемориальный музей в городе Владимире, посвящённый жизни и деятельности выдающегося учёного-физика А. Г. Столетова и генерала от инфантерии Н. Г. Столетова, сыгравшего видную роль в освобождении Болгарии от турецкого ига 29
- |— Музей во Владимире, Россия 42
- |— православный кладбищенский храм во Владимире 56
- |— театр города Владимира 28
- |— Здание присутственных мест 27
- |— достопримечательность во Владимире 55
- |— католический храм в городе Владимир 30
- |— Православный храм во Владимире 59
- |— Богородице-Рождественский монастырь (Владимир) 29
- |— Успенский Княгинин монастырь 27
- |— памятник XII века во Владимире 57
- |— Тюрьма в городе Владимир 29
- |— Владимирский государственный гуманитарный университет 27
- |— Владимирское духовное училище 30
- |— Белокаменные памятники Владимира и Суздаля 29
- |— Владимирский планетарий 30
- |— Храм Казанской иконы Божией Матери 30
- |— Здание городской думы 36
- |— Музей ложки 30
- |— некрополь во Владимире 29
- |— Владимирский кремль 28
- |— Владимиро-Суздальский музей-заповедник 28
- |— Владимирская духовная семинария 30
- |— Армянская церковь в г. Владимир России 29
- |— Лесной парк «Дружба» 29
- \\

Ярославль

- |— Православный храм в Ярославле 605
- |— Архитектурный ансамбль в Ярославле 59
- |— Церковь великомученика Димитрия Солунского (Смоленская) (Ярославль) 57
- |— образовательное учреждение в Ярославле 56

- |— Беседка на Волжской набережной 57
- |— ГАУЗ ЯО Клиническая больница скорой медицинской помощи имени Н. В. Соловьёва 30
- |— Бутусовский посёлок 28
- |— Вечный огонь (Ярославль) 56
- |— Владимирская церковь на Божедомке 56
- |— Крепостная башня в Ярославле 88
- |— Городской сад в Ярославле 28
- |— Воинское мемориальное кладбище (Ярославль) 29
- |— Башня в Ярославле 57
- |— Памятник Георгию Димитрову (Ярославль) 53
- |— Памятный знак городам-побратимам (Ярославль) 29
- |— Торговые ряды в Ярославле 30
- |— Губернаторский дом (Ярославль) 58
- |— Демидовский сад 57
- |— Демидовский столп (Ярославль) 59
- |— Дом Болконского 26
- |— Дом муз 60
- |— Исторический центр Ярославля 57
- |— Казанский монастырь (Ярославль) 37
- |— Монастырь в Ярославле 57
- |— приход в Ярославле 116
- |— Крестобогородская церковь 30
- |— кладбище в Ярославле 86
- |— белорусский поэт, публицист, литературовед, переводчик. 29
- |— Памятник Михаилу Фрунзе (Ярославль) 60
- |— Мемориальный дом-музей М. Богдановича 28
- |— Михайловские курганы 27
- |— мост в Ярославле 29
- |— Музей в Ярославле 35
- |— Митрополичьи палаты (Ярославль) 28
- |— художественный музей в Ярославле 88
- |— Музей истории города Ярославля 59
- |— Частный музей 30
- |— Памятник в Ярославле 177
- |— храм в Ярославле 171
- |— Николо-Сковородский монастырь 57
- |— Церковь в Ярославле 85
- |— Памятник жертвам белогвардейского мятежа 58
- |— Памятник Леониду Трефолеву (Ярославль) 56
- |— Парк в Ярославле 115
- |— Петровский монастырь (Ярославль) 28
- |— Петропавловский парк 41
- |— Приход Воздвижения Святого Креста (Ярославль) 57
- |— Российский академический театр драмы им. Ф. Волкова 57
- |— монастырь в Ярославле 81
- |— Храм в Ярославле 29
- |— Ярославский государственный театр кукол 30
- |— Тимерёвский археологический комплекс 26

- Женский православный монастырь Ярославской епархии 57
- кафедральный собор в Ярославле 41
- Памятник Фёдору Толбухину (Ярославль) 58
- Церковь Михаила Архангела 42
- Храм в честь Ярославской иконы Божией Матери 29
- Благовещенско-Вознесенский приход 56
- Вознесенско-Сретенский приход 59
- Церковь Рождества Христова 42
- Ярославский государственный цирк 39
- часовня в Ярославле 38
- Часовня Казанской Богоматери (Ярославль) 40
- Школа № 33 имени Карла Маркса 60
- Ярославская соборная мечеть 28
- Театр в Ярославле 30
- Ярославский дворец молодёжи 28
- Ярославский зоопарк 57
- Ярославский камерный театр 28
- Ярославский музей-заповедник 27
- высшее военное училище в России 60
- Ярославское художественное училище 27

Списки имеют в составе явные абстрактные названия, которые слабо описывают конкретные достопримечательности. Постоим списки не конкретных объектов для каждого города с помощью LLM.

Промпт: Дан список объектов в городе. Выдели в отдельный список python все не конкретные объекты.

Пример не конкретных объектов в городе:

- здание в Екатеринбурге
- Музей
- Памятник в Екатеринбурге

Пример хорошо идентифицированных объектов в городе :

- Дом Е.Д. Скрябиной
- Мемориальный дом-музей П. П. Бажова

Не включай в список значимые места в городе пример

- спорткомплекс
- гостиница
- мост
- университет

список объектов в городе: ...

Екатеринбург

- спорткомплекс в Екатеринбурге
- здание в Екатеринбурге
- здание и достопримечательность в Екатеринбурге
- Дом Е.Д. Скрябиной
- Музей
- Мемориальный дом-музей П. П. Бажова
- кинотеатр в Екатеринбурге
- парк в Екатеринбурге
- Храм Вознесения Господня (Екатеринбург)
- военно-учебное заведение в Екатеринбурге
- Троицкий собор (Екатеринбург)
- Литературно-мемориальный дом-музей Д. Н. Мамина-Сибиряка
- музей в Екатеринбурге
- Памятник в Екатеринбурге
- Успенский собор (Екатеринбург)
- мост через Исеть в Екатеринбурге
- православный собор в Екатеринбурге
- гостиница в Екатеринбурге
- высшее учебное заведение в Екатеринбурге
- стадион в Екатеринбурге
- Преображенская церковь
- Второй Дом Советов (Екатеринбург)
- сооружение в Екатеринбурге
- оперный театр в Екатеринбурге
- памятник в Екатеринбурге
- геологический и археологический памятник природы в Екатеринбурге
- Дом контор
- часовня в Екатеринбурге
- комплекс зданий в стиле конструктивизма
- церковь в Екатеринбурге
- профессиональное учебное заведение Екатеринбурга
- главная площадь Екатеринбурга
- Музей кукол и детской книги «Страна чудес»
- здание в Железнодорожном районе Екатеринбурга
- Екатеринбургское художественное училище им. И. Д. Шадра
- концертный зал в Екатеринбурге
- учебное заведение в Екатеринбурге
- Музей истории плодового садоводства Среднего Урала
- Литературно-мемориальный дом-музей Ф. М. Решетникова
- православный храм в Екатеринбурге
- Ново-Тихвинский монастырь
- Дом купца Чувильдина
- Свердловский областной краеведческий музей
- музыкальный театр в Екатеринбурге
- Дом актёра (Екатеринбург)
- улица в Екатеринбурге
- Мемориальный комплекс в Екатеринбурге
- металлургический завод (XVIII век)
- предприятие в Екатеринбурге
- Усадьба Железнова


```

        "православный храм в Екатеринбурге",
        "музыкальный театр в Екатеринбурге",
        "улица в Екатеринбурге",
        "Мемориальный комплекс в Екатеринбурге",
        "металлургический завод (XVIII век)",
        "предприятие в Екатеринбурге",
        "университет в Екатеринбурге",
        "бывшее название университета в
Екатеринбурге",
        "городская площадь в Екатеринбурге",
        "деревянный особняк начала XX века на
Вознесенской горке в Екатеринбурге",
        "историческое здание в Екатеринбурге",
        "Литературный музей в Екатеринбурге",
        "Библиотека в Екатеринбурге"}

non_specific_objects_rare_ecat = {"спорткомплекс в Екатеринбурге",
        "парк в Екатеринбурге",
        "военно-учебное заведение в
Екатеринбурге",
        "мост через Исеть в Екатеринбурге",
        "гостиница в Екатеринбурге",
        "стадион в Екатеринбурге",
        "оперный театр в Екатеринбурге",
        "главная площадь Екатеринбурга",
        "концертный зал в Екатеринбурге",
        "музыкальный театр в Екатеринбурге",
        "городская площадь в Екатеринбурге",
}

non_specific_objects_ecat = non_specific_objects_ecat -
non_specific_objects_rare_ecat

```

Нижний Новгород

- Дом В. И. Смирнова
- Памятник В. И. Ленину на площади Ленина (Нижний Новгород)
- круглая башня Нижегородского кремля
- проездная башня Нижегородского кремля
- Жилой дом горьковского архитектора А. А. Яковлева
- Старейший театр Нижнего Новгорода
- Крепость в центре Нижнего Новгорода
- Нижегородская соборная мечеть
- музей в Нижнем Новгороде (Россия)
- Автозаводский парк
- православный собор в Нижнем Новгороде
- православный храм
- Церковь Рождества Богородицы в селе Гнилицы
- Спасо-Преображенская церковь (Карповка)

- административное здание в Нижнем Новгороде, Россия
- Парк Кулибина
- Воскресенская церковь (Нижний Новгород)
- Церковь Сергия Радонежского
- Храм Святителей Московских (Нижний Новгород)
- Торговый дом Скоба
- историческое здание в Нижнем Новгороде, Россия
- проездная башня Нижегородского кремля, один из символов Нижнего Новгорода
- Лицей № 82 "Школа баррикад"
- Мемориал «Горьковчане – фронту»
- один из старейших музеев России, крупнейший музей изобразительного искусства в Нижегородской области.
- Церковь в Нижнем Новгороде
- Музей-квартира А. М. Горького (Нижний Новгород)
- монастырь в Нижнем Новгороде
- учебное заведение среднего профессионального образования в Нижнем Новгороде, готовящее специалистов водного транспорта. С 2005 года – филиал Волжской государственной академии водного транспорта, которая сейчас называется ВГУВТ.
- парк культуры и отдыха в Нижнем Новгороде.
- Монумент героям Волжской военной флотилии (Нижний Новгород)
- здание в Нижегородском кремле, Россия
- Памятник героям и жертвам революции 1905 года
- Домик Петра I (Нижний Новгород)
- Государственный банк (Нижний Новгород)
- Усадьба С. М. Рукавишникова
- Нижегородский государственный академический театр оперы и балета имени А. С. Пушкина
- памятник в Нижнем Новгороде, Россия
- Церковь Жен-Мироносиц (Нижний Новгород)
- Нижегородский острог
- Сормовский парк
- Блиновский пассаж
- Памятник градостроительства и архитектуры
- Общежитие института инженеров водного транспорта с кинотеатром «Рекорд»
- Нижегородская государственная консерватория имени М. И. Глинки
- Церковь Рождества Иоанна Предтечи на Торгу
- единственный в мире музей литературного критика и писателя Николая Александровича Добролюбова, расположенный в Нижнем Новгороде.
- памятник архитектуры Нижнего Новгорода, корпус Художественного музея
- Собор на Нижегородской ярмарке
- башня Нижегородского кремля
- многоцелевой стадион в Нижнем Новгороде, Россия
- Крестовоздвиженский монастырь (Нижний Новгород)
- Дом П. Е. Кубаревой
- лестница на Волжском откосе в Нижнем Новгороде

- Храм Всемиловитого Спаса (Нижний Новгород)
- соборный храм в Нижегородском районе Нижнего Новгорода
- Спасо-Преображенская церковь (Старые Печёры)
- Нижегородская радиолaborатория
- парк в Нижегородском кремле, Россия
- памятник в России
- православный храм в Нижнем Новгороде
- церковь в Нижнем Новгороде
- Здание Удельной конторы
- Церковь Александра Невского в Сормово
- Усадьба Вяхириных
- Дом Ермолаевых — П. Парфёнова
- Доходный дом Б. Н. Юсупова
- Торговый флигель О. Н. Каменевой
- Дом Е. Трушенинниковой
- Дом Виноградовых
- Дом М. В. Медведева
- Здание Волжско-Камского банка
- Дом И. Н. Соболева
- Дом А. И. Фролова
- православный храм в г. Кстово, Нижегородская область
- Усадьба Е. И. Богоявленской
- Усадьба Киришаумов
- памятник
- Дом культуры ГУВД (Нижний Новгород)

```

non_specific_objects_nn = {"круглая башня Нижегородского кремля",
                           "проездная башня Нижегородского кремля",
                           "Старейший театр Нижнего Новгорода",
                           "Крепость в центре Нижнего Новгорода",
                           "музей в Нижнем Новгороде (Россия)",
                           "православный собор в Нижнем Новгороде",
                           "православный храм",
                           "административное здание в Нижнем
Новгороде, Россия",
                           "один из старейших музеев России,
крупнейший музей изобразительного искусства в Нижегородской области",
                           "Церковь в Нижнем Новгороде",
                           "монастырь в Нижнем Новгороде",
                           "чебное заведение среднего
профессионального образования в Нижнем Новгороде, готовящее
специалистов водного транспорта.С 2005 года — филиал Волжской
государственной академии водного транспорта, которая сейчас называется
ВГУВТ",
                           "парк культуры и отдыха в Нижнем
Новгороде",
                           "здание в Нижегородском кремле, Россия",
                           "памятник в Нижнем Новгороде, Россия",
                           "Памятник градостроительства и
архитектуры",

```



```

критика и писателя Николая Александровича Добролюбова, расположенный в
Нижнем Новгороде",
"башня Нижегородского кремля",
"многоцелевой стадион в Нижнем Новгороде,
Россия",
"соборный храм в Нижегородском районе
Нижнего Новгорода",
"парк в Нижегородском кремле, Россия",
"памятник в России",
"православный храм в Нижнем Новгороде",
"церковь в Нижнем Новгороде", "памятник"}
non_specific_objects_rare_nn = {"круглая башня Нижегородского кремля",
"проездная башня Нижегородского
кремля",
"Старейший театр Нижнего Новгорода",
"Крепость в центре Нижнего Новгорода",
"парк культуры и отдыха в Нижнем
Новгороде",
"единственный в мире музей
литературного критика и писателя Николая Александровича Добролюбова,
расположенный в Нижнем Новгороде",
"башня Нижегородского кремля",
"многоцелевой стадион в Нижнем
Новгороде, Россия",
"парк в Нижегородском кремле, Россия",
}
non_specific_objects_nn = non_specific_objects_nn -
non_specific_objects_rare_nn

```

Владимир

```

|— Никитская церковь
|— Владимирский авиамеханический колледж
|— православный храм во Владимире, памятник белокаменного зодчества
домонгольской Руси
|— Троицкая церковь
|— Мемориальный музей в городе Владимире, посвящённый жизни и
деятельности выдающегося учёного-физика А. Г. Столетова и генерала от
инфантерии Н. Г. Столетова, сыгравшего видную роль в освобождении
Болгарии от турецкого ига
|— Музей во Владимире, Россия
|— православный кладбищенский храм во Владимире
|— театр города Владимира
|— Здание присутственных мест
|— достопримечательность во Владимире
|— католический храм в городе Владимир
|— Православный храм во Владимире
|— Богородице-Рождественский монастырь (Владимир)

```

- Успенский Княгинин монастырь
- памятник XII века во Владимире
- Тюрьма в городе Владимир
- Владимирский государственный гуманитарный университет
- Владимирское духовное училище
- Белокаменные памятники Владимира и Суздаля
- Владимирский планетарий
- Храм Казанской иконы Божией Матери
- Здание городской думы
- Музей ложки
- некрополь во Владимире
- Владимирский кремль
- Владимиро-Суздальский музей-заповедник
- Владимирская духовная семинария
- Армянская церковь в г. Владимир России
- Лесной парк «Дружба»

```

non_specific_objects_vl = {
    "православный храм во Владимире, памятник белокаменного зодчества
домонгольской Руси",
    "Музей во Владимире, Россия",
    "православный кладбищенский храм во Владимире", "театр города
Владимира",
    "достопримечательность во Владимире", "католический храм в городе
Владимир",
    "Православный храм во Владимире", "памятник XII века во
Владимире",
    "Тюрьма в городе Владимир", "Белокаменные памятники Владимира и
Суздаля",
    "некрополь во Владимире", "Армянская церковь в г. Владимир
России"}
non_specific_objects_rare_vl = {
    "православный храм во Владимире, памятник белокаменного зодчества
домонгольской Руси",
    "театр города Владимира", "католический храм в городе Владимир",
    "Тюрьма в городе Владимир", "Армянская церковь в г. Владимир
России"}

non_specific_objects_vl = non_specific_objects_vl -
non_specific_objects_rare_vl

```

Ярославль

- Православный храм в Ярославле
- Архитектурный ансамбль в Ярославле
- Церковь великомученика Димитрия Солунского (Смоленская) (Ярославль)
- образовательное учреждение в Ярославле
- Беседка на Волжской набережной

— ГАУЗ ЯО Клиническая больница скорой медицинской помощи имени Н. В. Соловьёва

- Бутусовский посёлок
- Вечный огонь (Ярославль)
- Владимирская церковь на Божедомке
- Крепостная башня в Ярославле
- Городской сад в Ярославле
- Воинское мемориальное кладбище (Ярославль)
- Башня в Ярославле
- Памятник Георгию Димитрову (Ярославль)
- Памятный знак городам-побратимам (Ярославль)
- Торговые ряды в Ярославле
- Губернаторский дом (Ярославль)
- Демидовский сад
- Демидовский столп (Ярославль)
- Дом Болконского
- Дом муз
- Исторический центр Ярославля
- Казанский монастырь (Ярославль)
- Монастырь в Ярославле
- приход в Ярославле
- Крестобогородская церковь
- кладбище в Ярославле
- белорусский поэт, публицист, литературовед, переводчик.
- Памятник Михаилу Фрунзе (Ярославль)
- Мемориальный дом-музей М. Богдановича
- Михайловские курганы
- мост в Ярославле
- Музей в Ярославле
- Митрополичьи палаты (Ярославль)
- художественный музей в Ярославле
- Музей истории города Ярославля
- Частный музей
- Памятник в Ярославле
- храм в Ярославле
- Николо-Сковородский монастырь
- Церковь в Ярославле
- Памятник жертвам белогвардейского мятежа
- Памятник Леониду Трефолеву (Ярославль)
- Парк в Ярославле
- Петровский монастырь (Ярославль)
- Петропавловский парк
- Приход Воздвижения Святого Креста (Ярославль)
- Российский академический театр драмы им. Ф. Волкова
- монастырь в Ярославле
- Храм в Ярославле
- Ярославский государственный театр кукол
- Тимерёвский археологический комплекс
- Женский православный монастырь Ярославской епархии

- кафедральный собор в Ярославле
- Памятник Фёдору Толбухину (Ярославль)
- Церковь Михаила Архангела
- Храм в честь Ярославской иконы Божией Матери
- Благовещенско-Вознесенский приход
- Вознесенско-Сретенский приход
- Церковь Рождества Христова
- Ярославский государственный цирк
- часовня в Ярославле
- Часовня Казанской Богоматери (Ярославль)
- Школа № 33 имени Карла Маркса
- Ярославская соборная мечеть
- Театр в Ярославле
- Ярославский дворец молодёжи
- Ярославский зоопарк
- Ярославский камерный театр
- Ярославский музей-заповедник
- высшее военное училище в России
- Ярославское художественное училище

```

non_specific_objects_yar = {"Православный храм в Ярославле",
                             "Архитектурный ансамбль в Ярославле",
                             "образовательное учреждение в Ярославле",
                             "Крепостная башня в Ярославле", "Башня в
Ярославле",
                             "Торговые ряды в Ярославле",
                             "Монастырь в Ярославле", "приход в
Ярославле",
                             "кладбище в Ярославле", "мост в
Ярославле",
                             "Музей в Ярославле",
                             "художественный музей в Ярославле",
                             "Частный музей",
                             "Памятник в Ярославле", "храм в
Ярославле",
                             "Церковь в Ярославле", "Парк в Ярославле",
                             "монастырь в Ярославле", "Храм в
Ярославле",
                             "кафедральный собор в Ярославле",
                             "Театр в Ярославле", "часовня в
Ярославле",
                             "высшее военное училище в России"}
non_specific_objects_rare_yar = {"Крепостная башня в Ярославле",
                                  "Торговые ряды в Ярославле",
                                  "Парк в Ярославле",
                                  "кафедральный собор в Ярославле",
                                  "высшее военное училище в России"}
non_specific_objects_yar = non_specific_objects_yar -
non_specific_objects_rare_yar

```

Посмотрим какие записи с не специфичными названиями

```
df[(df['description'].isin(non_specific_objects_ecat)) & (
    df['City'] == 'Екатеринбург')].sample(10)
```

Num	Name
WikiData \	
2173 2173	Добровольцам-танкистам
Q4343452	
998 998	Уральский геологический музей
Q4476747	
846 846	Уральская государственная консерватория
Q4476712	
2170 2170	Добровольцам-танкистам
Q4343452	
2384 2384	улица Февральской Революции
Q4474328	
3194 3420	Центральная городская библиотека им. А. И. Гер...
Q4504106	
1878 1878	Уральский политехнический колледж
Q4476776	
1306 1306	Комсомолу Урала
Q107583229	
2774 3000	Уральский государственный технический университет
Q1938341	
1710 1710	Музей истории, науки и техники Свердловской же...
Q4441173	

	City	Lon	Lat
2173	Екатеринбург	60.606125	56.857285
998	Екатеринбург	60.595470	56.826492
846	Екатеринбург	60.599995	56.837444
2170	Екатеринбург	60.606125	56.857285
2384	Екатеринбург	60.593147	56.841328
3194	Екатеринбург	60.612167	56.821865
1878	Екатеринбург	60.634628	56.842449
1306	Екатеринбург	60.610554	56.844635
2774	Екатеринбург	60.654282	56.844051
1710	Екатеринбург	60.600773	56.858772

	description
2173	памятник в Екатеринбурге
998	музей в Екатеринбурге
846	высшее учебное заведение в Екатеринбурге
2170	памятник в Екатеринбурге
2384	улица в Екатеринбурге
3194	Библиотека в Екатеринбурге
1878	учебное заведение в Екатеринбурге
1306	памятник в Екатеринбурге
2774	бывшее название университета в Екатеринбурге

1710 здание в Железнодорожном районе Екатеринбурга

```
image \
2173 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
998 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
846 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
2170 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
2384 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
3194 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
1878 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
1306 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
2774 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
1710 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
```

```
en_txt
2173 there is a large tank sitting on a pedestal in...
998 arafed display of various types of rocks and o...
846 there is a picture of a building with a flag o...
2170 people are marching in a parade with flags and...
2384 an old photo of a run down house with a lot of...
3194 there is a tall building with a clock on the t...
1878 arafed view of a building with columns and col...
1306 arafed view of a church with a statue in front...
2774 arafed view of a large building with a courtya...
1710 arafed building with a red and white roof and ...
```

```
df[(df['description'].isin(non_specific_objects_nn)) & (
    df['City'] == 'Нижний Новгород')].sample(10)
```

	Num	Name	WikiData	City
Lon \				
4506	1280	Гарнизонная казарма	Q100701435	Нижний Новгород
44.004299				
3553	327	Домик Каширина	Q4165827	Нижний Новгород
43.990692				
4327	1101	Благовещенский монастырь	Q4087775	Нижний Новгород
43.976791				
3785	559	Дворец вице-губернатора	Q99622643	Нижний Новгород
44.003147				
4372	1146	Благовещенский собор	Q4087775	Нижний Новгород
43.976471				
5791	2645	Храм Живоначальной Троицы	Q16715808	Нижний Новгород
44.046577				
5930	2784	М. Горькому	Q4343244	Нижний Новгород
43.990711				
4358	1132	Благовещенский собор	Q4087775	Нижний Новгород
43.976471				
4331	1105	Благовещенский собор	Q4087775	Нижний Новгород
43.976471				
5582	2436	Дом Советов	Q28032873	Нижний Новгород

44.005836

	Lat	description \
4506	56.327614	здание в Нижегородском кремле, Россия
3553	56.323921	музей в Нижнем Новгороде (Россия)
4327	56.323032	монастырь в Нижнем Новгороде
3785	56.327217	административное здание в Нижнем Новгороде, Ро...
4372	56.322552	монастырь в Нижнем Новгороде
5791	56.308556	соборный храм в Нижегородском районе Нижнего Н...
5930	56.313480	памятник в России
4358	56.322552	монастырь в Нижнем Новгороде
4331	56.322552	монастырь в Нижнем Новгороде
5582	56.328762	административное здание в Нижнем Новгороде, Ро...

	image \
4506	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
3553	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
4327	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
3785	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
4372	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
5791	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
5930	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
4358	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
4331	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
5582	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...

	en_txt
4506	arafed view of a large yellow building with a ...
3553	there is a room with a bed, a table and a chair
4327	arafed view of a large white building with a g...
3785	there is a large white building with a canopy ...
4372	arafed white building with green roof and two ...
5791	there are people walking on the sidewalk near ...
5930	a picture of a clock and a statue in a snowy f...
4358	arafed view of a large white building with a g...
4331	arafed white building with green domes and a g...
5582	arafed building with a lot of windows and a lo...

```
df[(df['description'].isin(non_specific_objects_nn)) & (  
    df['City'] == 'Нижний Новгород')].sample(10)
```

	Num	Name	WikiData \
5772	2626	Храм Живоначальной Троицы	Q16715808
4301	1075	Благовещенский монастырь	Q4087775
4374	1148	Благовещенский собор	Q4087775
6136	2990	Церковь во имя Святого Пророка Божия Илии	Q59901
6114	2968	Церковь во имя Святого Пророка Божия Илии	Q59901
4349	1123	Благовещенский собор	Q4087775
4513	1287	Гарнизонная казарма	Q100701435
6228	3082	Печёрский Вознесенский монастырь	Q3860437

4194	968	Рождественская церковь	Q1574846
6139	2993	Церковь во имя Святого Пророка Божия Илии	Q59901

	City	Lon	Lat	\
5772	Нижний Новгород	44.046577	56.308556	
4301	Нижний Новгород	43.976791	56.323032	
4374	Нижний Новгород	43.976471	56.322552	
6136	Нижний Новгород	43.993713	56.328205	
6114	Нижний Новгород	43.993713	56.328205	
4349	Нижний Новгород	43.976471	56.322552	
4513	Нижний Новгород	44.004299	56.327614	
6228	Нижний Новгород	44.049721	56.322777	
4194	Нижний Новгород	43.984966	56.327251	
6139	Нижний Новгород	43.993713	56.328205	

	description	\
5772	соборный храм в Нижегородском районе Нижнего Н...	
4301	монастырь в Нижнем Новгороде	
4374	монастырь в Нижнем Новгороде	
6136	церковь в Нижнем Новгороде	
6114	церковь в Нижнем Новгороде	
4349	монастырь в Нижнем Новгороде	
4513	здание в Нижегородском кремле, Россия	
6228	монастырь в Нижнем Новгороде	
4194	Церковь в Нижнем Новгороде	
6139	церковь в Нижнем Новгороде	

	image	\
5772	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
4301	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
4374	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
6136	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
6114	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
4349	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
4513	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
6228	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
4194	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
6139	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	

	en_txt
5772	arafed view of a gate leading to a building wi...
4301	arafed view of a church with a golden dome and...
4374	arafed view of a large white building with two...
6136	a drawing of a city with a church and a river
6114	arafed view of a city with a church and a tower
4349	arafed view of a large white building with gre...
4513	there is a red chair sitting on the side of a ...
6228	arafed ceiling with a painting of the virgin a...
4194	arafed view of a church with a clock tower and...
6139	a close up of a building with a clock on it


```
df[(df['description'].isin(non_specific_objects_vl)) & (
    df['City'] == 'Владимир')].sample(10)
```

	Num	Name	WikiData	City	Lon
\					
7231	480	Дмитриевский собор	Q550346	Владимир	40.410942
7371	620	Золотые ворота	Q775020	Владимир	40.397278
7213	462	Дмитриевский собор	Q550346	Владимир	40.410942
7642	891	Князь-Владимирское кладбище	Q19691959	Владимир	40.434685
6997	246	Исторический музей	Q21639947	Владимир	40.410965
7366	615	Золотые ворота	Q775020	Владимир	40.397278
7034	283	Князь-Владимирская церковь	Q65062793	Владимир	40.436531
7018	267	Князь-Владимирская церковь	Q65062793	Владимир	40.436531
7374	623	Золотые ворота	Q775020	Владимир	40.397278
7164	413	Бывшая водонапорная башня	Q4114100	Владимир	40.398781

	Lat	description	\
7231	56.129208	Православный храм во Владимире	
7371	56.126831	памятник XII века во Владимире	
7213	56.129208	Православный храм во Владимире	
7642	56.143505	некрополь во Владимире	
6997	56.130333	Музей во Владимире, Россия	
7366	56.126831	памятник XII века во Владимире	
7034	56.142860	православный кладбищенский храм во Владимире	
7018	56.142860	православный кладбищенский храм во Владимире	
7374	56.126831	памятник XII века во Владимире	
7164	56.125111	достопримечательность во Владимире	

	image	\
7231	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
7371	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
7213	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
7642	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
6997	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
7366	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
7034	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
7018	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
7374	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
7164	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	

en_txt

```

7231   arafed building with a golden dome on top of it
7371   there is a white building with a golden dome o...
7213   arafed view of a church with a clock tower and...
7642   arafed man in a park setting next to a grave
6997   there is a red brick building with a clock on ...
7366   arafed view of a large white building with a g...
7034   arafed roof of a church with a cross on top
7018   people are walking around a white church with ...
7374   a view of a large white building with a golden...
7164   there is a man standing in front of a red bric...

```

```

df[(df['description'].isin(non_specific_objects_yar)) & (
    df['City'] == 'Ярославль')].sample(10)

```

	Num	Name	WikiData
City \			
14170	3908	Часовня Александра Невского	Q4507592
Ярославль			
12744	2482	Спасо-Афанасиевский Ярославский монастырь	Q4221355
Ярославль			
14062	3800	Церковь Софии Премудрости Божией в Савино	Q16715786
Ярославль			
10552	290	Богоявленская башня	Q4504600
Ярославль			
12157	1895	Николая Чудотворца	Q4504845
Ярославль			
13407	3145	Храм Петра и Павла	Q12140265
Ярославль			
12285	2023	Памятник 1000-летию Ярославля	Q25389078
Ярославль			
12851	2589	Сретенский храм	Q4438830
Ярославль			
13286	3024	Храм Великомученика Никиты	Q4319898
Ярославль			
11543	1281	Колокольня Никитского прихода	Q4319898
Ярославль			

	Lon	Lat	description \
14170	39.891064	57.625462	часовня в Ярославле
12744	39.895222	57.625370	Монастырь в Ярославле
14062	39.919182	57.623425	Церковь в Ярославле
10552	39.887489	57.620663	Православный храм в Ярославле
12157	39.865314	57.610310	храм в Ярославле
13407	39.843700	57.602531	Православный храм в Ярославле
12285	39.904823	57.617668	Памятник в Ярославле
12851	39.888905	57.625244	Храм в Ярославле
13286	39.868584	57.623791	приход в Ярославле
11543	39.868759	57.623619	приход в Ярославле

image \

```

14170 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAICAgICAQICAg...
12744 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
14062 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
10552 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
12157 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
13407 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAICAgICAQICAg...
12285 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
12851 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
13286 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
11543 /9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...

```

```

en_txt
14170 arafed image of a park with a statue and benches
12744 trees in front of a building with a clock tower
14062 there is a building with a clock tower and a f...
10552 aerial view of a large white building with a g...
12157 an old picture of a large building with two to...
13407 arafed image of a building with a fountain and...
12285 statue of a man holding a child in his arms
12851 arafed view of a church with a blue and gold wall
13286 arafed priest standing in front of a casket wi...
11543 aerial view of a church tower in a rural area

```

не специфичное описание нельзя в полной мере использовать для отбрасывания записей, так как Name остается информативным, но можно использовать для оценки качества

Поле Lat Lon

```

df.Lat.unique().shape
(322,)

df.Lon.unique().shape
(320,)

```

Построим отображение географических точек

```

import matplotlib.pyplot as plt
import pandas as pd

# Создаем DataFrame из дополнительных данных
additional_data = pd.DataFrame(
    {'city': ['Екатеринбург', 'Нижний Новгород', 'Владимир',
              'Ярославль'],
     'Lat': [56.8519, 56.3287, 56.1366, 57.6299],
     'Lon': [60.6122, 44.0020, 40.3966, 39.8737]})

plt.figure(figsize=(10, 8))

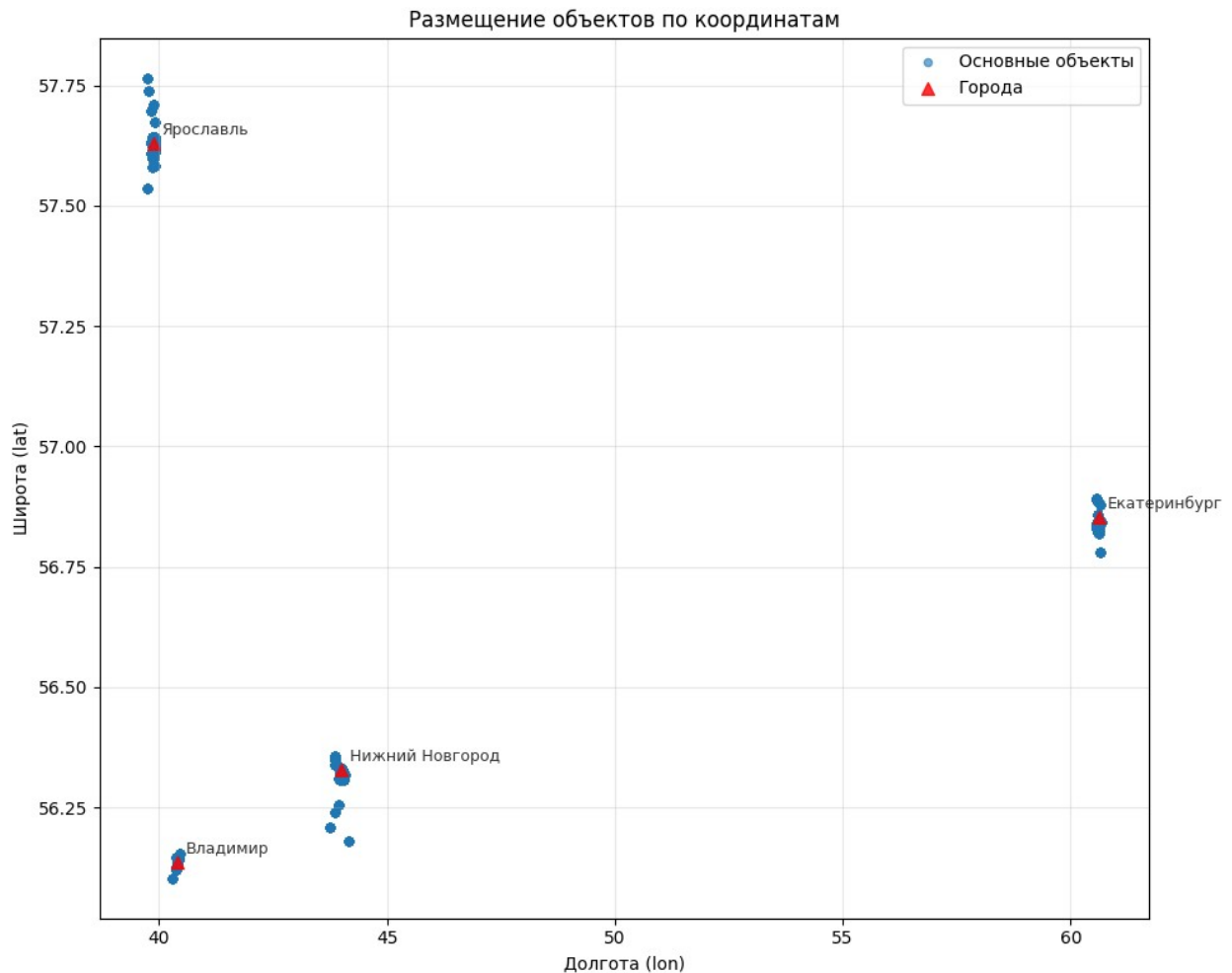
```

```
# Отрисовка основных объектов
plt.scatter(df['Lon'], df['Lat'], alpha=0.6, s=20, label='Основные
объекты')

# Отрисовка дополнительных объектов (городов)
plt.scatter(additional_data['Lon'], additional_data['Lat'], alpha=0.8,
s=50,
            color='red', marker='^', label='Города')

# Добавляем подписи для городов
for i, row in additional_data.iterrows():
    plt.annotate(row['city'], (row['Lon'], row['Lat']), xytext=(5, 5),
                textcoords='offset points', fontsize=9, alpha=0.8)

plt.xlabel('Долгота (lon)')
plt.ylabel('Широта (lat)')
plt.title('Размещение объектов по координатам')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Выделим окрестности городов и построим отдельные схемы

```
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
axes = axes.flatten()

for i, city in additional_data.iterrows():
    # Определяем границы окрестности ±0.2 градуса
    lat_min = city['Lat'] - 0.2
    lat_max = city['Lat'] + 0.2
    lon_min = city['Lon'] - 0.2
    lon_max = city['Lon'] + 0.2

    # Фильтруем объекты из df в окрестности города
    mask = (df['Lat'] >= lat_min) & (df['Lat'] <= lat_max) & (
        df['Lon'] >= lon_min) & (df['Lon'] <= lon_max)

    filtered_df = df[mask]

    # Отрисовываем на соответствующем subplot
    axes[i].scatter(filtered_df['Lon'], filtered_df['Lat'], alpha=0.6,
```

```

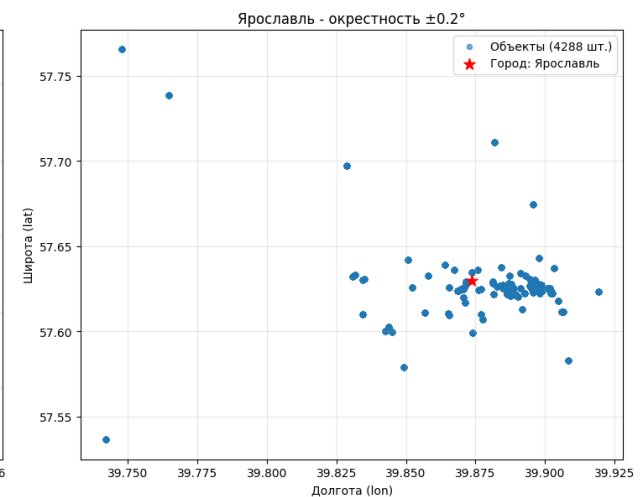
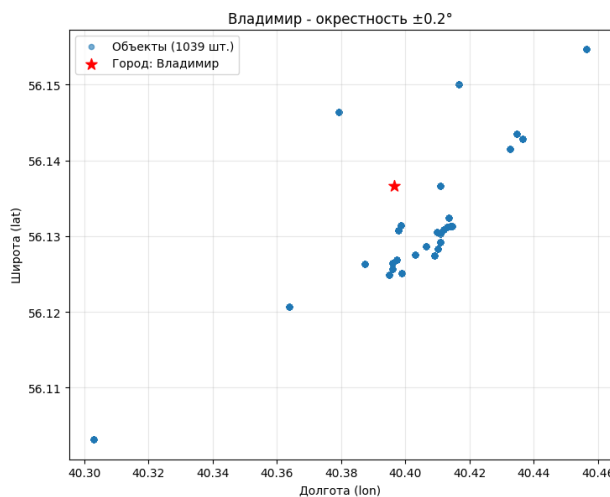
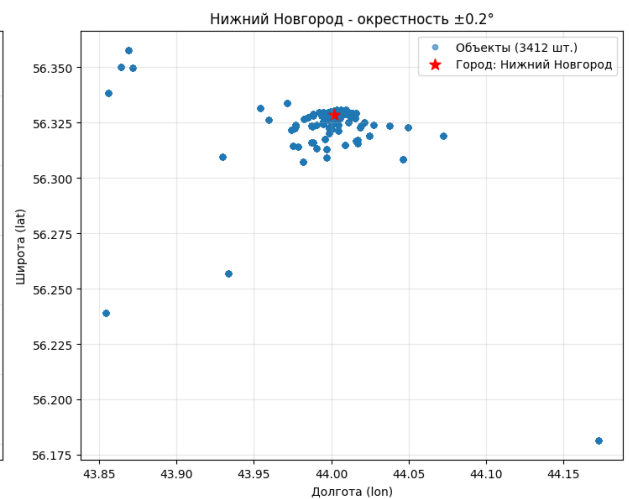
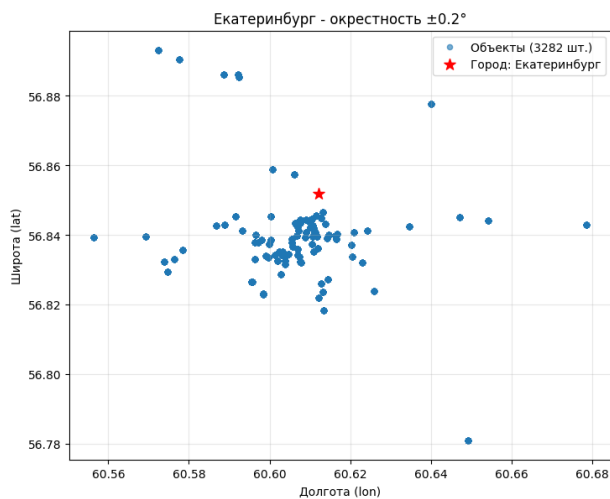
s=20,
                                label=f'Объекты ({len(filtered_df)} шт.)')

    axes[i].scatter(city['Lon'], city['Lat'], alpha=1.0, s=100,
color='red',
                                marker='*', label=f"Город: {city['city']}")

    axes[i].set_xlabel('Долгота (lon)')
    axes[i].set_ylabel('Широта (lat)')
    axes[i].set_title(f"{city['city']} - окрестность  $\pm 0.2^\circ$ ")
    axes[i].grid(True, alpha=0.3)
    axes[i].legend()

plt.tight_layout()
plt.show()

```



Построим частые объекты в отдельных городах, но редкие в конкретном городе

```
type_rare
```

```

['сад', 'сквер', 'гостиница', 'стадион', 'зоопарк', 'цирк',
'планетарий']

fig, axes = plt.subplots(2, 2, figsize=(15, 12))
axes = axes.flatten()

for i, city in additional_data.iterrows():
    # Определяем границы окрестности ±0.2 градуса
    lat_min = city['Lat'] - 0.2
    lat_max = city['Lat'] + 0.2
    lon_min = city['Lon'] - 0.2
    lon_max = city['Lon'] + 0.2

    # Фильтруем объекты из df в окрестности города
    mask = (df['Lat'] >= lat_min) & (df['Lat'] <= lat_max) & (
        df['Lon'] >= lon_min) & (df['Lon'] <= lon_max)

    filtered_df = df[mask]

    # Отрисовываем все объекты на соответствующем subplot
    axes[i].scatter(filtered_df['Lon'], filtered_df['Lat'], alpha=0.6,
s=20,
                    label=f'Объекты ({len(filtered_df)} шт.)')

    # Фильтруем и отмечаем объекты из type_rare зеленым цветом
    rare_mask =
filtered_df['description'].str.contains('|'.join(type_rare),
                                                case=False,
na=False)
    rare_objects = filtered_df[rare_mask]

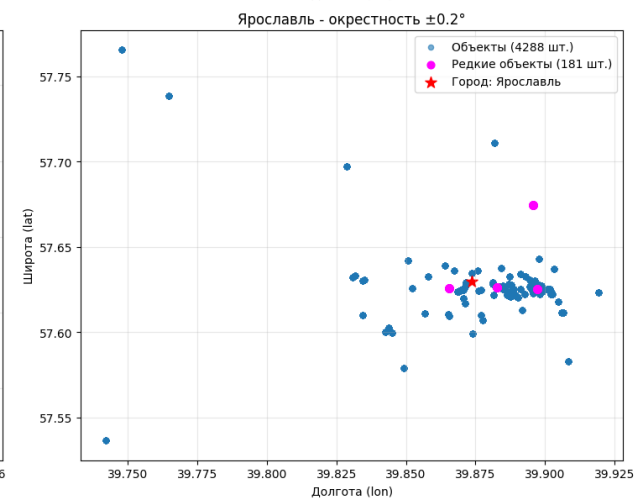
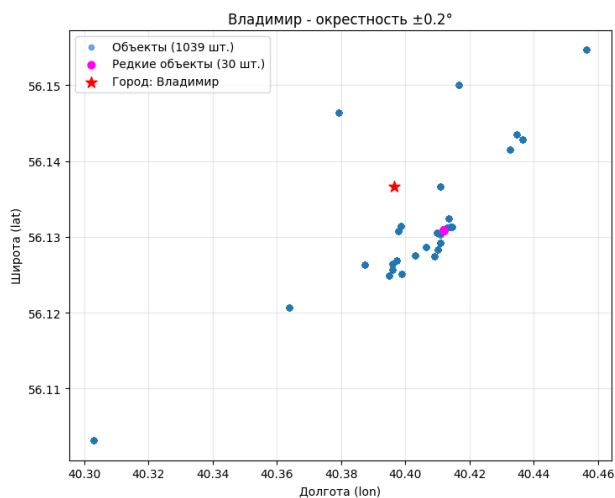
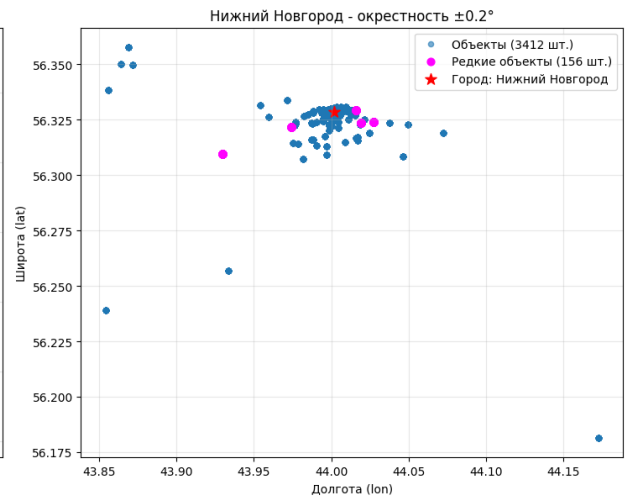
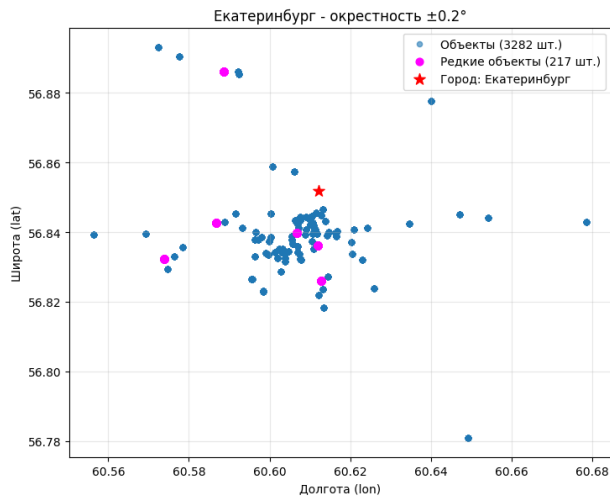
    if len(rare_objects) > 0:
        axes[i].scatter(rare_objects['Lon'], rare_objects['Lat'],
alpha=1.0,
                        s=40, color='magenta', marker='o',
                        label=f'Редкие объекты ({len(rare_objects)}
шт.)')

    axes[i].scatter(city['Lon'], city['Lat'], alpha=1.0, s=100,
color='red',
                    marker='*', label=f"Город: {city['city']}")

    axes[i].set_xlabel('Долгота (lon)')
    axes[i].set_ylabel('Широта (lat)')
    axes[i].set_title(f"{city['city']} - окрестность ±0.2°")
    axes[i].grid(True, alpha=0.3)
    axes[i].legend()

plt.tight_layout()
plt.show()

```



Поиск мусора в description

```
from sklearn.feature_extraction.text import TfidfVectorizer
from nltk.corpus import stopwords
# import pymorphy2

# Скачиваем русские стоп-слова (если еще не скачаны)
import nltk

nltk.download('stopwords')

# Получаем русские стоп-слова
russian_stop_words = stopwords.words('russian')

# Добавляем дополнительные стоп-слова, характерные для описаний
# достопримечательностей
additional_stop_words = ['россия', 'город', 'находиться',
                          'располагаться',
                          'являться', 'также', 'например', 'который',
                          'мочь',
```



```

        'один', 'год', 'время']
# russian_stop_words.extend(additional_stop_words)
# Инициализация векторизера с базовой предобработкой
# min_df отсекает слишком редкие слова, max_df - слишком частые (но мы
их тоже посмотрим)

vectorizer = TfidfVectorizer(stop_words=russian_stop_words, # Русские
# стоп-слова
                                max_features=1000, # Рассмотрим топ-1000
# слов
                                min_df=2,
                                # Слово должно встретиться минимум в 2
# документах
                                max_df=0.8,
                                # Исключаем слова, встречающиеся в >80%
# документов (как стоп-слова)
                                ngram_range=(1, 2),
                                # Учитываем не только отдельные слова, но
# и пары (биграммы)
                                analyzer='word', # Анализируем по словам
                                lowercase=True, # Приводим к нижнему
# регистру
                                )

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\k142\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!

import numpy as np

# Применяем TF-IDF
tfidf_matrix = vectorizer.fit_transform(df['description'])

# Получаем названия признаков (слова/биграммы)
feature_names = vectorizer.get_feature_names_out()

# Вычисляем средний TF-IDF для каждого слова по всем документам
mean_tfidf = np.asarray(tfidf_matrix.mean(axis=0)).flatten()

# Создаем DataFrame для анализа
tfidf_df = pd.DataFrame(
    {'feature': feature_names, 'mean_tfidf': mean_tfidf}).sort_values(
    'mean_tfidf', ascending=False)

```

Посмотрим на **топ-20** самых "важных" слов (высокий средний TF-IDF)

```

print("Топ-20 слов с наивысшим TF-IDF:")
print(tfidf_df.head(20))

```

Топ-20 слов с наивысшим TF-IDF:

	feature	mean_tfidf
982	ярославле	0.061273
295	екатеринбурге	0.058870
916	храм	0.044177
928	храм ярославле	0.036085
693	православный	0.035165
631	памятник	0.034393
697	православный храм	0.033272
535	музей	0.029968
942	церковь	0.024253
326	здание	0.023773
249	дом	0.021800
51	башня	0.020434
638	памятник екатеринбурге	0.019363
598	новгороде	0.018251
586	нижнем новгороде	0.018251
585	нижнем	0.018251
518	монастырь	0.018190
540	музей екатеринбурге	0.018144
330	здание екатеринбурге	0.017824
429	кремля	0.016293

Посмотрим на **топ-20** самых "неважных" (низкий TF-IDF)

```
print("\nТоп-20 слов с наинизшим TF-IDF:")
print(tfidf_df.tail(20))
```

Топ-20 слов с наинизшим TF-IDF:

	feature	mean_tfidf
100	видную	0.000375
101	видную роль	0.000375
870	турецкого ига	0.000375
620	освобождении	0.000375
170	генерала инфантерии	0.000375
169	генерала	0.000375
621	освобождении болгарии	0.000375
161	выдающегося учёного	0.000375
539	музей городе	0.000375
241	деятельности	0.000375
160	выдающегося	0.000375
242	деятельности выдающегося	0.000375
498	мемориальный музей	0.000375
688	посвящённый	0.000375
689	посвящённый жизни	0.000375
313	жизни	0.000375
314	жизни деятельности	0.000375
194	городе владимире	0.000375

109	владимире посвящённый	0.000375
836	сыгравшего видную	0.000375

Топ-20 часто встречающихся биграмм хорошо коррелируются с ранее найденными типами достопримечательностей

Поиск мусора en_txt

Подразумеваем, что:

- предыдущий анализ показал, что много записей относятся к добавлению фотографий к объектам
- по описанию фото можно определить качество фото и записи для описания объекта

```
from sklearn.feature_extraction.text import TfidfVectorizer
# import pymorphy2

# Скачиваем русские стоп-слова (если еще не скачаны)
import nltk

nltk.download('stopwords')

vectorizer = TfidfVectorizer(stop_words='english', # Русские стоп-
                             # слова
                             max_features=1000, # Рассмотрим топ-1000
                             # слов
                             min_df=2,
                             # Слово должно встретиться минимум в 2
                             # документах
                             max_df=0.8,
                             # Исключаем слова, встречающиеся в >80%
                             # документов (как стоп-слова)
                             ngram_range=(1, 2),
                             # Учитываем не только отдельные слова, но
                             # и пары (биграммы)
                             analyzer='word', # Анализируем по словам
                             lowercase=True, # Приводим к нижнему
                             # регистру
                             )

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\k142\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!

import numpy as np

# Применяем TF-IDF
tfidf_matrix = vectorizer.fit_transform(df['en_txt'])

# Получаем названия признаков (слова/биграммы)
feature_names = vectorizer.get_feature_names_out()
```

```
# Вычисляем средний TF-IDF для каждого слова по всем документам
mean_tfidf = np.asarray(tfidf_matrix.mean(axis=0)).flatten()

# Создаем DataFrame для анализа
tfidf_df = pd.DataFrame(
    {'feature': feature_names, 'mean_tfidf': mean_tfidf}).sort_values(
    'mean_tfidf', ascending=False)
```

Посмотрим на **топ-20** самых "важных" слов (высокий средний TF-IDF)

```
print("Топ-20 слов с наивысшим TF-IDF:")
print(tfidf_df.head(20))
```

Топ-20 слов с наивысшим TF-IDF:

	feature	mean_tfidf
110	building	0.070168
7	arafed	0.061384
453	large	0.046563
242	clock	0.044971
916	view	0.042362
34	arafed view	0.038284
958	white	0.037677
869	tower	0.036387
200	church	0.035389
384	green	0.033735
121	building clock	0.031621
246	clock tower	0.029392
959	white building	0.027376
459	large building	0.026662
690	roof	0.025082
809	steeple	0.022494
925	view large	0.022288
501	man	0.021005
799	statue	0.020443
393	green roof	0.020252

Посмотрим на **топ-20** самых "неважных" (низкий TF-IDF)

```
print("\nТоп-20 слов с наимизшим TF-IDF:")
print(tfidf_df.tail(20))
```

Топ-20 слов с наимизшим TF-IDF:

	feature	mean_tfidf
950	wall paintings	0.000391
537	middle town	0.000391
671	riding bike	0.000389
450	lamp post	0.000388

784	stand line	0.000384
736	sitting chairs	0.000382
910	uniform stand	0.000381
278	crystals	0.000375
427	house blue	0.000374
295	display various	0.000372
661	red fence	0.000371
213	church lit	0.000370
654	priests standing	0.000370
106	broken	0.000362
931	view stage	0.000362
581	park river	0.000361
772	stadium green	0.000354
973	white table	0.000350
478	leading grassy	0.000341
664	red line	0.000323

Отбор незначущих слов

```
import pandas as pd
import re

junc_words = ["wall paintings", "middle town", "riding bike", "lamp
post",
              "stand line", "sitting chairs", "uniform stand",
"crystals",
              "house blue", "display various", "red fence", "church
lit",
              "priests standing", "broken", "view stage", "park
river",
              "stadium green", "white table", "leading grassy", "red
line"]
# Создаем регулярное выражение из списка слов
pattern = '|'.join([re.escape(word) for word in junc_words])
df['is_junc_words'] = False
df['is_junc_words'] = df['en_txt'].str.contains(pattern, case=False,
na=False)

def find_matching_text(text):
    if pd.isna(text):
        return None
    for word in junc_words:
        if word.lower() in str(text).lower():
            return word
    return None

df['found_text'] = df['en_txt'].apply(find_matching_text)
```

```
df[df['is_junc_words']]
```

	Num	Name
WikiData \		
198	198	Дом-музей Бажова
Q4290214		
347	347	Суворовское училище
Q4174329		
348	348	Суворовское училище
Q4174329		
349	349	Суворовское училище
Q4174329		
352	352	Суворовское училище
Q4174329		
...	...	...

```
..
14525 4263 Ярославский филиал Военно-космической академии...
Q4538913
14531 4269 Ярославский филиал Военно-космической академии...
Q4538913
14533 4271 Ярославский филиал Военно-космической академии...
Q4538913
14534 4272 Ярославский филиал Военно-космической академии...
Q4538913
14547 4285 Ярославский филиал Военно-космической академии...
Q4538913
```

	City	Lon	Lat	\
198	Екатеринбург	60.613415	56.818394	
347	Екатеринбург	60.647209	56.845078	
348	Екатеринбург	60.647209	56.845078	
349	Екатеринбург	60.647209	56.845078	
352	Екатеринбург	60.647209	56.845078	
...	...	...	...	
14525	Ярославль	39.877613	57.607079	
14531	Ярославль	39.877613	57.607079	
14533	Ярославль	39.877613	57.607079	
14534	Ярославль	39.877613	57.607079	
14547	Ярославль	39.877613	57.607079	

	description	\
198	Мемориальный дом-музей П. П. Бажова	
347	военно-учебное заведение в Екатеринбурге	
348	военно-учебное заведение в Екатеринбурге	
349	военно-учебное заведение в Екатеринбурге	
352	военно-учебное заведение в Екатеринбурге	
...	...	
14525	высшее военное училище в России	
14531	высшее военное училище в России	
14533	высшее военное училище в России	

14534	высшее военное училище в России
14547	высшее военное училище в России

		image \
198	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
347	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
348	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
349	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
352	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAICAgICAQICAg...	
...		...
14525	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
14531	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
14533	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
14534	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
14547	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	

		en_txt
is_junc_words \		
198	there is a table with a white table cloth and ...	
	True	
347	arafed group of military men in uniform standi...	
	True	
348	there are many people in uniform standing in a...	
	True	
349	arafed military men in uniform stand in a line...	
	True	
352	arafed image of a group of men in uniform stan...	
	True	
...		
.		
14525	arafed female soldiers in uniform stand in a line	
	True	
14531	a group of men in uniform stand in front of a ...	
	True	
14533	soldiers in uniform stand in a line in front o...	
	True	
14534	arafed group of men in uniform standing in a line	
	True	
14547	soldiers in uniform stand in a line in front o...	
	True	

	found_text
198	white table
347	uniform stand
348	uniform stand
349	uniform stand
352	uniform stand
...	...
14525	uniform stand
14531	uniform stand

```

14533 uniform stand
14534 uniform stand
14547 uniform stand

[105 rows x 11 columns]

get_image(df.at[366, 'image'])

```



Построим предпросмотр фото из этого списка

```

import base64
from io import BytesIO
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

def show_image_blocks_8x1(df, num_blocks=30, start_index=0,
    figsize=(16, 2),
                                image_size=(64, 64)):
    """
    Показывает блоки изображений 8x1 из DataFrame с уменьшенным
    размером

```



```

Parameters:
- df: DataFrame с колонкой 'image' содержащей base64 строки
- num_blocks: количество блоков для показа (по умолчанию 1)
- start_index: начальный индекс в DataFrame (по умолчанию 0)
- figsize: размер фигуры (по умолчанию (16, 2))
- image_size: размер для ресайза изображений (по умолчанию (64,
64))
"""

for block_num in range(num_blocks):
    # Создаем фигуру для одной линии (1x8)
    fig, axes = plt.subplots(1, 8, figsize=figsize)

    # Если axes не является массивом, преобразуем в массив для
    единообразия
    if not hasattr(axes, '__iter__'):
        axes = [axes]

    # Вычисляем начальный индекс для текущего блока
    current_start = start_index + (block_num * 8)

    for i in range(8):
        idx = current_start + i

        # Проверяем, что индекс в пределах DataFrame
        if idx >= len(df):
            axes[i].axis('off')
            continue

        try:
            # Декодируем и загружаем изображение
            image_data = base64.b64decode(df.iloc[idx]['image'])
            image = Image.open(BytesIO(image_data)).convert('RGB')

            # Уменьшаем размер изображения для экономии памяти
            image = image.resize(image_size,
Image.Resampling.LANCZOS)

            # Показываем изображение
            axes[i].imshow(np.array(image))
            found_text = str(df.iloc[idx].name) + ':' +
df.iloc[idx][
                'found_text']
            # Обрезаем длинный текст для заголовка
            title_text = found_text if len(
                found_text) <= 15 else found_text[:12] + "..."
            axes[i].set_title(f'{title_text}', fontsize=8)
            axes[i].axis('off')

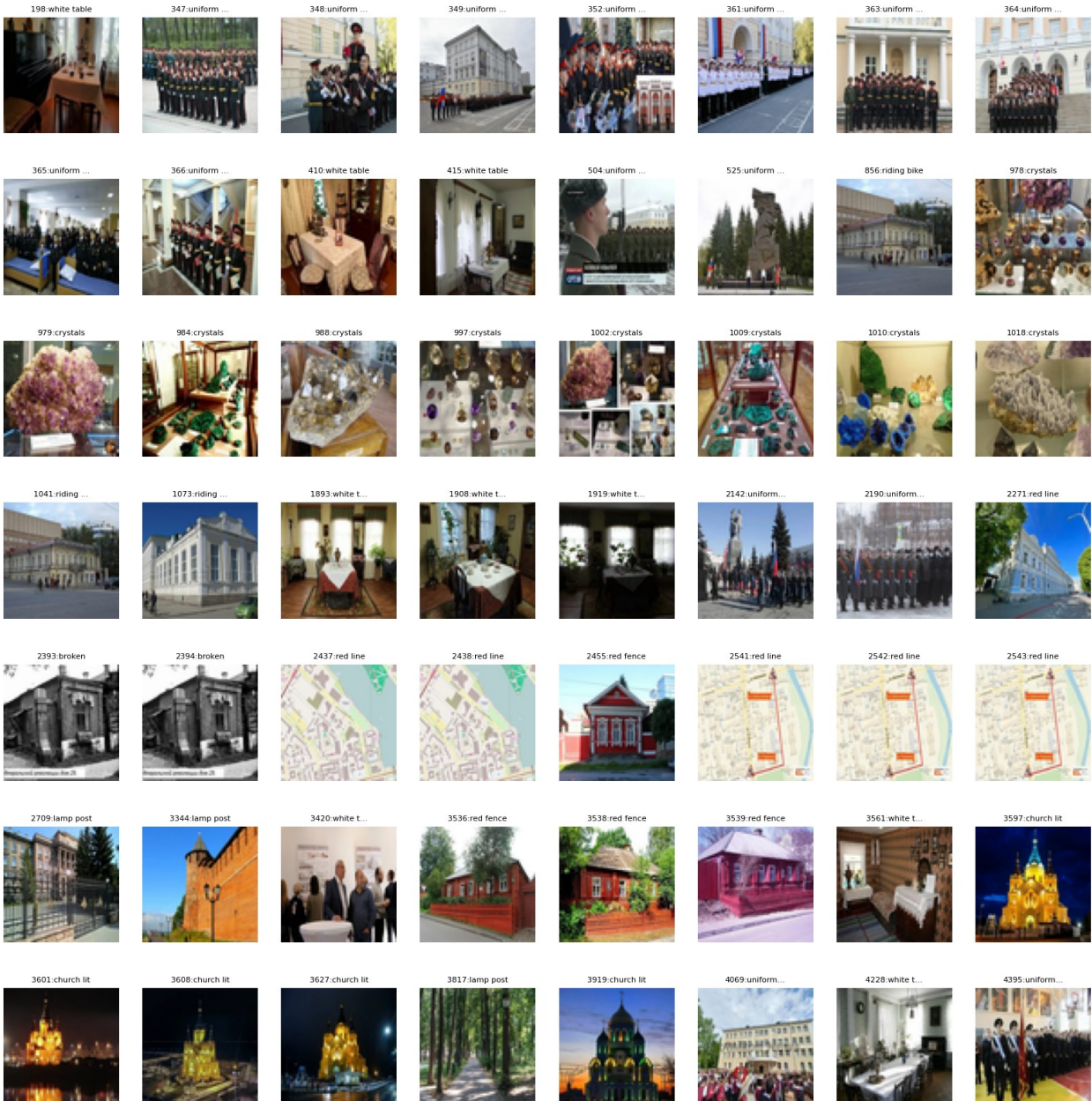
        except Exception as e:

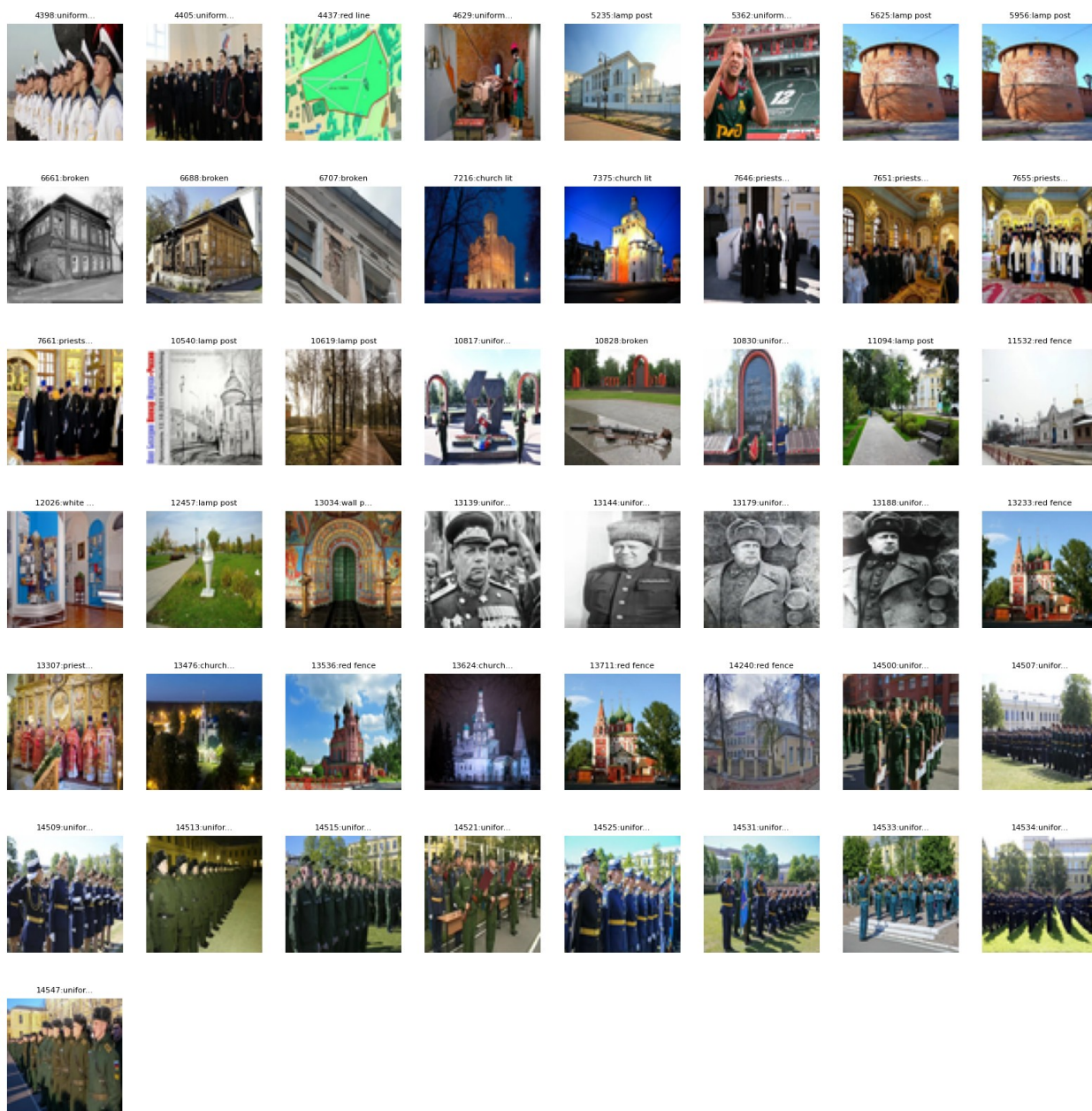
```

```
print(f"Ошибка при загрузке изображения {idx}: {e}")
axes[i].axis('off')
axes[i].set_title(f'Error', fontsize=6)
```

```
plt.tight_layout()
plt.show()
```

```
show_image_blocks_8x1(df[df['is_junc_words']], num_blocks=16)
```





Эти данные безболезненно можно удалить - они слабо относятся к достопримечательностям города.


```
df = df[~df['is_junc_words']]
```

Отбор 100

Для дальнейшего тестирования и получения чистых сэмплов, рекомендуется взять не менее 100 чистых образцов со стратификацией по городу и названию места. Это можно сделать, выбрав случайным образом несколько образцов из каждой категории (город + название места), чтобы обеспечить равномерное представительство и разнообразие данных.

!! Важно: Помните, что чистые сэмплы одного и того же места выдадут примерно одинаковое описание. Есть несколько стратегий, как повысить разнообразие ответов:

- выбор одного самого лучшего:
 - самое длинное описание
 - самое подходящие по мнению LLM
- конкатенация описаний (но важно не сконкатенировать слишком много)

Изображения можно отдать в мета-данные.

```
type_freq = ['дом', 'церковь', 'музей', 'памятник', 'собор', 'башня',  
'театр',  
            'улица', 'парк', 'монастырь', 'усадьба', 'университет',  
'мост',  
            'храм']  
type_rare = ['сад', 'сквер', 'гостиница', 'стадион', 'зоопарк',  
'цирк',  
            'планетарий']
```

есть набор df

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
Index: 12078 entries, 0 to 14633  
Data columns (total 9 columns):  
#   Column          Non-Null Count  Dtype  
---  -  
0   Num             12078 non-null  int64  
1   Name            12078 non-null  object  
2   WikiData        12078 non-null  object  
3   City            12078 non-null  object  
4   Lon             12078 non-null  float64  
5   Lat             12078 non-null  float64  
6   description     12078 non-null  object  
7   image           12078 non-null  object  
8   en_txt          12078 non-null  object
```

```
dtypes: float64(2), int64(1), object(6)
memory usage: 943.6+ KB
df.City.unique()
array(['Екатеринбург', 'Нижний Новгород', 'Владимир', 'Ярославль'],
      dtype=object)
```

надо отобрать 150 записей со страфикацией по каждому городу и названию Name.

```
type_freq = ['дом', 'церковь', 'музей', 'памятник', 'собор', 'башня',
             'театр',
             'улица', 'парк', 'монастырь', 'усадьба', 'университет',
             'мост',
             'храм']
type_rare = ['сад', 'сквер', 'гостиница', 'стадион', 'зоопарк',
             'цирк',
             'планетарий']
```

поле description должно содержать частые названия типа достопримечательности из списка type_freq и единичные из списка type_rare

выбрать наиболее представимые по длине описания

Добавим поле тип объекта и длину описания

```
import pandas as pd
import numpy as np

# Функция для определения типа достопримечательности по названию
def get_attraction_type(name):
    name_lower = name.lower()

    # Сначала проверяем редкие типы
    for rare_type in type_rare:
        if rare_type in name_lower:
            return rare_type

    # Затем частые типы
    for freq_type in type_freq:
        if freq_type in name_lower:
            return freq_type

    return 'other'

# Добавляем столбец с типом достопримечательности
df['attraction_type'] = df['description'].apply(get_attraction_type)

# Функция для вычисления длины описания (учитываем только русские
```

```

СИМВОЛЫ)
def count_russian_chars(text):
    russian_letters = set('абвгдеёжзийклмнопрстуфхцчшщъыьэюя')
    return sum(1 for char in str(text).lower() if char in
russian_letters)

df['description_length'] =
df['description'].apply(count_russian_chars)

# Фильтруем только нужные типы (исключаем 'other')
valid_types = type_freq + type_rare
df_filtered = df[df['attraction_type'].isin(valid_types)].copy()

df_filtered[df_filtered['City'] == 'Ярославль'][
    'attraction_type'].value_counts()

attraction_type
храм      830
памятник  458
монастырь 316
церковь   255
музей     238
дом        228
парк      155
башня     145
театр     145
сад        84
собор      69
зоопарк    57
цирк       39
мост       29
Name: count, dtype: int64

```

Проведем выборку объектов в комбинации City+Name и распределенном по типу объекта

```

def stratified_sample(df, n_total=150):
    cities = df['City'].unique()
    samples_per_city = n_total // len(cities)

    sampled_dfs = []
    used_combinations = set() # Множество для отслеживания
использованных комбинаций City+Name

    for city in cities:
        city_df = df[df['City'] == city]

        # Распределяем выборку по типам внутри города
        types_in_city = city_df['attraction_type'].unique()
        base_samples_per_type = samples_per_city // len(types_in_city)

```

```

remainder = samples_per_city % len(types_in_city)

city_samples = []

for i, attr_type in enumerate(types_in_city):
    type_df = city_df[city_df['attraction_type'] == attr_type]

    # Определяем сколько брать из этого типа
    n_samples = base_samples_per_type
    if i < remainder:
        n_samples += 1

    if len(type_df) >= n_samples:
        # Выбираем наиболее представительные по длине описания
        median_length = type_df['description_length'].median()
        type_df = type_df.copy()
        type_df['length_diff'] = abs(
            type_df['description_length'] - median_length)

        # Сортируем по репрезентативности и выбираем
уникальные имена
        sampled_type = type_df.sort_values('length_diff')

        # Фильтруем уже использованные комбинации и берем
уникальные имена
        unique_samples = []
        for _, row in sampled_type.iterrows():
            combination = (row['City'], row['Name'])
            if combination not in used_combinations:
                unique_samples.append(row)
                used_combinations.add(combination)
            if len(unique_samples) >= n_samples:
                break

        sampled_type = pd.DataFrame(unique_samples)
    else:
        # Если записей меньше, чем нужно, берем все уникальные
        unique_samples = []
        for _, row in type_df.iterrows():
            combination = (row['City'], row['Name'])
            if combination not in used_combinations:
                unique_samples.append(row)
                used_combinations.add(combination)
        sampled_type = pd.DataFrame(unique_samples)

    city_samples.append(sampled_type)

# Фильтруем пустые DataFrame
city_samples = [sample for sample in city_samples if not
sample.empty]

```

```

        if city_samples:
            sampled_dfs.extend(city_samples)

    result_df = pd.concat(sampled_dfs,
                           ignore_index=True) if sampled_dfs else
pd.DataFrame()

    # Если общее количество не равно n_total, делаем корректировку
    if len(result_df) > n_total:
        # Удаляем лишние записи, начиная с наименее репрезентативных
        result_df = result_df.nlargest(n_total, 'description_length')
    elif len(result_df) < n_total:
        # Добавляем недостающие записи из наиболее репрезентативных
        additional_needed = n_total - len(result_df)
        remaining_df = df.copy()

        # Создаем маску для исключения уже использованных комбинаций
        mask = remaining_df.apply(
            lambda row: (row['City'], row['Name']) not in
used_combinations,
            axis=1)
        available_df = remaining_df[mask]

        if not available_df.empty:
            additional_samples =
available_df.nlargest(additional_needed,
'description_length')
            result_df = pd.concat([result_df, additional_samples],
                                   ignore_index=True)

    # Финальная проверка уникальности
    duplicates = result_df.duplicated(subset=['City', 'Name']).sum()
    if duplicates > 0:
        print(
            f"Предупреждение: в итоговой выборке найдено {duplicates}
дубликатов. Удаляем...")
        result_df = result_df.drop_duplicates(subset=['City', 'Name'])

    print(f"Итоговый размер выборки: {len(result_df)} записей")
    print(
        f"Уникальных комбинаций City+Name: {result_df[['City',
'Name']].drop_duplicates().shape[0]}")

    return result_df

# Выполняем стратифицированную выборку
sampled_df = stratified_sample(df_filtered, 150)

```


Предупреждение: в итоговой выборке найдено 30 дубликатов. Удаляем...
Итоговый размер выборки: 120 записей
Уникальных комбинаций City+Name: 120

Проверяем распределение

```
print("Распределение по городам:")
print(sampled_df['City'].value_counts())
print("\nРаспределение по типам достопримечательностей:")
print(sampled_df['attraction_type'].value_counts())

# Удаляем вспомогательные столбцы
# sampled_df = sampled_df.drop(['attraction_type',
# 'description_length'], axis=1)

print(f"\nИтоговый размер выборки: {len(sampled_df)} записей")
```

Распределение по городам:

City	
Нижний Новгород	36
Екатеринбург	32
Ярославль	32
Владимир	20

Name: count, dtype: int64

Распределение по типам достопримечательностей:

attraction_type	
храм	13
музей	13
дом	11
памятник	11
церковь	11
театр	9
монастырь	9
собор	8
парк	7
башня	7
сад	7
университет	3
мост	3
стадион	2
улица	2
гостиница	1
планетарий	1
цирк	1
зоопарк	1

Name: count, dtype: int64

Итоговый размер выборки: 120 записей

sampld_df

	Num	Name	WikiData	
City \				
0	2372	Доходный дом купца Чувильдина	Q27921892	Екатеринбург
1	2219	№29 Дом В.И. Чувильдина	Q27921892	Екатеринбург
2	2318	Дом Актёра	Q4165449	Екатеринбург
3	993	Уральский геологический музей	Q4476747	Екатеринбург
4	1661	Музей истории Екатеринбурга	Q4306294	Екатеринбург
..	...	...	...	...
115	2773	Успенский собор	Q1140551	Ярославль
116	4043	Ярославская соборная мечеть	Q4281966	Ярославль
117	3851	Цирк	Q4538876	Ярославль
118	4114	Ярославский зоопарк	Q16723452	Ярославль
119	783	Дмитриевская башня	Q18406802	Нижний Новгород

	Lon	Lat	
description \			
0	60.607727	56.832176	Дом купца Чувильдина
1	60.607689	56.832268	Дом купца Чувильдина
2	60.597950	56.838634	Дом актёра (Екатеринбург)
3	60.595470	56.826492	музей в Екатеринбурге
4	60.611423	56.840691	музей в Екатеринбурге
..	...	...	...
...			
115	39.901978	57.622429	кафедральный собор в Ярославле
116	39.875633	57.636059	Ярославская соборная мечеть
117	39.865444	57.625584	Ярославский государственный цирк
118	39.895737	57.674446	Ярославский зоопарк
119	44.005676	56.327045	проездная башня Нижегородского кремля, один из...

	image \
0	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
1	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
2	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
3	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
4	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
..	...
115	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
116	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
117	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
118	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...
119	/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAUEBAUEAwUFBA...

	en_txt	is_junc_words
0	there is a large building with a clock on the ...	False
1	there is a large building with a clock on top ...	False
2	nighttime shot of a building with stairs and t...	False
3	there is a large rock sitting on a wooden tabl...	False
4	arated room with a large display of old photos...	False
..	...	...
115	arated view of a large white building with gol...	False
116	there is a small pond in front of a building w...	False
117	there is a large building with a sign on the f...	False
118	there is a large teddy bear sitting on top of ...	False
119	people walking on the sidewalk near a brick bu...	False

	found_text	attraction_type	description_length	length_diff
0	None	дом	18	0.0
1	None	дом	18	0.0
2	None	дом	21	3.0
3	None	музей	19	0.0
4	None	музей	19	0.0
..	...	...	...	...
115	None	собор	27	0.0
116	None	собор	25	2.0
117	None	цирк	30	0.0
118	None	зоопарк	18	0.0
119	None	башня	64	NaN

```
[120 rows x 14 columns]

sampled_df[sampled_df['City'] == 'Ярославль']
['attraction_type'].value_counts()

attraction_type
храм          3
церковь       3
дом           3
башня         3
памятник      3
монастырь     3
музей         3
сад           2
парк          2
театр         2
собор         2
мост          1
цирк          1
зоопарк       1
Name: count, dtype: int64

sampled_df[sampled_df['City'] == 'Нижний Новгород']
['attraction_type'].value_counts()

attraction_type
дом           4
башня         4
памятник      3
театр         3
собор         3
музей         3
парк          3
храм          3
церковь       3
монастырь     3
сад           3
стадион       1
Name: count, dtype: int64
```

Формирование расширенного контекста документов

Выведем формальное описание в файл в формате markdown и прогоним через LLM для перевода и насыщения контекста

```
sampled_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 120 entries, 0 to 119
```

```
Data columns (total 14 columns):
```

#	Column	Non-Null Count	Dtype
0	Num	120 non-null	int64
1	Name	120 non-null	object
2	WikiData	120 non-null	object
3	City	120 non-null	object
4	Lon	120 non-null	float64
5	Lat	120 non-null	float64
6	description	120 non-null	object
7	image	120 non-null	object
8	en_txt	120 non-null	object
9	is_junc_words	120 non-null	bool
10	found_text	0 non-null	object
11	attraction_type	120 non-null	object
12	description_length	120 non-null	int64
13	length_diff	119 non-null	float64

```
dtypes: bool(1), float64(3), int64(2), object(8)
```

```
memory usage: 13.2+ KB
```

```
sampled_df[
```

```
    ['City', 'attraction_type', 'Name', 'description', 'en_txt',  
    'Lon', 'Lat',  
    'WikiData']]
```

	City	attraction_type	Name \
0	Екатеринбург	дом	Доходный дом купца Чувильдина
1	Екатеринбург	дом	№29 Дом В.И. Чувильдина
2	Екатеринбург	дом	Дом Актёра
3	Екатеринбург	музей	Уральский геологический музей
4	Екатеринбург	музей	Музей истории Екатеринбурга
...	...	...	...
115	Ярославль	собор	Успенский собор
116	Ярославль	собор	Ярославская соборная мечеть
117	Ярославль	цирк	Цирк
118	Ярославль	зоопарк	Ярославский зоопарк
119	Нижний Новгород	башня	Дмитриевская башня

	description \
0	Дом купца Чувильдина
1	Дом купца Чувильдина
2	Дом актёра (Екатеринбург)
3	музей в Екатеринбурге
4	музей в Екатеринбурге
...	...
115	кафедральный собор в Ярославле

```

116          Ярославская соборная мечеть
117          Ярославский государственный цирк
118          Ярославский зоопарк
119 проездная башня Нижегородского кремля, один из...

```

```

                                en_txt      Lon
Lat \
0  there is a large building with a clock on the ... 60.607727
56.832176
1  there is a large building with a clock on top ... 60.607689
56.832268
2  nighttime shot of a building with stairs and t... 60.597950
56.838634
3  there is a large rock sitting on a wooden tabl... 60.595470
56.826492
4  arafed room with a large display of old photos... 60.611423
56.840691
..          ...          ...
...
115 arafed view of a large white building with gol... 39.901978
57.622429
116 there is a small pond in front of a building w... 39.875633
57.636059
117 there is a large building with a sign on the f... 39.865444
57.625584
118 there is a large teddy bear sitting on top of ... 39.895737
57.674446
119 people walking on the sidewalk near a brick bu... 44.005676
56.327045

```

```

      WikiData
0      Q27921892
1      Q27921892
2      Q4165449
3      Q4476747
4      Q4306294
..      ...
115    Q1140551
116    Q4281966
117    Q4538876
118    Q16723452
119    Q18406802

```

```
[120 rows x 8 columns]
```

```

def print_records_compact(df, file=None):
    """
    Компактная версия процедуры печати записей
    file: файловый объект или None (по умолчанию - консоль)
    """

```

```

print(f"\nВЫВОД ЗАПИСЕЙ (всего: {len(df)})", file=file)
print("=" * 60, file=file)

for index, row in df.iterrows():
    print(f"\n Индекс[{index}] {row['Name']}", file=file)
    print(f"    Город: {row['City']} | Тип: {row['attraction_type']}",
          file=file)
    print(f"    Координаты: {row['Lon']}, {row['Lat']}", file=file)
    print(f"    Описание: {row['description']}", file=file)
    print(f"    Описание изображения (анг): {row['en_txt']}",
          file=file)
    print(f"    Wikidata: {row['WikiData']}", file=file)

print_records_compact(sampled_df.head(5))

```

ВЫВОД ЗАПИСЕЙ (всего: 5)

=====

Индекс[0] Доходный дом купца Чувильдина
 Город: Екатеринбург | Тип: дом
 Координаты: 60.607727, 56.832176
 Описание: Дом купца Чувильдина
 Описание изображения (анг): there is a large building with a clock on the top of it
 Wikidata: Q27921892

Индекс[1] №29 Дом В.И. Чувильдина
 Город: Екатеринбург | Тип: дом
 Координаты: 60.607689, 56.832268
 Описание: Дом купца Чувильдина
 Описание изображения (анг): there is a large building with a clock on top of it
 Wikidata: Q27921892

Индекс[2] Дом Актёра
 Город: Екатеринбург | Тип: дом
 Координаты: 60.59795, 56.838634
 Описание: Дом актёра (Екатеринбург)
 Описание изображения (анг): nighttime shot of a building with stairs and trees in the foreground
 Wikidata: Q4165449

Индекс[3] Уральский геологический музей
 Город: Екатеринбург | Тип: музей
 Координаты: 60.59547, 56.826492
 Описание: музей в Екатеринбурге
 Описание изображения (анг): there is a large rock sitting on a wooden table in a room

Wikidata: Q4476747

Индекс[4] Музей истории Екатеринбурга

Город: Екатеринбург | Тип: музей

Координаты: 60.611423, 56.840691

Описание: музей в Екатеринбурге

Описание изображения (анг): a rafed room with a large display of old photos and other items

Wikidata: Q4306294

```
with open('output.txt', 'w', encoding='utf-8') as f:  
    print_records_compact(sampled_df, file=f)
```

Сырые данные прогоняем через LLM для перевода и насыщения.

Сделай описание по данным ниже для системы RAG. английский перевести на русский пример

Индекс: 34 Название: Царский мост Тип объекта: Мост Локация: Город Екатеринбург, Россия Координаты: 60.613163, 56.823517 Функция: Пересекает реку Исеть, обеспечивая автомобильное и пешеходное сообщение между районами города. Статус: Объект культурного наследия и важный исторический памятник. Идентификатор во внешней БД: Wikidata (Q4503250)

```
import pandas as pd  
  
def parse_file_simple(filename):  
    with open(filename, 'r', encoding='utf-8') as file:  
        content = file.read()  
  
    records = content.split('---')  
    data = []  
  
    for record in records:  
        record = record.strip()  
        if not record:  
            continue  
  
        lines = record.split('\n')  
        num = None  
        text_lines = []  
  
        for line in lines:  
            line = line.strip()  
            if line.startswith('**Индекс:'):   
                # Извлекаем число после "Индекс:"  
                num_str = line.replace('**Индекс:', '').replace('**',  
                '').strip()
```



```

        num = int(num_str)
    elif line:
        text_lines.append(line)

    if num is not None:
        data.append({'num': num, 'text': '\n'.join(text_lines)})

df = pd.DataFrame(data)
df = df.set_index('num')
return df

# Использование
dfs = parse_file_simple('whole_context.md')

dfs.head()

```

	num	text
0	**Название:** Доходный дом купца Чувильдина\n*	...
1	**Название:** №29 Дом В.И. Чувильдина\n**Тип о...	...
2	**Название:** Дом Актёра\n**Тип объекта:** Дом...	...
3	**Название:** Уральский геологический музей\n*	...
4	**Название:** Музей истории Екатеринбурга\n**Т...	...

```

dfs.info()

<class 'pandas.core.frame.DataFrame'>
Index: 120 entries, 0 to 119
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0    text    120 non-null     object
dtypes: object(1)
memory usage: 1.9+ KB

```

```

sampled_df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 120 entries, 0 to 119
Data columns (total 14 columns):
#   Column              Non-Null Count  Dtype
---  -
0    Num                  120 non-null    int64
1    Name                 120 non-null    object
2    WikiData             120 non-null    object
3    City                 120 non-null    object
4    Lon                  120 non-null    float64
5    Lat                  120 non-null    float64
6    description          120 non-null    object
7    image                120 non-null    object
8    en_txt               120 non-null    object

```

```

9   is_junc_words      120 non-null    bool
10  found_text         0 non-null    object
11  attraction_type     120 non-null  object
12  description_length  120 non-null  int64
13  length_diff        119 non-null  float64
dtypes: bool(1), float64(3), int64(2), object(8)
memory usage: 13.2+ KB

```

```

# Связываем датафреймы по индексу и добавляем нужные поля
dfs = dfs.join(sampled_df[['City', 'Name', 'Lon', 'Lat', 'image']])

dfs

```

```

                                text
City \
num
0   **Название:** Доходный дом купца Чувильдина\n*...
Екатеринбург
1   **Название:** №29 Дом В.И. Чувильдина\n**Тип о...
Екатеринбург
2   **Название:** Дом Актёра\n**Тип объекта:** Дом...
Екатеринбург
3   **Название:** Уральский геологический музей\n*...
Екатеринбург
4   **Название:** Музей истории Екатеринбурга\n**Т...
Екатеринбург
..
.
115 **Название:** Успенский собор\n**Тип объекта:*...
Ярославль
116 **Название:** Ярославская соборная мечеть\n**Т...
Ярославль
117 **Название:** Цирк\n**Тип объекта:** Цирк\n**Л...
Ярославль
118 **Название:** Ярославский зоопарк\n**Тип объек...
Ярославль
119 **Название:** Дмитриевская башня\n**Тип объект... Нижний
Новгород

```

	Name	Lon	Lat
num			
0	Доходный дом купца Чувильдина	60.607727	56.832176
1	№29 Дом В.И. Чувильдина	60.607689	56.832268
2	Дом Актёра	60.597950	56.838634
3	Уральский геологический музей	60.595470	56.826492
4	Музей истории Екатеринбурга	60.611423	56.840691
..	...	...	...
115	Успенский собор	39.901978	57.622429
116	Ярославская соборная мечеть	39.875633	57.636059


```
row["image"],
                                "Index": index}) for
                                index, row in dfs.iterrows()]
```

Для токенизации и создания эмбедингов будем использовать `cointegrated/rubert-tiny2` <https://huggingface.co/cointegrated/rubert-tiny2>

```
from sentence_transformers import SentenceTransformer

print(
    f"Model's maximum sequence length:
    {SentenceTransformer('cointegrated/rubert-tiny2').max_seq_length}")

Model's maximum sequence length: 2048
```

Предварительно этого окна достаточно для всех документов

```
# from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Мы используем иерархический список разделителей, специально
# предназначенных для разделения документов Markdown
# Этот список взят из класса MarkdownTextSplitter в LangChain

MARKDOWN_SEPARATORS = ["\n#{1,6} ", "`\n", "\n\\*\\*\\*+\\n", "\n---+\\n",
                        "\n__+\\n", "\n\\n", "\n", " ", "", ]

text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
                                                # Максимальное
количество символов в чанке
                                                chunk_overlap=100,
# Количество символов,
которые будут перекрываться между чанками
                                                add_start_index=True,
# Если "True", то
включает начальный индекс чанка в метаданные
                                                strip_whitespace=True,
# Если значение "True",
то пробелы удаляются из начала и конца каждого документа
#
separators=MARKDOWN_SEPARATORS,
)

docs_processed = []
for doc in RAW_KNOWLEDGE_BASE:
    docs_processed += text_splitter.split_documents([doc])
```

```

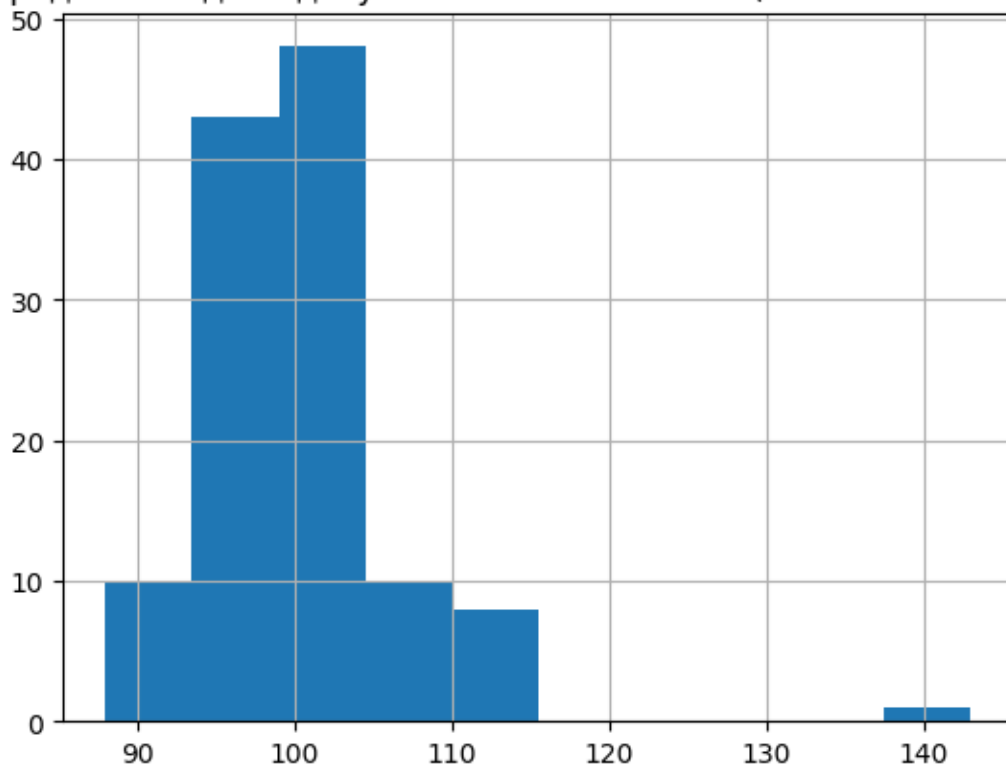
from matplotlib import pyplot as plt
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("cointegrated/rubert-tiny2")
lengths = [len(tokenizer.encode(doc.page_content)) for doc in docs_processed]

# Построим график распределения длин документов, подсчитываемых как количество токенов
fig = pd.Series(lengths).hist()
plt.title("Распределение длин документов в базе знаний (в количестве токенов)")
plt.show()

```

Распределение длин документов в базе знаний (в количестве токенов)



Основная процедура сплита документов

```

from langchain_text_splitters import RecursiveCharacterTextSplitter
from typing import Optional, List, Tuple

from transformers import AutoTokenizer

EMBEDDING_MODEL_NAME = "cointegrated/rubert-tiny2"

def split_documents(chunk_size: int, knowledge_base:

```

```

List[LangchainDocument],
            tokenizer_name: Optional[str] =
EMBEDDING_MODEL_NAME, ) -> \
    List[LangchainDocument]:
    """
    Разобьём документы на блоки максимального размера `chunk_size` и
    вернём список документов.
    """
    text_splitter =
RecursiveCharacterTextSplitter.from_huggingface_tokenizer(
    AutoTokenizer.from_pretrained(tokenizer_name),
    chunk_size=chunk_size,
    chunk_overlap=int(chunk_size / 10), add_start_index=True,
    strip_whitespace=True, # separators=MARKDOWN_SEPARATORS,
)

docs_processed = []
for doc in knowledge_base:
    docs_processed += text_splitter.split_documents([doc])

# Удалим дубли
unique_texts = {}
docs_processed_unique = []
for doc in docs_processed:
    if doc.page_content not in unique_texts:
        unique_texts[doc.page_content] = True
        docs_processed_unique.append(doc)

    return docs_processed_unique

docs_processed = split_documents(2048,
                                # Выбираем размер чанка,
                                # соответствующий модели
                                RAW_KNOWLEDGE_BASE,

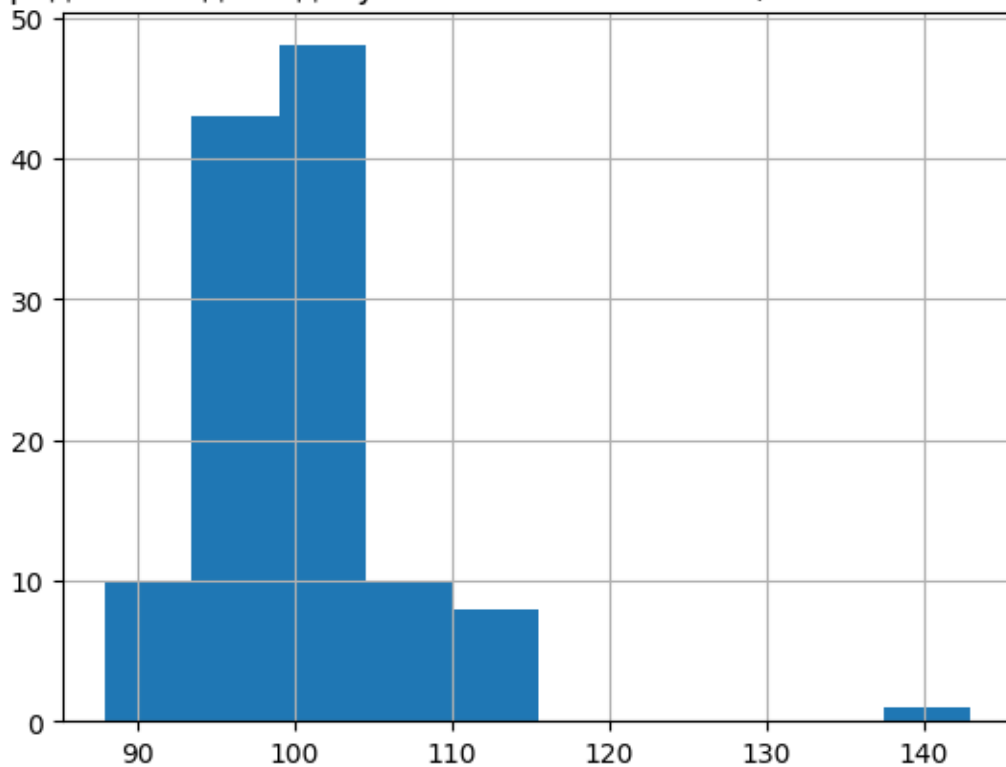
tokenizer_name=EMBEDDING_MODEL_NAME, )

# Представим размеры блоков, которые возможно получить в токенах из
общей модели
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(EMBEDDING_MODEL_NAME)
lengths = [len(tokenizer.encode(doc.page_content)) for doc in
docs_processed]
fig = pd.Series(lengths).hist()
plt.title("Распределение длин документов в базе знаний (в количестве
токенов)")
plt.show()

```

Распределение длин документов в базе знаний (в количестве токенов)



Обертка для langchain

```
from langchain_community.embeddings import HuggingFaceEmbeddings

embedding_model =
HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL_NAME,
                        multi_process=True,
                        # model_kwargs={"device":
"cuda"},
                        encode_kwargs={
                            "normalize_embeddings":
True},
                        # `True` для косинусного
сходства
                        )
```

Векторная база Chroma

```
from langchain_chroma import Chroma
from tqdm import tqdm

# Создаем базу данных
documents_with_progress = tqdm(docs_processed, desc="Creating
embeddings")
```

```

KNOWLEDGE_VECTOR_DATABASE = Chroma.from_documents(
    documents=list(documents_with_progress),
    embedding=embedding_model,
    persist_directory="./chroma_db",
    collection_metadata={"hnsw:space": "cosine"}
    # Указываем косинусное расстояние
)

KNOWLEDGE_VECTOR_DATABASE = Chroma(persist_directory="./chroma_db",
                                   embedding_function=embedding_model)
print("Vector database loaded from './chroma_db'")

Vector database loaded from './chroma_db'

# Получение информации о коллекции
print(f"Collection count:
{KNOWLEDGE_VECTOR_DATABASE._collection.count()}")

Collection count: 120

import chromadb

# Подключаемся к существующей базе
client = chromadb.PersistentClient(path="./chroma_db")

# Получаем список всех коллекций
collections = client.list_collections()
print(f"Найдено коллекций: {len(collections)}")

Найдено коллекций: 1

if False:
    for collection in collections:
        client.delete_collection(collection.name)
        print(f"Коллекция удалена: {collection.name}")

    print("Все коллекции удалены")

Коллекция удалена: langchain
Все коллекции удалены

```

Тестирование векторной БД

```

# Поиск похожих документов
results = KNOWLEDGE_VECTOR_DATABASE.similarity_search("дом в городе
владимир",
                                                    k=5)

for res in results:
    print('=' * 20)
    print(res.page_content)

```



```

=====
**Название:** Владимирский академический областной драматический театр
**Тип объекта:** Театр
**Локация:** Город Владимир, Россия
**Координаты:** 40.396076, 56.125618
**Описание:** театр города Владимира
**Описание изображения:** большое здание с множеством окон
**Идентификатор во внешней БД:** Wikidata (Q56230588)
=====
**Название:** Доходный дом купца Чувильдина
**Тип объекта:** Дом
**Локация:** Город Екатеринбург, Россия
**Координаты:** 60.607727, 56.832176
**Описание:** Дом купца Чувильдина
**Описание изображения:** большое здание с часами наверху
**Идентификатор во внешней БД:** Wikidata (Q27921892)
=====
**Название:** №29 Дом В.И. Чувильдина
**Тип объекта:** Дом
**Локация:** Город Екатеринбург, Россия
**Координаты:** 60.607689, 56.832268
**Описание:** Дом купца Чувильдина
**Описание изображения:** большое здание с часами наверху
**Идентификатор во внешней БД:** Wikidata (Q27921892)
=====
**Название:** Дом Виноградовых
**Тип объекта:** Дом
**Локация:** Город Нижний Новгород, Россия
**Координаты:** 43.987938, 56.323109
**Описание:** Дом Виноградовых
**Описание изображения:** размытое здание с часами наверху
**Идентификатор во внешней БД:** Wikidata (Q84340772)
=====
**Название:** Дом-музей Столетовых
**Тип объекта:** Музей
**Локация:** Город Владимир, Россия
**Координаты:** 40.413654, 56.132446
**Описание:** Мемориальный музей в городе Владимире, посвящённый жизни
и деятельности выдающегося учёного-физика А. Г. Столетова и генерала
от инфантерии Н. Г. Столетова, сыгравшего видную роль в освобождении
Болгарии от турецкого ига
**Описание изображения:** размытый мужчина стоит перед синим зданием с
белой дверью
**Идентификатор во внешней БД:** Wikidata (Q107422934)

user_query = "дом в городе владимир"
query_vector = embedding_model.embed_query(user_query)

len(docs_processed)

```

120

docs_processed[0].page_content

```
'**Название:** Доходный дом купца Чувильдина\n**Тип объекта:** Дом\n\n**Локация:** Город Екатеринбург, Россия\n\n**Координаты:** 60.607727, 56.832176\n\n**Описание:** Дом купца Чувильдина\n\n**Описание изображения:** большое здание с часами наверху\n\n**Идентификатор во внешней БД:** Wikidata (Q27921892)'
```

В выборке есть дома из Владимира, но не на первом месте. Необходимо их отранжировать.

Визуализация PCA

Выполните визуализацию эмбедингов при помощи метода главных компонент (PCA) и Uniform Manifold Approximation and Projection (UMAP). Пожалуйста, выполните PCA независимо от расстар.

```
def get_all_embeddings_from_db(vectorstore):  
    """Быстро получает эмбединги напрямую из базы данных"""  
    try:  
        # Пытаемся получить коллекцию и эмбединги  
        collection = vectorstore._collection  
        results = collection.get(include=['embeddings'])  
  
        if 'embeddings' in results and results['embeddings']:  
            embeddings_array = np.array(results['embeddings'])  
            print(f"Загружено {len(embeddings_array)} эмбедингов из  
базы")  
            return embeddings_array  
        else:  
            print("Эмбединги не найдены в базе, вычисляем заново...")  
            return None  
    except Exception as e:  
        print(f"Ошибка при загрузке эмбедингов из базы: {e}")  
        return None  
  
def get_all_embeddings_fast(vectorstore, docs_processed,  
query_vector=None):  
    """Оптимизированная версия получения эмбедингов"""  
    # Пытаемся загрузить из базы  
    embeddings_array = get_all_embeddings_from_db(vectorstore)  
  
    # Если не получилось, вычисляем с оптимизацией  
    if embeddings_array is None:  
        print("Вычисляем эмбединги...")  
        doc_embeddings = []
```

```

# Обрабатываем батчами для ускорения
batch_size = 32
for i in tqdm(range(0, len(docs_processed), batch_size)):
    batch_docs = docs_processed[i:i + batch_size]
    batch_texts = [doc.page_content for doc in batch_docs]

    try:
        batch_embeddings =
vectorstore._embedding_function.embed_documents(
            batch_texts)
        doc_embeddings.extend(batch_embeddings)
    except Exception as e:
        print(f"Ошибка в батче {i}: {e}")
        continue

    embeddings_array = np.array(doc_embeddings)

    return embeddings_array

all_embeddings = get_all_embeddings_fast(KNOWLEDGE_VECTOR_DATABASE,
                                         docs_processed, query_vector)

```

Ошибка при загрузке эмбедингов из базы: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()
 Вычисляем эмбединги...

```
100%|██████████| 4/4 [01:26<00:00, 21.66s/it]
```

```

query_array = np.array(query_vector)
if len(query_array.shape) == 1:
    query_array = query_array.reshape(1, -1)
embeddings_array = np.vstack([all_embeddings, query_array])

embeddings_array.shape

(121, 312)

```

```

# Создаем метки для точек
labels = [f"Doc_{i}_{docs_processed[i].metadata['City']}" for i in
          range(len(docs_processed))]

```

```

# Определяем тип точек
doc_types = ['document'] * len(docs_processed)

```

```

# Если есть вектор запроса, добавляем его к данным
if query_vector is not None:
    labels.append("Query")
    doc_types.append('query')

```

```

# Проверяем соответствие размеров
print(f"Документов: {len(docs_processed)}")

```

```
print(f"Query присутствует: {query_vector is not None}")
print(f"Размер all_embeddings: {embeddings_array.shape}")
print(f"Длина labels: {len(labels)}")
print(f"Длина doc_types: {len(doc_types)}")
```

Документов: 120

Query присутствует: True

Размер all_embeddings: (121, 312)

Длина labels: 121

Длина doc_types: 121

Метод 1: PCA визуализация

```
print("=== PCA Визуализация ===")
```

```
pca = PCA(n_components=2, random_state=42)
```

```
embeddings_pca = pca.fit_transform(embeddings_array)
```

Создаем DataFrame для Plotly

```
pca_df = pd.DataFrame(
    {'x': embeddings_pca[:, 0], 'y': embeddings_pca[:, 1], 'label':
     labels,
     'type': doc_types})
```

Визуализация PCA

```
fig_pca = px.scatter(pca_df, x='x', y='y', color='type',
                    title='PCA Projection of Embeddings',
                    labels={'x': 'PC1', 'y': 'PC2'},
                    hover_data=['label'],
                    color_discrete_map={'document': 'blue', 'query':
                                         'red'})
fig_pca.update_traces(marker=dict(size=10, opacity=0.7))
fig_pca.show()
```

=== PCA Визуализация ===

```
{
  "config": {
    "plotlyServerURL": "https://plot.ly"
  },
  "data": [
    {
      "customdata": [
        ["Doc_0_Екатеринбург"],
        ["Doc_1_Екатеринбург"],
        ["Doc_2_Екатеринбург"],
        ["Doc_3_Екатеринбург"],
        ["Doc_4_Екатеринбург"],
        ["Doc_5_Екатеринбург"],
        ["Doc_6_Екатеринбург"],
        ["Doc_7_Екатеринбург"],
        ["Doc_8_Екатеринбург"],
        ["Doc_9_Екатеринбург"],
        ["Doc_10_Екатеринбург"],
        ["Doc_11_Екатеринбург"],
        ["Doc_12_Екатеринбург"],
        ["Doc_13_Екатеринбург"],
        ["Doc_14_Екатеринбург"],
        ["Doc_15_Екатеринбург"],
        ["Doc_16_Екатеринбург"],
        ["Doc_17_Екатеринбург"],
        ["Doc_18_Екатеринбург"],
        ["Doc_19_Екатеринбург"],
        ["Doc_20_Екатеринбург"],
        ["Doc_21_Екатеринбург"],
        ["Doc_22_Екатеринбург"],
        ["Doc_23_Екатеринбург"],
        ["Doc_24_Екатеринбург"],
        ["Doc_25_Екатеринбург"],
        ["Doc_26_Екатеринбург"],
        ["Doc_27_Екатеринбург"],
        ["Doc_28_Екатеринбург"],
        ["Doc_29_Екатеринбург"],
        ["Doc_30_Екатеринбург"],
        ["Doc_31_Екатеринбург"],
        ["Doc_32_Нижний Новгород"],
        ["Doc_33_Нижний Новгород"]
      ]
    }
  ]
}
```

```
Новгород"],["Дос_34_Нижний Новгород"],["Дос_35_Нижний Новгород"],
["Дос_36_Нижний Новгород"],["Дос_37_Нижний Новгород"],["Дос_38_Нижний
Новгород"],["Дос_39_Нижний Новгород"],["Дос_40_Нижний Новгород"],
["Дос_41_Нижний Новгород"],["Дос_42_Нижний Новгород"],["Дос_43_Нижний
Новгород"],["Дос_44_Нижний Новгород"],["Дос_45_Нижний Новгород"],
["Дос_46_Нижний Новгород"],["Дос_47_Нижний Новгород"],["Дос_48_Нижний
Новгород"],["Дос_49_Нижний Новгород"],["Дос_50_Нижний Новгород"],
["Дос_51_Нижний Новгород"],["Дос_52_Нижний Новгород"],["Дос_53_Нижний
Новгород"],["Дос_54_Нижний Новгород"],["Дос_55_Нижний Новгород"],
["Дос_56_Нижний Новгород"],["Дос_57_Нижний Новгород"],["Дос_58_Нижний
Новгород"],["Дос_59_Нижний Новгород"],["Дос_60_Нижний Новгород"],
["Дос_61_Нижний Новгород"],["Дос_62_Нижний Новгород"],["Дос_63_Нижний
Новгород"],["Дос_64_Нижний Новгород"],["Дос_65_Нижний Новгород"],
["Дос_66_Нижний Новгород"],["Дос_67_Владимир"],["Дос_68_Владимир"],
["Дос_69_Владимир"],["Дос_70_Владимир"],["Дос_71_Владимир"],
["Дос_72_Владимир"],["Дос_73_Владимир"],["Дос_74_Владимир"],
["Дос_75_Владимир"],["Дос_76_Владимир"],["Дос_77_Владимир"],
["Дос_78_Владимир"],["Дос_79_Владимир"],["Дос_80_Владимир"],
["Дос_81_Владимир"],["Дос_82_Владимир"],["Дос_83_Владимир"],
["Дос_84_Владимир"],["Дос_85_Владимир"],["Дос_86_Владимир"],
["Дос_87_Ярославль"],["Дос_88_Ярославль"],["Дос_89_Ярославль"],
["Дос_90_Ярославль"],["Дос_91_Ярославль"],["Дос_92_Ярославль"],
["Дос_93_Ярославль"],["Дос_94_Ярославль"],["Дос_95_Ярославль"],
["Дос_96_Ярославль"],["Дос_97_Ярославль"],["Дос_98_Ярославль"],
["Дос_99_Ярославль"],["Дос_100_Ярославль"],["Дос_101_Ярославль"],
["Дос_102_Ярославль"],["Дос_103_Ярославль"],["Дос_104_Ярославль"],
["Дос_105_Ярославль"],["Дос_106_Ярославль"],["Дос_107_Ярославль"],
["Дос_108_Ярославль"],["Дос_109_Ярославль"],["Дос_110_Ярославль"],
["Дос_111_Ярославль"],["Дос_112_Ярославль"],["Дос_113_Ярославль"],
["Дос_114_Ярославль"],["Дос_115_Ярославль"],["Дос_116_Ярославль"],
["Дос_117_Ярославль"],["Дос_118_Ярославль"],["Дос_119_Нижний
Новгород"]], "hovertemplate": "type=document<br>PC1=%{x}<br>PC2=%
{y}<br>label=%{customdata[0]}<extra></
extra>", "legendgroup": "document", "marker":
{"color": "blue", "opacity": 0.7, "size": 10, "symbol": "circle"}, "mode": "mar
kers", "name": "document", "orientation": "v", "showlegend": true, "type": "sc
atter", "x": {"bdata": "GuucIV1So7/StlxtJRuav9z8iC2XoM0/
23f5fphDwr+1tXyFTBrJv8YQR/vDAca/7KWZgZVUyL+6SVB/GZTJvxqyri2MpMS/
zMMqQCD+x78kUMF6kp7HP9yG3aMuysQ/tiz747s0wz/seNVHT8y8P/v82V1768A/
IeOCjDXgwT/Gm7A09NTAv6c0n0egR80/ts0cCbHTu79b/v7Gd+Cnv3GTxyA8bqm/
cwdAkZd0tb8rmRJG5a3Nv/bkjLF6IcU/p3xakqrexT/m3jgruCqiv074LH1Sd6a/
Z0Q903ZrxD+eusQiGDvEv7ozsQkqI8K/oVSus9jhv7+oTtmln8/Hv805zLeI3qm/
uy9Af83Io7/2ETMLBDBWv6pN26p/DVkJHk+Faofrr79ZmWUwQauzvW6hSK9+KK/
Rakw5VWctT94QR+E27qkPxZu2EbTTRI/YAGpiMFswL/Qk9oKECjCvzMeipILccS/
xyFH2c2+yz/p8RrjgF3BP1Ll9e9rGcg/dslfx8RHvb+0F0HMK0eav6fj4IlnZ62/
slwkVlRRy7/1b1QVwRzGv5VMB84QXM0/0wXeZ0ZLzz9L2yxPI+bQP1TcZMvE2cs/
yl4thzx50j8m8D0qbv7LPzVmMV7E9c8/fgMkv6Krwz/hY4ll9izKP5p9/ExcDsY/
C1tFtdQZkr+tP1fDwEynv/sJn017XIY/ojeUfoGRwL/g5F82Bv7LP03kDyeRh8I//
Au7ZWc6uD/E+JZu0+rIP4PvK0+1r8W/phro+ayAtb+icQYNm0ayv64h21QAtbC/
```

```
HTDgcCnY0D9LZ6kqp2XTPzQvBM0YJ8g/lku4D+CQwj/WDa4x3rHJv9wkW1UrVcg/
K7fdVV1SxT/qoTJxdn00v9UqDQ88tJS/NZou5eynsb/xSFtxCHGMv7VfmHZ8pcW/
xVzMq84txj8DRaTBPXMp35FJfQ0E8s/yIM4hsR4wz9LDUIiaSjMP3NBz1TWtck/
uN6ZkUDvsb9UMtK+b9DDP1i478PdJ6m/ZK8wzhNYXj/tkDiQze2Dvy8kfeXWQa0/
rqjozNx3vL9giNsJkiTAv+rm0RCnPe8/0zioJJ0qvr+lqA27QRy1v2iiejKT3cg/
mygw1GMGvj+lw3Jl60y7PzYYuXcmpcK/AzR6KzWfwr/708fTlxnAvwYmMmqmIsC/
jv7SbPfYyr/0p3g5UwbLvXQk/jZNzcu/MCVG34cEz7+ggZbA5XPBP7l02790yrw/
eroB4i3LuL8oVVsd1rzLv1liobrHdK8/", "dtype": "f8"}, {"xaxis": "x", "y":
{"bdata": "19wbVfbEnz/9Qh3be/VFvw6/Fojjoao/EJmTXy+lwT+4Hm1ZMX/
FPwvNcnu/ap4/LMNSkkUijw9Cz/RWbJDAPxn7fQvJn8Q/o43iflX5tz/
3EmjMmVLHPxk0eg01Wcg/r0BIkdZeyj9F0MpcnyPJP3pFqwca4MQ/
0LJzWFIowj+6McxQyq60P20RKJ0+m78/6YwlTwEwj9uej0FbmVJP/SFvIukPck/
XPtHhu+mvj/HdK3S81PAP6Wy7fIxWls/l8qkqK7Vtz/mELru49axP/mnMKHn0bI/
IodnHrHzjT8Vru3Vz03CP9Dt+MY0Krw/6HEIPl2wj/zFnT5Dr7FP6iSNwhEF80/
FfmBq7cfxL9zaBiyieLHv5MODinmf8a/hLEo2Ex4vb8Tje4R6DPBvzkSxyx6xMi/
Cn2VLuEgwr/6JdjLD2PFv8+i/Z1m27u/dSZAeZ4Wxb9Mka1Y/ynBv7B8EdTLyMu/
AViMSiyrTL+KoPp6dM01v7oVPEblu7q/4Z7bnF0Ryr9UiWvIrXvNvzTTDz5o8Mu/
uR0Evsflvr+bQpTsa9DCv/MKq0DJTsS/TGQoTEiKub/ay0QkuLS3v8IwTXix9b2/
NPxu+9g2wr/VVBA0N/KNv50bIHyaBqG//FJYDjmQwr/cs0NRAY67vxqKJs2LMG/
o5D4Dg5Dxr/dXCzhsHzIv6b0RFTwfby/Ae9UMgd9zb/yHHL4E+6cv2918yAUNKc/
zgkJzA4jsj+diUt6ZfuEP+HEXMc1k5Y/2rve8Kjyhr/77cnkCsuWv6FYsHlQlrc/
TfwjYRaNnj/y5YSR77aJPzdJHNKR2qs/G3HASNYDrj8qq/DVfjCcP39Nb4adl3K/
n3g4ekcfnd9UeJo7QyaaP9QIC6nrQWg/0hPAWdGloz+6osdU86x9P8uZsj4m1ba/
CVT3F0hCvD9yPBjEBACnPyWdxLevErI/j/+mjtDxqT8+yrh6jz0tP0vyN0EuY7o/
uRWKW8kgdj+8+z+UIp+4P2CYI3lcGMG/m7JBNi08sT+by3WcrIurP17+JAl9aKc/
si0x+gD6or8kbKZwLDezPzz06l4qs4S/eXERTj1Qs79PlYpH9yS/v3yfx5zD+LE/
5LC1gg7Esb8hh6GYRluxPyzyU7BfCHg/r+0H9i0ZrD9mc10K0Z6mPzUL98Zv74Y/
+1cmN6chsD/bs/2N01a0P6hIy9SkqZE/PKfV24m+p79ktf0P2N08P2MQ4E+jQsA/
qZQnn5X5uz/AnsB9FvV3v39XZN5pQ8a/", "dtype": "f8"}, {"yaxis": "y"},
{"customdata": [{"Query"}], "hovertemplate": "type=query<br>PC1=%
{x}<br>PC2=%{y}<br>label=%{customdata[0]}<extra></
extra>", "legendgroup": "query", "marker":
{"color": "red", "opacity": 0.7, "size": 10, "symbol": "circle"}, "mode": "mark
ers", "name": "query", "orientation": "v", "showlegend": true, "type": "scatte
r", "x": {"bdata": "5KdAwBymwL8=", "dtype": "f8"}, "xaxis": "x", "y":
{"bdata": "ckwyx0zAmz8=", "dtype": "f8"}, "yaxis": "y"}], "layout":
{"legend": {"title": {"text": "type"}, "tracegroupgap": 0}, "template":
{"data": {"bar": [{"error_x": {"color": "#2a3f5f"}, "error_y":
{"color": "#2a3f5f"}, "marker": {"line":
{"color": "#E5ECF6", "width": 0.5}, "pattern":
{"fillmode": "overlay", "size": 10, "solidity": 0.2}}, "type": "bar"}], "barpo
lar": [{"marker": {"line": {"color": "#E5ECF6", "width": 0.5}, "pattern":
{"fillmode": "overlay", "size": 10, "solidity": 0.2}}, "type": "barpolar"}], "
carpet": [{"aaxis":
{"endlinecolor": "#2a3f5f", "gridcolor": "white", "linecolor": "white", "min
orgridcolor": "white", "startlinecolor": "#2a3f5f"}, "baxis":
{"endlinecolor": "#2a3f5f", "gridcolor": "white", "linecolor": "white", "min
orgridcolor": "white", "startlinecolor": "#2a3f5f"}, "type": "carpet"}], "ch
oropleth": [{"colorbar":
```



```

{"linewidth":0,"ticks":"","type":"choropleth"},"contour":
[{"colorbar":{"linewidth":0,"ticks":"","colorscale":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1.0,"#f0f921"]],"type":"contour"},"contourcarpet":{"colorbar":
{"linewidth":0,"ticks":"","type":"contourcarpet"},"heatmap":
{"colorbar":{"linewidth":0,"ticks":"","colorscale":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1.0,"#f0f921"]],"type":"heatmap"},"histogram":{"marker":{"pattern":
{"fillmode":"overlay","size":10,"solidity":0.2},"type":"histogram"}},
"histogram2d":{"colorbar":{"linewidth":0,"ticks":"","colorscale":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1.0,"#f0f921"]],"type":"histogram2d"},"histogram2dcontour":
{"colorbar":{"linewidth":0,"ticks":"","colorscale":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1.0,"#f0f921"]],"type":"histogram2dcontour"},"mesh3d":{"colorbar":
{"linewidth":0,"ticks":"","type":"mesh3d"},"parcoords":{"line":
{"colorbar":{"linewidth":0,"ticks":"","type":"parcoords"},"pie":
[{"automargin":true,"type":"pie"},"scatter":{"fillpattern":
{"fillmode":"overlay","size":10,"solidity":0.2},"type":"scatter"},"scatter3d":
[{"line":{"colorbar":{"linewidth":0,"ticks":"","type":"scatter3d"},"marker":
{"colorbar":
{"linewidth":0,"ticks":"","type":"scatter3d"},"scattercarpet":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scattercarpet"},"scattergeo":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scattergeo"},"scattergl":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scattergl"},"scattermap":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scattermap"},"scattermapbox":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scattermapbox"},"scatterpolar":
[{"marker":{"colorbar":

```

```

{"linewidth":0,"ticks":"","type":"scatterpolar"},"scatterpolargl":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scatterpolargl"},"scatterternary":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scatterternary"},"surface":
[{"colorbar":{"linewidth":0,"ticks":"","colorscale":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1.0,"#f0f921"]],"type":"surface"},"table":{"cells":{"fill":
{"color":"#EBF0F8"},"line":{"color":"white"},"header":{"fill":
{"color":"#C8D4E3"},"line":
{"color":"white"},"type":"table"}}},"layout":{"annotationdefaults":
{"arrowcolor":"#2a3f5f","arrowhead":0,"arrowwidth":1},"autotypenumbers":
"strict","coloraxis":{"colorbar":
{"linewidth":0,"ticks":"","colorscale":{"diverging":
[[0,"#8e0152"],[0.1,"#c51b7d"],[0.2,"#de77ae"],[0.3,"#f1b6da"],
[0.4,"#fde0ef"],[0.5,"#f7f7f7"],[0.6,"#e6f5d0"],[0.7,"#b8e186"],
[0.8,"#7fbcb4"],[0.9,"#4d9221"],[1,"#276419"]],"sequential":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]],"sequentialminus":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]]},"colorway":
["#636efa","#EF553B","#00cc96","#ab63fa","#FFA15A","#19d3f3","#FF6692",
"#B6E880","#FF97FF","#FECB52"],"font":{"color":"#2a3f5f"},"geo":
{"bgcolor":"white","lakecolor":"white","landcolor":"#E5ECF6","showlakes":
true,"showland":true,"subunitcolor":"white"},"hoverlabel":
{"align":"left"},"hovermode":"closest","mapbox":
{"style":"light"},"paper_bgcolor":"white","plot_bgcolor":"#E5ECF6","polar":
{"angularaxis":
{"gridcolor":"white","linecolor":"white","ticks":"","bgcolor":"#E5ECF6",
"radialaxis":
{"gridcolor":"white","linecolor":"white","ticks":"","type":"polar"},"scene":
{"xaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"linecolor":
"white","showbackground":true,"ticks":"","zerolinecolor":"white"},
"yaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"linecolor":
"white","showbackground":true,"ticks":"","zerolinecolor":"white"},
"zaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"linecolor":

```



```

lor:"white", "showbackground":true, "ticks":"","zerolinecolor":"white"}
}, "shapedefaults":{"line":{"color":"#2a3f5f"}}, "ternary":{"aaxis":
{"gridcolor":"white", "linecolor":"white", "ticks":""}, "baxis":
{"gridcolor":"white", "linecolor":"white", "ticks":""}, "bgcolor":"#E5ECF
6", "caxis":
{"gridcolor":"white", "linecolor":"white", "ticks":""}}, "title":
{"x":5.0e-2}, "xaxis":
{"automargin":true, "gridcolor":"white", "linecolor":"white", "ticks":"","
"title":
{"standoff":15}, "zerolinecolor":"white", "zerolinewidth":2}, "yaxis":
{"automargin":true, "gridcolor":"white", "linecolor":"white", "ticks":"","
"title":
{"standoff":15}, "zerolinecolor":"white", "zerolinewidth":2}}}, "title":
{"text":"PCA Projection of Embeddings"}, "xaxis":
{"anchor":"y", "domain":[0.0,1.0], "title":{"text":"PC1"}}, "yaxis":
{"anchor":"x", "domain":[0.0,1.0], "title":{"text":"PC2"}}}}

```

Визуализация TSNE

```

from sklearn.manifold import TSNE

# Метод 2: t-SNE визуализация
print("=== t-SNE Визуализация ===")
tsne = TSNE(n_components=2, random_state=42,
            perplexity=min(30, len(embeddings_array) - 1))
embeddings_tsne = tsne.fit_transform(embeddings_array)

# Создаем DataFrame для t-SNE
tsne_df = pd.DataFrame(
    {'x': embeddings_tsne[:, 0], 'y': embeddings_tsne[:, 1], 'label':
labels,
    'type': ['document'] * len(docs_processed) + (
        ['query'] if query_vector is not None else [])})

# Визуализация t-SNE
fig_tsne = px.scatter(tsne_df, x='x', y='y', color='type',
                    title='t-SNE Projection of Embeddings',
                    labels={'x': 't-SNE1', 'y': 't-SNE2'},
                    hover_data=['label'],
                    color_discrete_map={'document': 'green',
'query': 'red'})
fig_tsne.update_traces(marker=dict(size=10, opacity=0.7))
fig_tsne.show()

=== t-SNE Визуализация ===

{"config":{"plotlyServerURL":"https://plot.ly"},"data":[{"customdata":
[["Doc_0"],["Doc_1"],["Doc_2"],["Doc_3"],["Doc_4"],["Doc_5"],
[["Doc_6"],["Doc_7"],["Doc_8"],["Doc_9"],["Doc_10"],["Doc_11"],
[["Doc_12"],["Doc_13"],["Doc_14"],["Doc_15"],["Doc_16"],["Doc_17"],

```

```
["Doc_18"], ["Doc_19"], ["Doc_20"], ["Doc_21"], ["Doc_22"], ["Doc_23"],  
["Doc_24"], ["Doc_25"], ["Doc_26"], ["Doc_27"], ["Doc_28"], ["Doc_29"],  
["Doc_30"], ["Doc_31"], ["Doc_32"], ["Doc_33"], ["Doc_34"], ["Doc_35"],  
["Doc_36"], ["Doc_37"], ["Doc_38"], ["Doc_39"], ["Doc_40"], ["Doc_41"],  
["Doc_42"], ["Doc_43"], ["Doc_44"], ["Doc_45"], ["Doc_46"], ["Doc_47"],  
["Doc_48"], ["Doc_49"], ["Doc_50"], ["Doc_51"], ["Doc_52"], ["Doc_53"],  
["Doc_54"], ["Doc_55"], ["Doc_56"], ["Doc_57"], ["Doc_58"], ["Doc_59"],  
["Doc_60"], ["Doc_61"], ["Doc_62"], ["Doc_63"], ["Doc_64"], ["Doc_65"],  
["Doc_66"], ["Doc_67"], ["Doc_68"], ["Doc_69"], ["Doc_70"], ["Doc_71"],  
["Doc_72"], ["Doc_73"], ["Doc_74"], ["Doc_75"], ["Doc_76"], ["Doc_77"],  
["Doc_78"], ["Doc_79"], ["Doc_80"], ["Doc_81"], ["Doc_82"], ["Doc_83"],  
["Doc_84"], ["Doc_85"], ["Doc_86"], ["Doc_87"], ["Doc_88"], ["Doc_89"],  
["Doc_90"], ["Doc_91"], ["Doc_92"], ["Doc_93"], ["Doc_94"], ["Doc_95"],  
["Doc_96"], ["Doc_97"], ["Doc_98"], ["Doc_99"], ["Doc_100"], ["Doc_101"],  
["Doc_102"], ["Doc_103"], ["Doc_104"], ["Doc_105"], ["Doc_106"],  
["Doc_107"], ["Doc_108"], ["Doc_109"], ["Doc_110"], ["Doc_111"],  
["Doc_112"], ["Doc_113"], ["Doc_114"], ["Doc_115"], ["Doc_116"],  
["Doc_117"], ["Doc_118"],  
["Doc_119"]], "hovernment": "type=document<br>t-SNE1=%{x}<br>t-SNE2=%  
{y}<br>label=%{customdata[0]}<extra></  
extra>", "legendgroup": "document", "marker":  
{"color": "green", "opacity": 0.7, "size": 10, "symbol": "circle"}, "mode": "ma  
rkers", "name": "document", "orientation": "v", "showlegend": true, "type": "s  
catter", "x":  
{"bdata": "CI62QFnAs0DYF9VA5SYHQRF3BkFq+P1AtmgfQXPUGFeJCFBwZd7Q09sh8C1  
NIbAjAq0wGa9pMDiDafAYmp3wB041kB2NctA8uLdQALuD0Hcag9BLFLXQCDCUHWKE/  
AQBxTwEmLqkB4vJ1Aw96rv4kMwEB1K7ZAZSMpQW3ZKEHYVLZApTypQPl6cUCuUpFAxpAsw  
GSZYkAzhPW/CFBkQPV0akBBelxAPVX5QMKME0GPegtBGCbdPV4RLD6BWuI/  
a5gCQaNisj+Pwa4/1oAvQP55AkBGrTJANTFdv95NU7+a5Ik/CU+0vqS4P7/LmsW/  
4T6qPz6KgJ+R6pU/VNmmQAlrdkCX7ZhA1cK0QEdRoMBiXZfArgWpwHEjvsC4W9u/  
J7f0v5z+3L/7s2y/  
IRrbwLee2MANIMDAFC2pwJ6HMEGoF6TAXqyBwHfmgMAaDInAygKJwGVEqMBFxbM/  
+5IFwHhn08Cwz03AwcIXwAIzHsAAAnFfA5f2zP0UFhsDhorU+0yU0QNj0NUBnCUJAwk3uP1  
3QN0CtgRNA5BLPv8eb1L8zYb+/9uEYwAPzIcDTreLAPx5XPswmT2/  
lPk+48EKQB7BFkBhQSRBNsAnQb0enb+W+6i/  
i0VqQDN1aD8DgEtA", "dtype": "f4"}, "xaxis": "x", "y": {"bdata": "/  
SBNv37zPL+QZmbAsJmlwH0bsMCKf5TAYTL4v0cWFSBpzATA8YfkWAKXxEANAtFATUTvQBU  
i5UAB1s9AWwvxQP1p2sDls9fA0wPkwIjeCMEKcQnBijwbwLePUsAyy85AgbKwQMDNMD10  
1rAnpm6QMAUmMA/m5PAQnSnwBkFqcBHU/M/  
MQ0pQHgCqT8PByVAWE66wGFZxj1357nATHybQD6ogkCAsJBAJrsXP+4/Gz1nQUM/  
0PXRQArmsUCgUc9A+4gCQMrSeD8P6W4/  
Bm8Jwd80AsEl1wzBoH8IqeqqCkFHq+dAxwAXQbbd3kBVR01AQmqwQG2Yw0D7Lq5APWCHP4  
MqHz+d+q4/HC4ewSi7i0DEJBLADbE4P6mBSkB0QzrA2341wMSTBcDiOmW/  
5HFjQCgZLEBrxwJAjEn0PyVpCT8Sw09A0m6TQ0qjKsBzQzXAZPiDv1pGEb/  
OzgrBWjg0QM02lECf6VLAK9diQDbgeUAhNUdA4tsTwNRF7z8ACYa/  
mJ8AwEDYI8CiBgrAhybgwNPTsMBSuX7AobblwPcp8cCb6ohAd6AFQfuL6T+l2ITA6IduwK  
d5YMDY0UHAPHnCWiwq18B6GHm/  
6PqKvX2dU0CLAAdAmxdHwGFQssDu52RA", "dtype": "f4"}, "yaxis": "y"},  
{"customdata": [{"Query"}], "hovernment": "type=query<br>t-SNE1=%
```

```

{x}<br>t-SNE2=%{y}<br>label=%{customdata[0]}<extra></
extra>", "legendgroup": "query", "marker":
{"color": "red", "opacity": 0.7, "size": 10, "symbol": "circle"}, "mode": "mark
ers", "name": "query", "orientation": "v", "showlegend": true, "type": "scatte
r", "x": {"bdata": "bY8qQ0==", "dtype": "f4"}, "xaxis": "x", "y":
{"bdata": "+48pQA==", "dtype": "f4"}, "yaxis": "y"}], "layout": {"legend":
{"title": {"text": "type"}, "tracegroupgap": 0}, "template": {"data": {"bar":
[{"error_x": {"color": "#2a3f5f"}, "error_y":
{"color": "#2a3f5f"}, "marker": {"line":
{"color": "#E5ECF6", "width": 0.5}, "pattern":
{"fillmode": "overlay", "size": 10, "solidity": 0.2}}, "type": "bar"}], "barpo
lar": [{"marker": {"line": {"color": "#E5ECF6", "width": 0.5}, "pattern":
{"fillmode": "overlay", "size": 10, "solidity": 0.2}}, "type": "barpolar"}], "
carpet": [{"aaxis":
{"endlinecolor": "#2a3f5f", "gridcolor": "white", "linecolor": "white", "min
orgridcolor": "white", "startlinecolor": "#2a3f5f"}, "baxis":
{"endlinecolor": "#2a3f5f", "gridcolor": "white", "linecolor": "white", "min
orgridcolor": "white", "startlinecolor": "#2a3f5f"}, "type": "carpet"}], "ch
oropleth": [{"colorbar":
{"outlinewidth": 0, "ticks": ""}, "type": "choropleth"}], "contour":
[{"colorbar": {"outlinewidth": 0, "ticks": ""}, "colorscale":
[[0.0, "#0d0887"], [0.1111111111111111, "#46039f"],
[0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"],
[0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"],
[0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"],
[0.8888888888888888, "#fdca26"],
[1.0, "#f0f921"]], "type": "contour"}], "contourcarpet": [{"colorbar":
{"outlinewidth": 0, "ticks": ""}, "type": "contourcarpet"}], "heatmap":
[{"colorbar": {"outlinewidth": 0, "ticks": ""}, "colorscale":
[[0.0, "#0d0887"], [0.1111111111111111, "#46039f"],
[0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"],
[0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"],
[0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"],
[0.8888888888888888, "#fdca26"],
[1.0, "#f0f921"]], "type": "heatmap"}], "histogram": [{"marker": {"pattern":
{"fillmode": "overlay", "size": 10, "solidity": 0.2}}, "type": "histogram"}],
"histogram2d": [{"colorbar": {"outlinewidth": 0, "ticks": ""}, "colorscale":
[[0.0, "#0d0887"], [0.1111111111111111, "#46039f"],
[0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"],
[0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"],
[0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"],
[0.8888888888888888, "#fdca26"],
[1.0, "#f0f921"]], "type": "histogram2d"}], "histogram2dcontour":
[{"colorbar": {"outlinewidth": 0, "ticks": ""}, "colorscale":
[[0.0, "#0d0887"], [0.1111111111111111, "#46039f"],
[0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"],
[0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"],
[0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"],
[0.8888888888888888, "#fdca26"],

```

```

[1.0,"#f0f921"]], "type": "histogram2dcontour"}], "mesh3d": [{"colorbar":
{"linewidth":0, "ticks": ""}, "type": "mesh3d"}], "parcoords": [{"line":
{"colorbar": {"linewidth":0, "ticks": ""}}, "type": "parcoords"}], "pie":
[{"automargin": true, "type": "pie"}], "scatter": [{"fillpattern":
{"fillmode": "overlay", "size": 10, "solidity": 0.2}, "type": "scatter"}], "sc
atter3d": [{"line": {"colorbar": {"linewidth":0, "ticks": ""}}, "marker":
{"colorbar":
{"linewidth":0, "ticks": ""}, "type": "scatter3d"}], "scattercarpet":
[{"marker": {"colorbar":
{"linewidth":0, "ticks": ""}, "type": "scattercarpet"}], "scattergeo":
[{"marker": {"colorbar":
{"linewidth":0, "ticks": ""}, "type": "scattergeo"}], "scattergl":
[{"marker": {"colorbar":
{"linewidth":0, "ticks": ""}, "type": "scattergl"}], "scattermap":
[{"marker": {"colorbar":
{"linewidth":0, "ticks": ""}, "type": "scattermap"}], "scattermapbox":
[{"marker": {"colorbar":
{"linewidth":0, "ticks": ""}, "type": "scattermapbox"}], "scatterpolar"
: [{"marker": {"colorbar":
{"linewidth":0, "ticks": ""}, "type": "scatterpolar"}], "scatterpolargl
": [{"marker": {"colorbar":
{"linewidth":0, "ticks": ""}, "type": "scatterpolargl"}], "scatterterna
ry": [{"marker": {"colorbar":
{"linewidth":0, "ticks": ""}, "type": "scatterternary"}], "surface":
[{"colorbar": {"linewidth":0, "ticks": ""}, "colorscale":
[[0.0, "#0d0887"], [0.1111111111111111, "#46039f"],
[0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"],
[0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"],
[0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"],
[0.8888888888888888, "#fdca26"],
[1.0, "#f0f921"]], "type": "surface"}], "table": [{"cells": {"fill":
{"color": "#EBF0F8"}, "line": {"color": "white"}}, "header": {"fill":
{"color": "#C8D4E3"}, "line":
{"color": "white"}}, "type": "table"}], "layout": {"annotationdefaults":
{"arrowcolor": "#2a3f5f", "arrowhead": 0, "arrowwidth": 1}, "autotypenumbers
": "strict", "coloraxis": {"colorbar":
{"linewidth":0, "ticks": ""}, "colorscale": {"diverging":
[[0, "#8e0152"], [0.1, "#c51b7d"], [0.2, "#de77ae"], [0.3, "#f1b6da"],
[0.4, "#fde0ef"], [0.5, "#f7f7f7"], [0.6, "#e6f5d0"], [0.7, "#b8e186"],
[0.8, "#7fb341"], [0.9, "#4d9221"], [1, "#276419"]], "sequential":
[[0.0, "#0d0887"], [0.1111111111111111, "#46039f"],
[0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"],
[0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"],
[0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"],
[0.8888888888888888, "#fdca26"], [1.0, "#f0f921"]], "sequentialminus":
[[0.0, "#0d0887"], [0.1111111111111111, "#46039f"],
[0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"],
[0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"],
[0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"],

```

```
[0.8888888888888888,"#fdca26",[1.0,"#f0f921"]]},{"colorway":
["#636efa","#EF553B","#00cc96","#ab63fa","#FFA15A","#19d3f3","#FF6692",
"#B6E880","#FF97FF","#FECB52"],"font":{"color":"#2a3f5f"},"geo":
{"bgcolor":"white","lakecolor":"white","landcolor":"#E5ECF6","showlake
s":true,"showland":true,"subunitcolor":"white"},"hoverlabel":
{"align":"left"},"hovermode":"closest","mapbox":
{"style":"light"},"paper_bgcolor":"white","plot_bgcolor":"#E5ECF6","po
lar":{"angularaxis":
{"gridcolor":"white","linecolor":"white","ticks":""},"bgcolor":"#E5ECF
6","radialaxis":
{"gridcolor":"white","linecolor":"white","ticks":""}},"scene":
{"xaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"lineco
lor":"white","showbackground":true,"ticks":"","zerolinecolor":"white"},
"yaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"lineco
lor":"white","showbackground":true,"ticks":"","zerolinecolor":"white"},
"zaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"lineco
lor":"white","showbackground":true,"ticks":"","zerolinecolor":"white"}
},"shapedefaults":{"line":{"color":"#2a3f5f"},"ternary":{"aaxis":
{"gridcolor":"white","linecolor":"white","ticks":""},"baxis":
{"gridcolor":"white","linecolor":"white","ticks":""},"bgcolor":"#E5ECF
6","caxis":
{"gridcolor":"white","linecolor":"white","ticks":""}},"title":
{"x":5.0e-2},"xaxis":
{"automargin":true,"gridcolor":"white","linecolor":"white","ticks":"","
"title":
{"standoff":15},"zerolinecolor":"white","zerolinewidth":2},"yaxis":
{"automargin":true,"gridcolor":"white","linecolor":"white","ticks":"","
"title":
{"standoff":15},"zerolinecolor":"white","zerolinewidth":2}}},"title":
{"text":"t-SNE Projection of Embeddings"},"xaxis":
{"anchor":"y","domain":[0.0,1.0],"title":{"text":"t-SNE1"},"yaxis":
{"anchor":"x","domain":[0.0,1.0],"title":{"text":"t-SNE2"}}}]}
```

Визуализация SVD

```
# Метод 3: Truncated SVD
print("=== Truncated SVD Визуализация ===")
from sklearn.decomposition import TruncatedSVD

svd = TruncatedSVD(n_components=2, random_state=60)
embeddings_svd = svd.fit_transform(embeddings_array)

# Создаем DataFrame для SVD
svd_df = pd.DataFrame(
    {'x': embeddings_svd[:, 0], 'y': embeddings_svd[:, 1], 'label':
labels,
```

```

        'type': ['document'] * len(docs_processed) + (
            ['query'] if query_vector is not None else [])})

# Визуализация SVD
fig_svd = px.scatter(svd_df, x='x', y='y', color='type',
                    title='Truncated SVD Projection of Embeddings',
                    labels={'x': 'SVD1', 'y': 'SVD2'},
                    hover_data=['label'],
                    color_discrete_map={'document': 'purple',
                                        'query': 'red'})
fig_svd.update_traces(marker=dict(size=10, opacity=0.7))
fig_svd.show()

```

=== Truncated SVD Визуализация ===

```

{"config":{"plotlyServerURL":"https://plot.ly"},"data":[{"customdata":
[["Doc_0"],["Doc_1"],["Doc_2"],["Doc_3"],["Doc_4"],["Doc_5"],
["Doc_6"],["Doc_7"],["Doc_8"],["Doc_9"],["Doc_10"],["Doc_11"],
["Doc_12"],["Doc_13"],["Doc_14"],["Doc_15"],["Doc_16"],["Doc_17"],
["Doc_18"],["Doc_19"],["Doc_20"],["Doc_21"],["Doc_22"],["Doc_23"],
["Doc_24"],["Doc_25"],["Doc_26"],["Doc_27"],["Doc_28"],["Doc_29"],
["Doc_30"],["Doc_31"],["Doc_32"],["Doc_33"],["Doc_34"],["Doc_35"],
["Doc_36"],["Doc_37"],["Doc_38"],["Doc_39"],["Doc_40"],["Doc_41"],
["Doc_42"],["Doc_43"],["Doc_44"],["Doc_45"],["Doc_46"],["Doc_47"],
["Doc_48"],["Doc_49"],["Doc_50"],["Doc_51"],["Doc_52"],["Doc_53"],
["Doc_54"],["Doc_55"],["Doc_56"],["Doc_57"],["Doc_58"],["Doc_59"],
["Doc_60"],["Doc_61"],["Doc_62"],["Doc_63"],["Doc_64"],["Doc_65"],
["Doc_66"],["Doc_67"],["Doc_68"],["Doc_69"],["Doc_70"],["Doc_71"],
["Doc_72"],["Doc_73"],["Doc_74"],["Doc_75"],["Doc_76"],["Doc_77"],
["Doc_78"],["Doc_79"],["Doc_80"],["Doc_81"],["Doc_82"],["Doc_83"],
["Doc_84"],["Doc_85"],["Doc_86"],["Doc_87"],["Doc_88"],["Doc_89"],
["Doc_90"],["Doc_91"],["Doc_92"],["Doc_93"],["Doc_94"],["Doc_95"],
["Doc_96"],["Doc_97"],["Doc_98"],["Doc_99"],["Doc_100"],["Doc_101"],
["Doc_102"],["Doc_103"],["Doc_104"],["Doc_105"],["Doc_106"],
["Doc_107"],["Doc_108"],["Doc_109"],["Doc_110"],["Doc_111"],
["Doc_112"],["Doc_113"],["Doc_114"],["Doc_115"],["Doc_116"],
["Doc_117"],["Doc_118"],
["Doc_119"]], "hovertemplate": "type=document<br>SVD1=%{x}<br>SVD2=%
{y}<br>label=%{customdata[0]}<extra></
extra>", "legendgroup": "document", "marker":
{"color": "purple", "opacity": 0.7, "size": 10, "symbol": "circle"}, "mode": "m
arkers", "name": "document", "orientation": "v", "showlegend": true, "type": "
scatter", "x": {"bdata": "MJrXt0jI7b92/qM6eVjuvxgT2x5ETu6/pwpa0e6x7b/
uTV80Sh7tv5oNfT8nDe6/dGhTQ79D7b+qsShIwortv8H+kzWcSe2/
d6Kn8p1L7b+X11lYv3LuvwwjIcrXde6/TbhUUUQW7r80yFC60VLuv9vce0rofu6/
kHecyEdl7r/NtPNHv6vtv2rXAI3hne2/GJ4jmEtd7b9TCpqKYHltv6Tc7uZ/W02/
otQXSom77b8pP/C4IRftvwo+udM5lu6/QoB+2Z5k7r/OK01x/hDvvzdb8IWUW+6/
MHGfIQ/f7r/MAH8nr7Dtv3Zchc3H802/716XhNCU7b/60ts6Dd7sv6DqdJs2/u2/
y5WtKuE97r/UXej1kGvtv/yyJrWeI06/Bm4cHmAK7L8V4pFibEbuV0epqLRXP+6/
A9V89Ue37b9bp0j5gT/uv+Jb+JH5bu6/FvFq3iEx7r/SoAdqn+jsv7MN57VSv0y/

```



```
iJ9KFeol7r8BwHY2BfvuvyeP+dqLF+6/wvy0WPj27L9AX0tznLvtv/FbEgkfk02/
J1uoilGw7b/335f73Q7tv/jurSNLpu2/1MaKQ2D/7b/HqZkRvq3tv0KY+CK9/+2/N/
32eBMA7b/U3BxcduXtv6HU/a4vMe6/Ni7q6ZiA7r+VQZjzQnPuv8UVSf3iau6/
ze6ziakp7r+K3nlQqwLuv/27D0Iw3e6/1A5dyVvS7L+0HydGjjjuv+71AlG4g06/
Ed4WLZT77b94xNt3RXXuv0exnHqJpe2/XKDZU85n7r/4ijkpmbztv8kpk3idWu2/
QtRYoR6L7b+AG5EKyfrsv7t5R0Jn902/L9p0y5l97r8wPE/6SNXsv/QLNmVzwe6/
GDhX1TbN7r9qQBLvQWbuv6xtwxItVu2/AJHXGMIQ7b8kbVSYT7/tvx2fpvBl10y/
MMPRObYy7r87x2xokT7uv+aqZ+xuQ06/HL81iPft7r/SyrU1V4juv1HkSR/
q1e2/0TNBXPz7b91+pBQ0Zvuv1SAf0Mq/02/o5EeDF997r8gihkC0+vuvxK95+h0hu6/
Piz8JLt07r/NHVGJzLLtv/Kt9HT+602/RaCaGhc07b8SttVCsKrtvw0fmQDzLe6/
SNmCzQ1L7r9h4cnfDYHuv7QZ6c7Lyu2/
FNddNoZw7b80QfCRLq7tv8KAe+cpM+6/0N2cu4Ab7b+AkbM0+pPtv+xm59zaDu2/
ET2aD4oG7L8J8TAU4Nnuv2wK9J3IKe6/
Jbqvd3Sa7r+9u3WXN0HsvwA+ayICU06/","dtype":"f8"},"xaxis":"x","y":
{"bdata":"iBG9WUlkPl9L9+jEYIedv5J/cpaFI8S/SA0DF8ijwr9ImkokrETJvxp/
RaPFdsa/2wCt1KV1yL/ZQW0fxtDJv6IUAacAssS/Ufbru1BCyL+QdY/
oazfHPwC724C6W8Q/ijozeLbEwj89omu6Jdy7P4UghmuVa8A/
Utwub2hqwT8klMLfuTPBv8uU8Pd8ms0/0K9EuVw7vL/Jdw0qqcaov7zHdW3VQaq/
DgLQZ7Xrtb9lAbjr2eDNvyjHJug0pcQ/kX1ILxF8xT8a7TnibVGlv8j06Guquqi/
GTaXSfndwz/tNRHIH4/Ev0y1sD9cl8K/nxi3TZIrWl96e6gMhdvHv3k63oD6cqu/
Nvb2DDNFpb+AlZNXn4NYvwCuBj+jtVW/NnCrUGdWrr/0ab//n360vx6lhY3sw6S/qE/
XUAkFtT+2IN6bceiiP7Wa3fP/R7E/zjtBsCrZwL8H0uVz9A7Cv8KurgXYQ8S/
bZDVfe1vyz+qkuLayMDAP0MghY6Ey8c/3y0nsNo5vb9v7qNij1Wcv/Gj8fVW062/
DftZCm0hy7+BD4TTuT7Gv0WNUIe6pM0/Dlb6fmwnzz9g7felQ+bQP1iauzu0rMs/
Wtcplm2a0j+8KDQEC9LLPxIhKnaTsc8/KLHR3d9Ewz+DPlvHCMjJPzGh9SzhScU/
2b7x480Dlb/Mt5EVKJSov45ufEkYA3k/E/W5e/CFwL9AJkycrbnLP3Twc9b7JcI/
EPleHPH2tz/3uWIPJJ7IP7i1oE1Y8MW/p/loMTaHtr9Nh4RcU7ayv9ef3b6In7C/
4DPwcJ3W0D+g0B6mB4zTP7IrfyE/BMg/lqfwdyc9wj/nipBYyoDJv7XG5Vrr6cc/
IN6Cza3UxD89VkwBmBKTv8EivGC2B5W/tDhMUDxrsb+Vv2lLkEqPv3gBHNl+rcW/
s+JGfnLZxT/4pxJZNNvMP9zB2ZJGvco/HYx0Wz3dwj8TB2dn9LbLPx8QhesfFMk/
RL00a0Kksr8/GlldjGnDP/pJM7Vaiaq/0Hl+Y45nY78XX1J08dq0v4oQoM5Fi6W/bC/
Jllx5vb/SzX0+4W3Av5cPcwnLm8e/KjETJnnwvr+/JcYu/La1v8LRq0G+isg/
sZz9jaE4vT/WVYCMiWg7PzSmLnhU/sK/eEGtKtrZwr/YalumBnvAv8od3M3wnsC/
Eu4LU7wCy7/QxIxBbmVlv0VDi00/1Mu/UHbHuIujzr/05YrADNDAP9KFB77JBbw/
g6zMLWUTur9XPC4wCtXLv96yuSwvCK4/","dtype":"f8"},"yaxis":"y"},
{"customdata": [{"Query"}], "hovertemplate": "type=query<br>SVD1=%
{x}<br>SVD2=%{y}<br>label=%{customdata[0]}<extra></
extra>", "legendgroup": "query", "marker":
{"color": "red", "opacity": 0.7, "size": 10, "symbol": "circle"}, "mode": "mark
ers", "name": "query", "orientation": "v", "showlegend": true, "type": "scatte
r", "x": {"bdata": "Nypj9f4z4r8=", "dtype": "f8"}, "xaxis": "x", "y":
{"bdata": "0fuNAWybt78=", "dtype": "f8"}, "yaxis": "y"}], "layout":
{"legend": {"title": {"text": "type"}, "tracegroupgap": 0}, "template":
{"data": {"bar": [{"error_x": {"color": "#2a3f5f"}, "error_y":
{"color": "#2a3f5f"}, "marker": {"line":
{"color": "#E5ECF6", "width": 0.5}, "pattern":
{"fillmode": "overlay", "size": 10, "solidity": 0.2}}, "type": "bar"}], "barpo
lar": [{"marker": {"line": {"color": "#E5ECF6", "width": 0.5}, "pattern":
{"fillmode": "overlay", "size": 10, "solidity": 0.2}}, "type": "barpolar"}], "
```

```

carpet":[{"aaxis":
{"endlinecolor":"#2a3f5f","gridcolor":"white","linecolor":"white","min
orgridcolor":"white","startlinecolor":"#2a3f5f"},"baxis":
{"endlinecolor":"#2a3f5f","gridcolor":"white","linecolor":"white","min
orgridcolor":"white","startlinecolor":"#2a3f5f"},"type":"carpet"}],"ch
oropleth":[{"colorbar":
{"outlinewidth":0,"ticks":"","type":"choropleth"}],"contour":
[{"colorbar":{"outlinewidth":0,"ticks":"","colorscale":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1.0,"#f0f921"]],"type":"contour"}],"contourcarpet":[{"colorbar":
{"outlinewidth":0,"ticks":"","type":"contourcarpet"}],"heatmap":
[{"colorbar":{"outlinewidth":0,"ticks":"","colorscale":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1.0,"#f0f921"]],"type":"heatmap"}],"histogram":[{"marker":{"pattern":
{"fillmode":"overlay","size":10,"solidity":0.2},"type":"histogram"}],
"histogram2d":[{"colorbar":{"outlinewidth":0,"ticks":"","colorscale":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1.0,"#f0f921"]],"type":"histogram2d"}],"histogram2dcontour":
[{"colorbar":{"outlinewidth":0,"ticks":"","colorscale":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1.0,"#f0f921"]],"type":"histogram2dcontour"}],"mesh3d":[{"colorbar":
{"outlinewidth":0,"ticks":"","type":"mesh3d"}],"parcoords":[{"line":
{"colorbar":{"outlinewidth":0,"ticks":"","type":"parcoords"}],"pie":
[{"automargin":true,"type":"pie"}],"scatter":[{"fillpattern":
{"fillmode":"overlay","size":10,"solidity":0.2},"type":"scatter"}],"sc
atter3d":[{"line":{"colorbar":{"outlinewidth":0,"ticks":"","type":"scatter3d"}],"marker":
{"colorbar":
{"outlinewidth":0,"ticks":"","type":"scatter3d"}],"scattercarpet":
[{"marker":{"colorbar":
{"outlinewidth":0,"ticks":"","type":"scattercarpet"}],"scattergeo":
[{"marker":{"colorbar":
{"outlinewidth":0,"ticks":"","type":"scattergeo"}],"scattergl":
[{"marker":{"colorbar":

```



```

{"linewidth":0,"ticks":"","type":"scattergl"},"scattermap":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scattermap"},"scattermapbox":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scattermapbox"},"scatterpolar":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scatterpolar"},"scatterpolargl":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scatterpolargl"},"scatterternary":
[{"marker":{"colorbar":
{"linewidth":0,"ticks":"","type":"scatterternary"},"surface":
[{"colorbar":{"linewidth":0,"ticks":"","colorscale":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1.0,"#f0f921"]],"type":"surface"},"table":{"cells":{"fill":
{"color":"#EBF0F8"},"line":{"color":"white"},"header":{"fill":
{"color":"#C8D4E3"},"line":
{"color":"white"},"type":"table"}}},"layout":{"annotationdefaults":
{"arrowcolor":"#2a3f5f","arrowhead":0,"arrowwidth":1},"autotypenumbers":
"strict","coloraxis":{"colorbar":
{"linewidth":0,"ticks":"","colorscale":{"diverging":
[[0,"#8e0152"],[0.1,"#c51b7d"],[0.2,"#de77ae"],[0.3,"#f1b6da"],
[0.4,"#fde0ef"],[0.5,"#f7f7f7"],[0.6,"#e6f5d0"],[0.7,"#b8e186"],
[0.8,"#7fb341"],[0.9,"#4d9221"],[1,"#276419"]],"sequential":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]],"sequentialminus":
[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]]]},"colorway":
["#636efa","#EF553B","#00cc96","#ab63fa","#FFA15A","#19d3f3","#FF6692",
"#B6E880","#FF97FF","#FECB52"],"font":{"color":"#2a3f5f"},"geo":
{"bgcolor":"white","lakecolor":"white","landcolor":"#E5ECF6","showlakes":
true,"showland":true,"subunitcolor":"white"},"hoverlabel":
{"align":"left"},"hovermode":"closest","mapbox":
{"style":"light"},"paper_bgcolor":"white","plot_bgcolor":"#E5ECF6","polar":
{"angularaxis":
{"gridcolor":"white","linecolor":"white","ticks":"","bgcolor":"#E5ECF6",
"radialaxis":
{"gridcolor":"white","linecolor":"white","ticks":"","scene":
{"xaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"lineco

```

```

lor:"white","showbackground":true,"ticks":"","zerolinecolor":"white"}
,"yaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"linecolor":"white","showbackground":true,"ticks":"","zerolinecolor":"white"}
,"zaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"linecolor":"white","showbackground":true,"ticks":"","zerolinecolor":"white"}
}, "shapedefaults":{"line":{"color":"#2a3f5f"},"ternary":{"aaxis":
{"gridcolor":"white","linecolor":"white","ticks":""},"baxis":
{"gridcolor":"white","linecolor":"white","ticks":""},"bgcolor":"#E5ECF6","caxis":
{"gridcolor":"white","linecolor":"white","ticks":""}}},"title":
{"x":5.0e-2},"xaxis":
{"automargin":true,"gridcolor":"white","linecolor":"white","ticks":"","title":
{"standoff":15},"zerolinecolor":"white","zerolinewidth":2},"yaxis":
{"automargin":true,"gridcolor":"white","linecolor":"white","ticks":"","title":
{"standoff":15},"zerolinecolor":"white","zerolinewidth":2}}},"title":
{"text":"Truncated SVD Projection of Embeddings"},"xaxis":
{"anchor":"y","domain":[0.0,1.0],"title":{"text":"SVD1"},"yaxis":
{"anchor":"x","domain":[0.0,1.0],"title":{"text":"SVD2"}}}}

```

Для снижения размерности я использовал методы PCA, TSNE и SVD. Метод PCA уловил кластера и отобразил запрос относительно релевантных объектов.

Подключение LLM Gigachat

```

import os
from dotenv import load_dotenv

load_dotenv()

GIGACHAT_AUTH_KEY = os.getenv('GIGACHAT_AUTH_KEY')

print(GIGACHAT_AUTH_KEY)  # Проверка

from gigachat import GigaChat
from gigachat.models import Chat, Messages, MessagesRole

class GigaChatPipeline:
    def __init__(self, credentials, temperature=0.2, max_tokens=500):
        self.giga = GigaChat(credentials=credentials,
                               verify_ssl_certs=False,
                               scope="GIGACHAT_API_PERS")
        self.temperature = temperature
        self.max_tokens = max_tokens

    def __call__(self, prompt):

```

```

# Если промт - список сообщений (как в вашем формате)
if isinstance(prompt, list):
    messages = self._convert_messages_format(prompt)
# Если промт - строка (старый формат)
elif isinstance(prompt, str):
    messages = [Messages(role=MessagesRole.USER,
content=prompt)]
else:
    raise ValueError("Prompt must be either string or list of
messages")

response = self.giga.chat(
    Chat(messages=messages, temperature=self.temperature,
max_tokens=self.max_tokens, ))

return [{"generated_text":
response.choices[0].message.content}]

def _convert_messages_format(self, messages_list):
    """Конвертирует сообщения из формата HuggingFace в формат
GigaChat"""
    gigachat_messages = []

    role_mapping = {"system": MessagesRole.SYSTEM,
                    "user": MessagesRole.USER,
                    "assistant": MessagesRole.ASSISTANT}

    for msg in messages_list:
        role = msg.get("role", "user")
        content = msg.get("content", "")

        gigachat_role = role_mapping.get(role, MessagesRole.USER)
        gigachat_messages.append(
            Messages(role=gigachat_role, content=content))

    return gigachat_messages

# Использование как замена pipeline
READER_LLM = GigaChatPipeline(credentials=GIGACHAT_AUTH_KEY,
temperature=0.2,
max_tokens=500)

print(READER_LLM("What is 4+4? Answer:"))

[{'generated_text': '### Шаг 1: Понимание условия\nЗадача состоит в
простом сложении двух чисел — 4 и 4.\n\n### Шаг 2: Выполнение операции
сложения\nСложим числа:\n$ 4 + 4 = 8 $'\n\n### Итоговый ответ:\n$4 +
4 = 8$'}]
```

Формирование промпта

Приведенный ниже шаблон RAG — это то, что мы передадим в Reader LLM: важно, чтобы он был отформатирован в шаблоне чата Reader LLM. Мы даем ему наш контекст и вопрос пользователя.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(EMBEDDING_MODEL_NAME)

tokenizer.chat_template = """{% for message in messages %}
{% if message['role'] == 'user' %}
Human: {{ message['content'] }}
{% elif message['role'] == 'assistant' %}
Assistant: {{ message['content'] }}
{% elif message['role'] == 'system' %}
System: {{ message['content'] }}
{% endif %}
{% endfor %}
Assistant: """

prompt_in_chat_format = [{"role": "system", "content": """
Используя информацию, содержащуюся в контексте, дайте
развернутый ответ на вопрос.
Отвечайте только на заданный вопрос, ответ должен быть кратким и
соответствовать теме.
Опиши в свободной форме найденную достопримечательность.
Опиши как рассказывает экскурсовод. В конце приведи дополнительную
информацию (1-2 предложения).
Избегай структуры.
Укажите номер исходного документа, если это необходимо.
Если ответ не может быть выведен из контекста, не давайте
ответ. """ },
                        {"role": "user", "content": """Context:
{context}
---
А теперь вопрос, на который вам нужно ответить:

Вопрос: {question} """ }, ]

RAG_PROMPT_TEMPLATE =
tokenizer.apply_chat_template(prompt_in_chat_format,
                             tokenize=False,

add_generation_prompt=True)
print(RAG_PROMPT_TEMPLATE)

System:
Используя информацию, содержащуюся в контексте, дайте
развернутый ответ на вопрос.
```

Отвечайте только на заданный вопрос, ответ должен быть кратким и соответствовать теме.
 Опиши в свободной форме найденную достопримечательность.
 Опиши как рассказывает экскурсовод. В конце приведи дополнительную информацию (1-2 предложения).
 Избегай структуры.
 Укажите номер исходного документа, если это необходимо.
 Если ответ не может быть выведен из контекста, не давайте ответ.
 Human: Context:
 {context}

 А теперь вопрос, на который вам нужно ответить:
 Вопрос: {question}
 Assistant:

Тестирование LLM Reader

```

user_query = 'дом в городе владимир'
user_query = 'В каких городах есть зоопарк'
user_query = 'Как выглядит зоопарк в Ярославле'

print(f"\nStarting retrieval for {user_query=}...")
retrieved_docs =
KNOWLEDGE_VECTOR_DATABASE.similarity_search(query=user_query,
                                             k=5)

print(
    "\n=====Top
document=====")
print(retrieved_docs[0].page_content)
print(

    "=====Metadata=====
=====")
# print(retrieved_docs[0].metadata['City'])

Starting retrieval for user_query='Как выглядит зоопарк в
Ярославле'...

=====Top
document=====
**Название:** Ярославский зоопарк
**Тип объекта:** Зоопарк
**Локация:** Город Ярославль, Россия
**Координаты:** 39.895737, 57.674446
**Описание:** Ярославский зоопарк
**Описание изображения:** большая плюшевая игрушка, сидящая на

```

```

гигантском шаре
**Идентификатор во внешней БД:** Wikidata (Q16723452)
=====Metadata=====
=====

retrieved_docs_text = [doc.page_content for doc in retrieved_docs]
context = "\nExtracted documents:\n"
context += "".join([f"Document {str(i)}::\n" + doc for i, doc in
                    enumerate(retrieved_docs_text)])

context

'\nExtracted documents:\nDocument 0::\n**Название:** Ярославский
зоопарк\n**Тип объекта:** Зоопарк\n**Локация:** Город Ярославль,
Россия\n**Координаты:** 39.895737, 57.674446\n**Описание:**
Ярославский зоопарк\n**Описание изображения:** большая плюшевая
игрушка, сидящая на гигантском шаре\n**Идентификатор во внешней БД:**
Wikidata (Q16723452)Document 1::\n**Название:** Парк 1000-летия
Ярославля\n**Тип объекта:** Парк\n**Локация:** Город Ярославль,
Россия\n**Координаты:** 39.871151, 57.616882\n**Описание:** Парк в
Ярославле\n**Описание изображения:** вид на парк с фонтаном и озером
на заднем плане\n**Идентификатор во внешней БД:** Wikidata
(Q4421071)Document 2::\n**Название:** Ярославский музей-заповедник\
\n**Тип объекта:** Музей\n**Локация:** Город Ярославль, Россия\
\n**Координаты:** 39.89003, 57.620876\n**Описание:** Ярославский музей-
заповедник\n**Описание изображения:** белое здание с синей вывеской\
\n**Идентификатор во внешней БД:** Wikidata (Q4538889)Document 3::\
\n**Название:** Парк на Стрелке\n**Тип объекта:** Парк\n**Локация:**
Город Ярославль, Россия\n**Координаты:** 39.902702, 57.622063\
\n**Описание:** Парк в Ярославле\n**Описание изображения:** размытый
вид парка со статуей птицы на вершине колонны\n**Идентификатор во
внешней БД:** Wikidata (Q19909049)Document 4::\n**Название:**
Власьевский сквер\n**Тип объекта:** Сад\n**Локация:** Город Ярославль,
Россия\n**Координаты:** 39.882793, 57.626457\n**Описание:** Городской
сад в Ярославле\n**Описание изображения:** женщина идет по дорожке в
парке\n**Идентификатор во внешней БД:** Wikidata (Q13668166)

final_prompt = RAG_PROMPT_TEMPLATE.format(question=user_query,
context=context)

answer = READER_LLM(final_prompt)
print(answer[0]['generated_text'])

```

Ярославский зоопарк – крупный зоологический парк в центре города Ярославля, расположенный рядом с набережной Волги. Экскурсоводы рассказывают, что здесь содержатся около тысячи видов животных, включая редких и исчезающих представителей фауны. Территория зоопарка ухожена, оборудованы удобные прогулочные маршруты, зоны отдыха и кафе. Особое внимание уделяется комфорту посетителей и правильному содержанию животных. Дополнительная информация: Ярославский зоопарк

является одним из старейших и крупнейших в России, основан в 1903 году. (Документ 0)

```
from transformers import Pipeline

def answer_with_rag(question: str, llm: Pipeline, knowledge_index,
                    reranker=None, num_retrieved_docs: int = 30,
                    num_docs_final: int = 5, ) -> Tuple[
    str, List[LangchainDocument]]:
    # Соберём документы с помощью ретривера
    print("=> Retrieving documents...")
    relevant_docs = knowledge_index.similarity_search(query=question,
k=num_retrieved_docs)
    relevant_docs = [doc.page_content for doc in
                     relevant_docs] # Оставляем только текст

    if reranker:
        print("=> Reranking documents...")
        relevant_docs = reranker.rerank(question, relevant_docs,
                                         k=num_docs_final)
        relevant_docs = [doc["content"] for doc in relevant_docs]
    else:
        relevant_docs = relevant_docs[:num_docs_final]

    # Финальный промпт
    context = "\nExtracted documents:\n"
    context += "".join([f"Document {str(i)}:::\n" + doc for i, doc in
                        enumerate(relevant_docs)])

    final_prompt = RAG_PROMPT_TEMPLATE.format(question=question,
                                              context=context)

    print("=> Generating answer...")
    answer = llm(final_prompt)[0]["generated_text"]

    return answer, relevant_docs

question = "Какие храмы посмотреть во Владимире"

answer, relevant_docs = answer_with_rag(question, READER_LLM,
                                       KNOWLEDGE_VECTOR_DATABASE,
                                       reranker=None)

print(answer)
```

Во Владимире стоит посетить следующие достопримечательности:

- ****Князь-Владимирская церковь****: православный кладбищенский храм, расположенный в историческом центре города.

- ****Дмитриевский собор****: древний православный храм, являющийся памятником белокаменного зодчества домонгольской Руси.
- ****Собор Успения Пресвятой Богородицы****: еще один значимый исторический храм, также относящийся к памятникам архитектуры домонгольского периода.

(Документ 0, Документ 2, Документ 3)

Дополнительно: Во Владимире сосредоточено значительное количество исторических храмов и архитектурных памятников, отражающих богатую культурную историю региона.

```
from transformers import Pipeline
from typing import Tuple, List
from sentence_transformers import SentenceTransformer
import numpy as np
import torch

class RuBertTiny2Reranker:
    def __init__(self, model_name: str = "cointegrated/rubert-tiny2"):
        self.model = SentenceTransformer(model_name)
        self.device = torch.device(
            "cuda" if torch.cuda.is_available() else "cpu")
        self.model.to(self.device)

    def encode(self, texts, **kwargs):
        """Кодирование текстов в эмбединги"""
        return self.model.encode(texts, **kwargs)

    def rerank(self, query: str, documents: List[str], k: int = None):
        """Rerank документов относительно запроса"""
        # Кодлируем запрос и документы
        query_embedding = self.encode([query])
        doc_embeddings = self.encode(documents)

        # Вычисляем косинусное сходство
        similarities = np.dot(doc_embeddings,
                               query_embedding.T).flatten()

        # Сортируем по убыванию сходства
        ranked_indices = np.argsort(similarities)[::-1]

        # Формируем результат в ожидаемом формате
        results = []
        for idx in ranked_indices:
            results.append(
                {'content': documents[idx], 'score':
float(similarities[idx]),
                'index': int(idx)})
```



```

        if k:
            results = results[:k]

    return results

reranker = RuBertTiny2Reranker()

question = "Какие храмы посмотреть во Владимире"

answer, relevant_docs = answer_with_rag(question, READER_LLM,
                                       KNOWLEDGE_VECTOR_DATABASE,
                                       reranker=reranker)

print(answer)

```

=> Retrieving documents...

=> Reranking documents...

=> Generating answer...

Во Владимире стоит посетить следующие достопримечательности:

- ****Князь-Владимирская церковь****: православный кладбищенский храм, расположенный в историческом центре города.
- ****Дмитриевский собор****: древнейший сохранившийся белокаменный храм домонгольского периода, включённый в список Всемирного наследия ЮНЕСКО.
- ****Собор Успения Пресвятой Богородицы****: памятник белокаменного зодчества домонгольской Руси, один из старейших храмов региона.

(Документ 0, Документ 2, Документ 3)

Дополнительно рекомендуется посетить также Армянскую церковь Владимира (Документ 1).

```
question = "Как выглядит Доходный дом купца Чувильдина?"
```

```
answer, relevant_docs = answer_with_rag(question, READER_LLM,
                                       KNOWLEDGE_VECTOR_DATABASE,
                                       reranker=reranker)
```

```
print(answer)
```

=> Retrieving documents...

=> Reranking documents...

=> Generating answer...

Доходный дом купца Чувильдина – величественное здание в центре Екатеринбурга, представляющее собой крупный особняк с массивной архитектурой и башней часов на верхнем этаже. Экскурсовод подчеркивает его архитектурное совершенство и историческое значение, указывая на характерные черты стиля эклектики начала XX века. Дополнительная информация: объект включен в реестр культурного наследия России и является памятником федерального значения. (Документ 0)

RAGAS (3 балла)

RAGAS - это система, созданная для автоматической оценки эффективности модели RAG. Она представляет собой комплексный инструмент, способный оценить как качество извлечения информации, так и качество созданного текста. Это достигается благодаря использованию различных метрик, которые анализируют разные аспекты ответов, сгенерированных языковой моделью.

Метрики оценки в RAGAS

Faithfulness

Оценивает, насколько ответы языковой модели верно передают информацию, содержащуюся в источниках данных. Это критически важно, чтобы гарантировать, что модель не искажает или не меняет информацию при её интерпретации.

$$\$ \text{Faithfulness} = \frac{\text{Number of claims that can be inferred from given context}}{\text{Total number of claims in the generated answer}} \$$$

Answer Relevancy

Демонстрирует, насколько ответы соответствуют заданным вопросам. Это важный аспект, так как даже правильный ответ с технической точки зрения может быть ненужным, если он не отвечает на конкретный вопрос пользователя.

$$\text{answer relevancy} = \frac{1}{N} \sum_{i=1}^N \cos(E_{g_i}, E_o)$$

$$\text{answer relevancy} = \frac{1}{N} \sum_{i=1}^N \frac{E_{g_i} \cdot E_o}{\|E_{g_i}\| \|E_o\|}$$

Где:

- E_{g_i} - это эмбединг сгенерированного ответа i
- E_o - это эмбединг исходного вопроса.
- N - это количество сгенерированных вопросов, которое по умолчанию равно 3.

Context Recall и Context Precision

Эти метрики измеряют эффективность модели в учете контекста вопроса для формирования ответа. "Context Recall" оценивает количество использованной информации из контекста в ответе, в то время как "Context Precision" оценивает точность и релевантность использования контекстной информации.

- $\text{context recall} = \frac{\text{Number of sentences that can be attributed to context}}{\text{Number of sentences in GT}}$
- $\text{Context Precision}@k = \frac{\sum \text{precision}@k}{\text{total number of relevant items in the top K results}}$
 - $\text{Precision}@k = \frac{\text{true positives}@k}{(\text{true positives}@k + \text{false positives}@k)}$

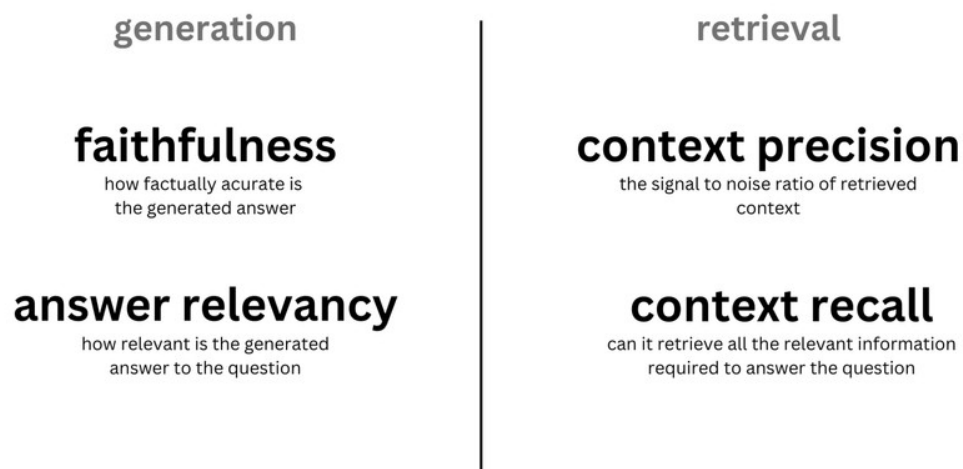
Answer Semantic Similarity

Оценивает степень семантической близости между ответом модели и эталонным ответом. Это позволяет определить, насколько естественно и точно модель способна воссоздавать человеческий язык.

Answer Correctness

Оценка корректности ответа. Это последний шаг в проверке, чтобы убедиться, предоставляет ли модель точную и достоверную информацию в своем ответе.

ragas score



В этом задании вам нужно реализовать метрику `answer_relevancy` в коде. Опционально за допбаллы можно реализовать другие метрики.

Добавление полей

Для этого датасет нужно привести к виду, в котором будут следующие поля:

1. question

- Смысл: Эта колонка содержит вопросы, на которые нужно найти ответы. В контексте задач QA, это основная информация, которую модель должна обработать для генерации ответа.
- Пример: "Каковы преимущества RAG?"

2. ground_truths

- Смысл: В этой колонке находятся правильные ответы на соответствующие вопросы. Это может использоваться для оценки производительности модели, так как позволяет сравнить сгенерированные ответы с эталонными.
- Пример: "RAG улучшает качество генерации, используя внешние источники информации."

3. answer

- Смысл: Эта колонка содержит ответы, которые генерирует модель на основе заданных вопросов. Это может быть результат работы модели, которую вы тестируете или обучаете.
- Пример: "RAG позволяет моделям извлекать информацию из внешних источников, что улучшает их способность отвечать на сложные вопросы."

4. contexts

- Смысл: Эта колонка предоставляет контекст или дополнительную информацию, которая может быть полезна для понимания вопроса или для формирования ответа. Это может быть текст, из которого был извлечен ответ, или другие релевантные документы.
- Пример: "RAG (Retrieval-Augmented Generation) — это метод, который использует внешние базы данных для получения информации при генерации текста."

HINT: Поле question можно тоже сгенерировать. Например, можно попросить модель задать вопрос по описанию изображения.

HINT: Для answer_relevancy нужны не все поля HINT: Инференс такого эксперимента может быть долгим. План минимум: возьмите 100 случайных сэмплов.

Обработка и добавление полей

План

- я не сохранял метаинформацию при генерации документов. поэтому придется обработать файл документов
- часть информации `questions` и `ground_truths` будет выделена напрямую запросами LLM
- `contexts` будет сгенерирован обработкой файла

Промпт для генерации вопросов к виду изображения

Проводится тестирование RAG по технологии RAGAS. В файле перечислены документы. Есть поле Описание изображения. Необходимо к этому полю создать Вопрос. Результат сгенерировать как массив вопросов. Всего должно получиться 120 элементов.

Пример вопроса для документа

Как выглядит Доходный дом купца Чувильдина?

Пример документа

Индекс: 0 Название: Доходный дом купца Чувильдина **Тип объекта:** Дом **Локация:** Город Екатеринбург, Россия **Координаты:** 60.607727, 56.832176 **Описание:** Дом купца Чувильдина
Описание изображения: большое здание с часами наверху **Идентификатор во внешней БД:** Wikidata (Q27921892)

Промпт для генерации ответов к описанию изображения

В файле перечислены документы. Есть поле Описание изображения. Необходимо Выделить это поле и записать в массив answer. Всего должно получиться 120 элементов.

Пример ответа для документа

Большое здание с часами наверху

Пример документа

Индекс: 0 Название: Доходный дом купца Чувильдина **Тип объекта:** Дом **Локация:** Город Екатеринбург, Россия **Координаты:** 60.607727, 56.832176 **Описание:** Дом купца Чувильдина
Описание изображения: большое здание с часами наверху **Идентификатор во внешней БД:** Wikidata (Q27921892)

```
import random

def extract_all_contexts(file_content):
    """Извлечение всех контекстов в один плоский массив"""

    documents = []
    blocks = file_content.split('---')

    for block in blocks:
        if not block.strip():
            continue
        doc = {}
        for line in block.split('\n'):
            if '**Название:**' in line:
                doc['title'] = line.split '**Название:**')[1].strip()
            elif '**Тип объекта:**' in line:
                doc['type'] = line.split '**Тип объекта:**')[1].strip()
            elif '**Локация:**' in line:
```

```

        doc['location'] = line.split('**Локация:**')
[1].strip()
    elif '**Координаты:**' in line:
        doc['coordinates'] = line.split('**Координаты:**')
[1].strip()
    elif '**Описание изображения:**' in line:
        doc['image_description'] = \
            line.split('**Описание изображения:**')[1].strip()
    elif '**Идентификатор во внешней БД:**' in line:
        doc['external_id'] = \
            line.split('**Идентификатор во внешней БД:**')
[1].strip()

    if doc:
        documents.append(doc)

# Создаем плоский массив всех контекстов
    all_contexts = []
    for doc in documents:
        contexts = [
            f"Фотография {doc['title']} показывает:
{doc['image_description']}",
            f"Внешний вид {doc['title']}: {doc['image_description']}",
            f"На изображении {doc['title']}:
{doc['image_description']}", ]

        all_contexts.extend([random.choice(contexts)])

    return all_contexts

# Использование
with open('whole_context.md', 'r', encoding='utf-8') as file:
    file_content = file.read()

all_contexts = extract_all_contexts(file_content)

print(f"Всего контекстов: {len(all_contexts)}")

Всего контекстов: 120

def data_preprocess(df=None):
    questions = [
        "Как выглядит Доходный дом купца Чувильдина?",
        "Как выглядит Дом В.И. Чувильдина?",
        "Как выглядит Дом Актёра?",
        "Как выглядит Уральский геологический музей?",
        "Как выглядит Музей истории Екатеринбурга?",
        "Как выглядит Дом Метенкова?",
        "Как выглядит Театр оперы и балета?",
        "Как выглядит Театр кукол?",
        "Как выглядит Театр музыкальной комедии?",
    ]

```

"Как выглядит Харитоновский сад?",
 "Как выглядит Храм-на-Крови?", "Как выглядит Храм на Крови?",
 "Как выглядит Свято-Никольский храм?",
 "Как выглядит Успенский собор на ВИЗе?",
 "Как выглядит Свято-Троицкий Кафедральный собор?",
 "Как выглядит Александро-Невский собор?",
 "Как выглядит памятник Серго Орджоникидзе?",
 "Как выглядит памятник Комсомолу Урала?",
 "Как выглядит памятник Ленину?", "Как выглядит Царский мост?",
 "Как выглядит Каменный мост?",
 "Как выглядит здание бывшей гостиницы \"Мадрид\"?",
 "Как выглядит Центральный стадион?",
 "Как выглядит Преображенская церковь на Уктусе?",
 "Как выглядит Иоанно-Предтеченский Кафедральный собор?",
 "Как выглядит Усадьба Железнова?",
 "Как выглядит Усадьба Тарасова?",
 "Как выглядит Ново-Тихвинский монастырь?",
 "Как выглядит улица Февральской Революции?",
 "Как выглядит улица Добролюбова?",
 "Как выглядит Уральский государственный университет имени А. М. Горького?",
 "Как выглядит Уральский государственный технический университет?",
 "Как выглядит торговый дом Скоба?",
 "Как выглядит Дом Виноградовых?",
 "Как выглядит Доходный дом купчихи П. Е. Кубаревой?",
 "Как выглядит Дом М. В. Медведева?",
 "Как выглядит Памятник жертвам и мученикам революции 1905 года?",
 "Как выглядит Шахматный дом (Игорный дом А.И. Троицкого)?",
 "Как выглядит Обелиск Минину и Пожарскому?",
 "Как выглядит Северная башня?",
 "Как выглядит Борисоглебская башня?",
 "Как выглядит Белая башня?",
 "Как выглядит кинотеатр «Рекорд»?",
 "Как выглядит Театр оперы и балета в Нижнем Новгороде?",
 "Как выглядит Нижегородский государственный академический театр драмы им. М. Горького?",
 "Как выглядит Собор Александра Невского?",
 "Как выглядит Спасский староярмарочный кафедральный собор?",
 "Как выглядит Нижегородская соборная мечеть?",
 "Как выглядит Нижегородский государственный художественный музей?",

"Как выглядит Усадьба Добролюбовых?",
 "Как выглядит Дом-музей Добролюбова?",
 "Как выглядит Автозаводский парк культуры и отдыха?",
 "Как выглядит Сормовский парк?",
 "Как выглядит Парк имени Кулибина?",
 "Как выглядит Церковь во имя Всемилостивейшего
 Спаса?",
 "Как выглядит Церковь во имя Всемирлостивейшего
 Спаса?",
 "Как выглядит Храм святителей Московских?",
 "Как выглядит Церковь Знамения Божией Матери и святых
 Жен-Мироносиц?",
 "Как выглядит Церковь Александра Невского в
 Сормово?",
 "Как выглядит Спасо-преображенская церковь?",
 "Как выглядит Благовещенский монастырь?",
 "Как выглядит Благовещенский собор?",
 "Как выглядит Печёрский Вознесенский монастырь?",
 "Как выглядит Усадьба Рукавишниковых?",
 "Как выглядит Усадьба Е. И. Богоявленской?",
 "Как выглядит Усадьба Киришбаумов?",
 "Как выглядит стадион «Локомотив»?",
 "Как выглядит Церковь Никиты Великомученика?",
 "Как выглядит Троицкая церковь?",
 "Как выглядит Армянская церковь Владимира?",
 "Как выглядит Собор Успения Пресвятой Богородицы?",
 "Как выглядит Исторический музей во Владимире?",
 "Как выглядит Музей ложки?",
 "Как выглядит Владимиро-Суздальский музей-
 заповедник?",
 "Как выглядит Дом-музей Столетовых?",
 "Как выглядит Храм Казанской иконы Божией Матери?",
 "Как выглядит Костел Святого Розария Пресвятой Девы
 Марии?",
 "Как выглядит Дмитриевский собор?",
 "Как выглядит Князь-Владимирская церковь?",
 "Как выглядит Владимирский академический областной
 драматический театр?",
 "Как выглядит Богородице-Рождественский мужской
 монастырь?",
 "Как выглядит Успенский Княгинин монастырь?",
 "Как выглядят Золотые ворота?",
 "Как выглядит памятник \"Белокаменные памятники
 Владимира и Суздаля\"?",
 "Как выглядит Владимирский государственный
 гуманитарный университет?",
 "Как выглядит Владимирский планетарий?",
 "Как выглядит памятник природы \"Лесной парк
 Дружба\"?",

Ярославле?",

"Как выглядит Церковь Рождества Христова в

"Как выглядит церковь Иоанна Предтечи в Толчкове?",

"Как выглядит церковь Михаила Архангела?",

"Как выглядит Крестоборогодская церковь?",

"Как выглядит Церковь Рождества Христова?",

"Как выглядит Храм Архангела Михаила?",

"Как выглядит Губернаторский дом в Ярославле?",

"Как выглядит Владимирская церковь?",

"Как выглядит Мемориальный дом-музей М.

Богдановича?",

"Как выглядит Власьевская (Знаменская) башня?",

"Как выглядит Знаменская церковь?",

"Как выглядит Волжская башня?",

"Как выглядит Демидовский сад?",

"Как выглядит Власьевский сквер?",

"Как выглядит памятник М.В. Фрунзе?",

"Как выглядит памятник Фёдору Ивановичу Толбухину?",

"Как выглядит Памятник Леониду Николаевичу

Трефолеву?",

"Как выглядит Казанский женский монастырь?",

"Как выглядит Николо-Сковородский монастырь?",

"Как выглядит Петровский монастырь?",

"Как выглядит мост Волжские Ворота?",

"Как выглядит Музей истории города Ярославля?",

"Как выглядит Ярославский музей-заповедник?",

"Как выглядит Ярославский художественный музей?",

"Как выглядит Парк 1000-летия Ярославля?",

"Как выглядит Парк на Стрелке?",

"Как выглядит Театр кукол в Ярославле?",

"Как выглядит Российский государственный

академический театр драмы имени Федора Волкова?",

"Как выглядит Успенский собор?",

"Как выглядит Ярославская соборная мечеть?",

"Как выглядит Ярославский цирк?",

"Как выглядит Ярославский зоопарк?",

"Как выглядит Дмитриевская башня?"]

ground_truths = ["большое здание с часами наверху",

"большое здание с часами наверху",

"ночной снимок здания с лестницей и деревьями на

переднем плане",

"в комнате большой камень, стоящий на деревянном

столе",

"размытая комната с большим количеством старых

фотографий и других предметов",

"размытая фотография комнаты с пианино и другими

музыкальными инструментами",

"размытое изображение здания со статуей и

лестницей",

церковью",	"здание с красной крышей и синим окном", "размытое изображение города с собором и
заднем плане",	"размытая беседка в парке с прудом и деревьями на
фонтаном",	"размытый вид церкви со шпилем и лестницей", "размытый вид церкви с золотым куполом и
наверху",	"размытое здание с золотым куполом наверху", "размытое здание с куполом и синим куполом
куполом",	"желтое здание с двумя башнями и желтым куполом", "размытый вид большого белого здания с золотым
птицу, на снегу",	"статуя мужчины на постаменте посреди парка", "вид с воздуха на статую мужчины, держащего
заднем плане",	"статуя мужчины, стоящего перед зданием", "мост через водоем", "мост через водоем", "размытое здание с часами наверху в снегу", "размытый вид стадиона с большим зданием на
куполом",	"размытое белое здание с зеленой крышей и зеленым
черным забором",	"люди, гуляющие по площади перед синей церковью", "размытое здание со шпилем и часовой башней", "размытое здание с зелено-красным фасадом и
куполами",	"размытый вид большого белого здания с золотыми
стеклянным фасадом",	"старая фотография дома с множеством окон", "машины, припаркованные перед зданием с синим
множеством деревьев",	"размытый вид большого здания с часовой башней", "размытый вид города с большим зданием и
металла",	"крупный план нескольких различных видов
воротами",	"размытое здание с часами наверху", "поезд идет по путям перед зданием", "размытый вид здания с часовой башней наверху", "памятник посреди парка с деревьями", "машины припаркованы на улице перед зданием с
холма",	"памятник с высоким обелиском посреди парка", "размытая кирпичная стена с башней на вершине
замком на заднем плане",	"лыжники поднимаются по заснеженному холму с

"размытая кирпичная башня с часами наверху",
"большое здание с вывеской на стороне",
"статуя медведя перед зданием",
"крупный план вывески со зданием на заднем
плане",
башней",
наверху",
"большое здание с двумя куполами наверху",
"большое белое здание с большим парадным входом",
"большое деревянное здание с граффити на стене",
"улица со зданием и деревом посередине",
"желтые ворота с часами наверху",
"две фотографии зебры и верблюда в зоопарке",
"дорожка, ведущая к детской площадке с горкой",
"размытая фотография церкви с большими часами на
стене",
священником",
"размытая группа людей в церкви со свечами и
"большое здание с золотым куполом наверху",
"размытая церковь со шпилем и синей крышей",
"размытый вид церкви со шпилем и куполом",
"размытый вид церкви с часовой башней",
"вид с воздуха на большое зеленое и белое здание
с желтым куполом",
куполами",
зданием",
"большая столовая со столом и стульями",
"машины припаркованы на обочине дороги перед
"зеленое и красное здание с вывеской на углу",
"красно-белый логотип с надписью hl motor",
"размытый вид зеленого здания с двумя башнями и
часовой башней",
плане",
плане",
куполами",
здании",
белой дверью",
"фотография картины церкви с небом на заднем
"вид на пустынную местность с закатом на заднем
"размытый вид большого белого здания с золотыми
"фотография здания с часами на фасаде",
"вывеска с надписью movini на русском языке на
"размытый вид церкви с прудом перед ней",
"размытый мужчина стоит перед синим зданием с
"размытая белая церковь с золотым куполом и

крестом наверху",
небом",
наверху",
башней",

башней",
шпилем",
наверху",
куполами",

снегу",

куполом",
небом",
куполами и шпилем",
крышей",
зеленой крышей",

стенах",

окнами",

золотым куполом",

плане",

машинами",

"размытый вид церкви с часовой башней и облачным
"размытый вид церкви с золотым куполом и крестом
"вид с воздуха на большое белое здание с часовой
"большое здание с множеством окон",
"размытый вид большого белого здания с часовой
"размытая белая церковь с зеленой крышей и
"размытое белое здание с золотым куполом
"размытый вид большого белого здания с золотыми
"размытая статуя женщины перед зданием",
"размытая церковь со шпилем и часовой башней на
"размытая карта зеленой зоны с синей линией",
"размытое здание с зеленой крышей и зеленым
"размытая церковь с зеленой крышей под облачным
"размытая красно-белая церковь с зелеными
"желто-зеленое здание с двумя башнями и зеленой
"размытое белое здание с зелеными куполами и
"размытое изображение церкви со шпилем",
"комната со столом, стульями и картинами на
"размытая церковь со шпилем и колокольной",
"крупный план дома с металлической крышей и
"размытое здание с башней и часами наверху",
"машины припаркованы перед белым зданием с
"ночной вид большого здания с часами наверху",
"фонтан в центре парка с часовой башней на заднем
"женщина идет по дорожке в парке",
"здания в городе с припаркованными перед ними
"черно-белое фото мужчины в форме",
"размытый бюст мужчины с бородой",
"размытый вид белой церкви с желтым куполом",
"размытое здание с зеленой крышей и зелеными

```

шпилями в солнечный день",
    "винтажное фото церкви со шпилем и забором",
    "машины, припаркованные на улице перед белым
зданием",
    "размытая витрина с различными предметами на
столе с тарелкой",
    "белое здание с синей вывеской",
    "желтое здание с белой отделкой и окнами",
    "вид на парк с фонтаном и озером на заднем
плане",
    "размытый вид парка со статуей птицы на вершине
колонны",
    "артисты в костюмах на сцене с ребенком в
кроватьке",
    "размытая аудитория людей в большом зале,
слушающая выступающего",
    "размытый вид большого белого здания с золотыми
куполами",
    "небольшой пруд перед зданием с мостом",
    "большое здание с вывеской на фасаде",
    "большая плюшевая игрушка, сидящая на гигантском
шаре",
    "люди, идущие по тротуару возле кирпичного здания
с зеленой крышей"]
with open('whole_context.md', 'r', encoding='utf-8') as file:
    file_content = file.read()
    contexts = extract_all_contexts(file_content)
    print(len(questions))
    print(len(ground_truths))
    print(len(contexts))
    if df is not None:
        df['question'] = questions
        df['ground_truths'] = ground_truths
        df['contexts'] = contexts

    return df

dfs = pd.read_pickle('doc_source.pkl')
dfs.head(3)

```

		text
City \ num		
0	**Название:** Доходный дом купца Чувильдина\n*...	Екатеринбург
1	**Название:** №29 Дом В.И. Чувильдина\n**Тип о...	Екатеринбург
2	**Название:** Дом Актёра\n**Тип объекта:** Дом...	Екатеринбург

	Name	Lon	Lat	\
num				
0	Доходный дом купца Чувильдина	60.607727	56.832176	
1	№29 Дом В.И. Чувильдина	60.607689	56.832268	
2	Дом Актёра	60.597950	56.838634	

	image
num	
0	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
1	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...
2	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...

```
dfs = data_preprocess(dfs)
```

```
120
120
120
```

```
dfs.head(3).T
```

num	0	\
text	**Название:** Доходный дом купца Чувильдина\n*	...
City		Екатеринбург
Name		Доходный дом купца Чувильдина
Lon		60.607727
Lat		56.832176
image	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
question		Как выглядит Доходный дом купца Чувильдина?
ground_truths		большое здание с часами наверху
contexts		На изображении Доходный дом купца Чувильдина: ...

num	1	\
text	**Название:** №29 Дом В.И. Чувильдина\n**Тип о...	
City		Екатеринбург
Name		№29 Дом В.И. Чувильдина
Lon		60.607689
Lat		56.832268
image	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...	
question		Как выглядит Дом В.И. Чувильдина?
ground_truths		большое здание с часами наверху
contexts		Внешний вид №29 Дом В.И. Чувильдина: большое з...

num	2
text	**Название:** Дом Актёра\n**Тип объекта:** Дом...
City	Екатеринбург
Name	Дом Актёра
Lon	60.59795
Lat	56.838634
image	/9j/4AAQSkZJRgABAQAAQABAAD/2wBDAAUEBAUEAwUFBA...

```
question          Как выглядит Дом Актёра?
ground_truths     ночной снимок здания с лестницей и деревьями н...
contexts          Внешний вид Дом Актёра: ночной снимок здания с...
```

```
dfs.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 120 entries, 0 to 119
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   text                   120 non-null   object
1   City                   120 non-null   object
2   Name                   120 non-null   object
3   Lon                    120 non-null   float64
4   Lat                    120 non-null   float64
5   image                  120 non-null   object
6   question               120 non-null   object
7   ground_truths          120 non-null   object
8   contexts                120 non-null   object
9   answer                 120 non-null   object
10  processing_status       120 non-null   object
dtypes: float64(2), object(9)
memory usage: 15.3+ KB
```

Сохранение датасета

```
dfs.to_pickle('doc_source.pkl')

# dfs = pd.read_pickle('doc_source.pkl')
```

Создание пайплайна для обработки

```
question = "Как выглядит Доходный дом купца Чувильдина?"

answer, relevant_docs = answer_with_rag(question, READER_LLM,
                                         KNOWLEDGE_VECTOR_DATABASE,
                                         reranker=reranker)

print(answer)
```

```
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
```

Доходный дом купца Чувильдина – величественное здание в центре Екатеринбурга, представляющее собой образец городской застройки начала XX века. Это большой дом с массивной архитектурой, украшенный декоративными элементами и часами на фасаде, подчеркивающими статус владельца. Экскурсовод описывает его как символ купеческого богатства и процветания дореволюционного Екатеринбурга.

(Документ 0)

```
retrieved_docs =  
KNOWLEDGE_VECTOR_DATABASE.max_marginal_relevance_search(  
    query=question, k=3, fetch_k=10)
```

```
dfs.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 120 entries, 0 to 119
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	text	120 non-null	object
1	City	120 non-null	object
2	Name	120 non-null	object
3	Lon	120 non-null	float64
4	Lat	120 non-null	float64
5	image	120 non-null	object
6	question	120 non-null	object
7	ground_truths	120 non-null	object
8	contexts	120 non-null	object

```
dtypes: float64(2), object(7)
```

```
memory usage: 9.4+ KB
```

```
def answer_relevancy(progress_callback=None, save_interval=10):  
    """
```

Улучшенная процедура для генерации ответов с прогрессом и промежуточным сохранением

Parameters:

- progress_callback: функция для отслеживания прогресса

- save_interval: интервал для вывода прогресса

```
    """
```

Инициализация колонок

```
    if 'answer' not in dfs.columns:
```

```
        dfs['answer'] = None
```

```
    if 'processing_status' not in dfs.columns:
```

```
        dfs['processing_status'] = 'pending'
```

```
    processed_count = 0
```

```
    for index, row in dfs.iterrows():
```

```
        # Пропускаем уже обработанные вопросы
```

```
        if pd.isna(dfs.at[index, 'answer']) and dfs.at[index,  
'answer'] != '':
```

```
            continue
```

```
        question = row['question']
```



```

try:
    # Обновляем статус
    dfs.at[index, 'processing_status'] = 'processing'

    # Генерируем ответ
    answer, relevant_docs = answer_with_rag(question,
READER_LLM,
KNOWLEDGE_VECTOR_DATABASE,
reranker=reranker)

    # Записываем результаты
    dfs.at[index, 'answer'] = answer
    dfs.at[index, 'processing_status'] = 'completed'

    processed_count += 1

    # Выводим прогресс
    if processed_count % save_interval == 0:
        print(f"Обработано {processed_count}/{len(dfs)}
вопросов")
        if progress_callback:
            progress_callback(processed_count, len(dfs))

except Exception as e:
    error_msg = f"Ошибка: {str(e)}"
    dfs.at[index, 'answer'] = error_msg
    dfs.at[index, 'processing_status'] = 'failed'
    print(f"Ошибка при обработке вопроса {index}:
{error_msg}")

    print(
        f"Обработка завершена! Успешно обработано: {processed_count}
вопросов")

dfs['answer'] = ''

# Простой вызов
answer_relevancy()

# С прогресс-баром (требуется установки tqdm)
from tqdm import tqdm

def answer_relevancy_with_progress():
    # Создаем прогресс-бар
    pbar = tqdm(total=len(dfs), desc="Обработка вопросов")

    def update_progress(current, total):
        pbar.update(1)

```

```
answer_relevancy(progress_callback=update_progress)
pbar.close()
```

Вызов с прогресс-баром

```
answer_relevancy_with_progress()
```

```
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
Обработано 10/120 вопросов
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
```



```
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
Обработано 50/120 вопросов
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
Обработано 60/120 вопросов
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
```


=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
Обработано 80/120 вопросов
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
Обработано 90/120 вопросов
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...

[illegible]


```
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
Обработано 110/120 вопросов
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
=> Retrieving documents...
=> Reranking documents...
=> Generating answer...
Обработано 120/120 вопросов
Обработка завершена! Успешно обработано: 120 вопросов

Обработка вопросов:   0%|          | 0/120 [00:00<?, ?it/s]

Обработка завершена! Успешно обработано: 0 вопросов
```

Сохранение датасета

```
dfs.to_pickle('doc_source.pkl')
```

Реализация Answer Relevancy

```
# Для эмбедингов
# !pip install sentence-transformers

import pandas as pd
import numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer(EMBEDDING_MODEL_NAME)

def answer_relevancy(questions, answers, model):
    """
    Calculate Answer Relevancy using cosine similarity between
    question and answer embeddings
    """
    # Получаем эмбединги для вопросов и ответов
    question_embeddings = model.encode(questions.tolist())
    answer_embeddings = model.encode(answers.tolist())

    # Вычисляем косинусное сходство для каждой пары
    similarities = []
    for i in range(len(questions)):
        similarity = \
            cosine_similarity([question_embeddings[i]],
                              [answer_embeddings[i]])[0][
                                0]
        similarities.append(similarity)

    return similarities
```

Протестируйте ваш RAG (3 балла)

Вычисляем Answer Relevancy для всех строк

```
# Вычисляем Answer Relevancy для всех строк
answer_relevancy_scores = answer_relevancy(dfs['question'],
dfs['answer'],
model)

# Добавляем результаты в датасет
dfs['answer_relevancy'] = answer_relevancy_scores

# Выводим статистику
print("Answer Relevancy Statistics:")
print(f"Mean: {dfs['answer_relevancy'].mean():.4f}")
print(f"Std: {dfs['answer_relevancy'].std():.4f}")
```

```
print(f"Min: {dfs['answer_relevancy'].min():.4f}")
print(f"Max: {dfs['answer_relevancy'].max():.4f}")
```

Answer Relevancy Statistics:

Mean: 0.6346

Std: 0.0731

Min: 0.4834

Max: 0.8120

Примеры

Посмотрим на примеры с разной релевантностью

```
print("\nПримеры с высокой релевантностью (>0.8):")
high_relevancy = dfs[dfs['answer_relevancy'] > 0.8]
for idx, row in high_relevancy.head(3).iterrows():
    print(f"Score: {row['answer_relevancy']:.4f}")
    print(f"Question: {row['question']}")
    print(f"Answer: {row['answer']}")
    print("----")
```

Примеры с высокой релевантностью (>0.8):

Score: 0.8045

Question: Как выглядит Нижегородский государственный академический театр драмы им. М. Горького?

Answer: Экскурсовод описывает Нижегородский государственный академический театр драмы им. М. Горького как старейший театр Нижнего Новгорода, расположенный в историческом центре города. Здание театра отличается строгими архитектурными формами и величественным фасадом, подчеркивающим его статус культурного учреждения региона.
(Документ 0)

Score: 0.8120

Question: Как выглядит памятник "Белокаменные памятники Владимира и Суздаля"?

Answer: Экскурсовод описывает памятник «Белокаменные памятники Владимира и Суздаля» как величественное белоснежное сооружение с золотыми куполами, возвышающееся среди живописной природы.
(Документ 0)

Дополнительная информация: Этот объект является частью Владимиро-Суздальского музея-заповедника, расположенного в городе Владимир, Россия.

Примеры с низкой релевантностью (<0.5):

```
low_relevancy = dfs[dfs['answer_relevancy'] < 0.55]
for idx, row in low_relevancy.head(3).iterrows():
```

```
print(f"Score: {row['answer_relevancy']:.4f}")
print(f"Question: {row['question']}")
print(f"Answer: {row['answer']}")
print("----")
```

Score: 0.5149

Question: Как выглядит Дом Актёра?

Answer: Дом Актёра в Екатеринбурге – это уютное историческое здание, расположенное в центре города. Экскурсовод описывает его как место, наполненное атмосферой театральной жизни и творчества, где каждый уголок хранит память о знаменитых актёрах и спектаклях. Здание имеет выразительный архитектурный стиль, украшенное лепниной и декоративными элементами фасада. Ночной снимок показывает величественную лестницу и деревья на переднем плане, подчёркивая живописность места.

(Документ 0)

Score: 0.5010

Question: Как выглядит Дом Метенкова?

Answer: Экскурсовод описывает Дом Метенкова в Екатеринбурге как уютное здание, расположенное в историческом центре города, окружённое зелёными насаждениями. Внутри музея представлены экспозиции, посвящённые жизни и творчеству музыканта Михаила Метенкова, включая старинные музыкальные инструменты и личные вещи.

(Документ 3)

Score: 0.5243

Question: Как выглядит Театр кукол?

Answer: Ярославский государственный театр кукол – уютное театральное пространство, расположенное в городе Ярославле, известное своими яркими костюмами артистов и атмосферной сценой, где часто выступают вместе дети и зрители.

(Документ 0)

Статистика метрики по всему набору

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

def plot_overall_relevancy_report(df):
    """Визуализация общей статистики Answer Relevancy с matplotlib"""

    scores = df['answer_relevancy']

    # Создаем фигуру с subplots
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
```

```

# График 1: Гистограмма распределения
ax1.hist(scores, bins=20, color='skyblue', edgecolor='black',
alpha=0.7)
ax1.axvline(scores.mean(), color='red', linestyle='--',
linewidth=2,
            label=f'Mean: {scores.mean():.3f}')
ax1.axvline(scores.median(), color='green', linestyle='--',
linewidth=2,
            label=f'Median: {scores.median():.3f}')
ax1.set_xlabel('Answer Relevancy Score')
ax1.set_ylabel('Frequency')
ax1.set_title('Distribution of Answer Relevancy Scores')
ax1.legend()
ax1.grid(True, alpha=0.3)

# График 2: Категории качества
categories = {
    'Excellent (0.8-1.0)': ((scores >= 0.8) & (scores <=
1.0)).sum(),
    'Good (0.6-0.8)': ((scores >= 0.6) & (scores < 0.8)).sum(),
    'Fair (0.4-0.6)': ((scores >= 0.4) & (scores < 0.6)).sum(),
    'Poor (0.0-0.4)': (scores < 0.4).sum()}

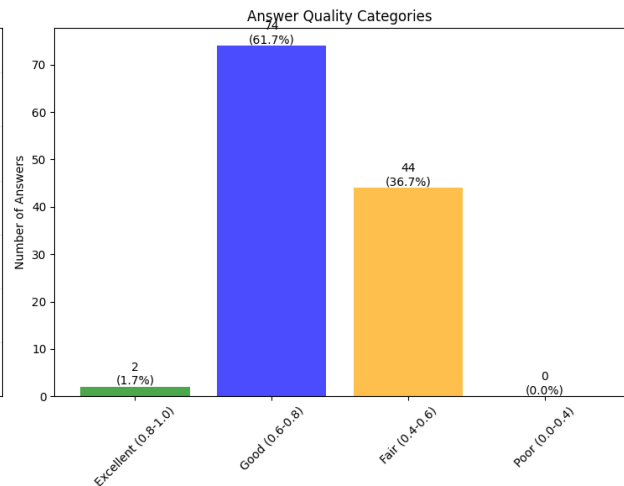
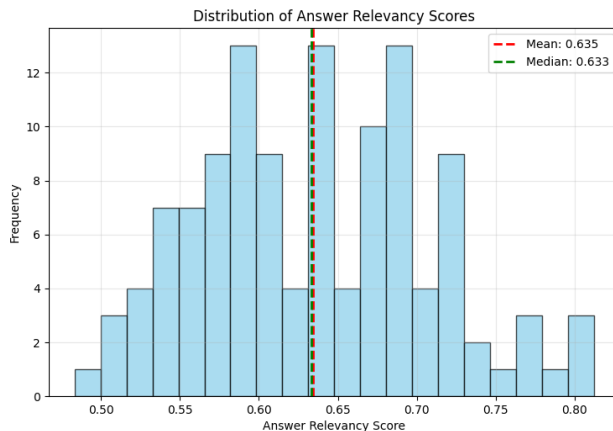
colors = ['green', 'blue', 'orange', 'red']
bars = ax2.bar(categories.keys(), categories.values(),
color=colors,
                alpha=0.7)
ax2.set_title('Answer Quality Categories')
ax2.set_ylabel('Number of Answers')
ax2.tick_params(axis='x', rotation=45)

# Добавляем значения на столбцы
for bar, count in zip(bars, categories.values()):
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width() / 2., height,
            f'{count}\n({count / len(scores) * 100:.1f}%)',
ha='center',
            va='bottom')

plt.tight_layout()
plt.show()

plot_overall_relevancy_report(dfs)

```



Вывод

Answer Relevancy Демонстрирует, насколько ответы соответствуют заданным вопросам. В модели большинство ответов хорошо соответствуют вопросам. Однако приличный объем ответов в данный момент является удовлетворительным. Возможно это из-за того что я настроил LLM на генерацию более полного ответа.

Сохранение результатов

```
# dfs.to_pickle('doc_source.pkl')
```

Дополнительная часть (+3 балла)

Реализуйте одну из метрик retrieval-части на выбор.

Для того, чтобы понять model as a judge подход из RAGAS, советуем посетить [этот кукбук](#)

Расчет метрик Context Recall, Context Precision

- **Context Recall:** насколько ответ покрывает ground truth
- **Context Precision:** насколько ответ релевантен контексту

```
def calculate_context_metrics(ground_truths, answers, contexts):
    """
    расчет метрик через эмбединги
    """
    model = SentenceTransformer(EMBEDDING_MODEL_NAME)

    context_recall_scores = []
    context_precision_scores = []

    for gt, answer, context in zip(ground_truths, answers, contexts):
        # Получаем эмбединги
```

```

gt_embedding = model.encode([str(gt)])
answer_embedding = model.encode([str(answer)])
context_embedding = model.encode([str(context)])

# Context Recall: насколько ответ покрывает ground truth
recall_similarity = \
    cosine_similarity(gt_embedding, answer_embedding)[0][0]
context_recall_scores.append(recall_similarity)

# Context Precision: насколько ответ релевантен контексту
precision_similarity = \
    cosine_similarity(answer_embedding, context_embedding)[0][0]
context_precision_scores.append(precision_similarity)

return context_recall_scores, context_precision_scores

context_recall_simple, context_precision_simple =
calculate_context_metrics(
    dfs['ground_truths'], dfs['answer'], dfs['contexts'])

# Добавляем метрики в датасет
dfs['context_recall_simple'] = context_recall_simple
dfs['context_precision_simple'] = context_precision_simple

# Статистика по метрикам
print("=== CONTEXT RECALL ===")
print(
    f"Mean: {dfs['context_recall_simple'].mean():.4f}, Std: "
    f"{dfs['context_recall_simple'].std():.4f}")

print("\n=== CONTEXT PRECISION ===")
print(
    f"Mean: {dfs['context_precision_simple'].mean():.4f}, Std: "
    f"{dfs['context_precision_simple'].std():.4f}")

=== CONTEXT RECALL ===
Mean: 0.5762, Std: 0.0877

=== CONTEXT PRECISION ===
Mean: 0.7454, Std: 0.0735

# Примеры с разными показателями
print("\n Примеры с высоким Context Recall (>0.8):")
high_recall = dfs[dfs['context_recall_simple'] > 0.7]
for idx, row in high_recall.head(2).iterrows():
    print(f"Recall: {row['context_recall_simple']:.3f}")
    print(f"GT: {row['ground_truths'][:100]}...")
    print(f"Answer: {row['answer'][:100]}...")
    print("----")

```

□ Примеры с высоким Context Recall (>0.8):

Recall: 0.719

GT: желтое здание с двумя башнями и желтым куполом...

Answer: Экскурсовод описывает Свято-Троицкий Кафедральный собор
Екатеринбурга как желтое здание с двумя башн...

Recall: 0.701

GT: вид с воздуха на статую мужчины, держащего птицу, на снегу...

Answer: Экскурсовод описывает памятник Комсомолу Урала в Екатеринбурге
как статую мужчины, стоящего на поста...

```
print("\n△ Примеры с низким Context Precision (<0.55):")
```

```
low_precision = dfs[dfs['context_precision_simple'] < 0.55]
```

```
for idx, row in low_precision.head(2).iterrows():
```

```
    print(f"Precision: {row['context_precision_simple']:.3f}")
```

```
    print(f"Context: {row['contexts'][:100]}...")
```

```
    print(f"Answer: {row['answer'][:100]}...")
```

```
    print("----")
```

△ Примеры с низким Context Precision (<0.55):

Precision: 0.525

Context: Фотография Цирк показывает: большое здание с вывеской на
фасаде...

Answer: В представленном контексте информация о Ярославском цирке
отсутствует.

Источник: нет...

Статистика метрик по всему набору

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import pandas as pd
```

```
def plot_overall_context_recall_report(df):
```

```
    """Визуализация общей статистики Context Recall с matplotlib"""
```

```
    if 'context_recall_simple' in df.columns:
```

```
        scores = df['context_recall_simple']
```

```
        metric_name = 'Context Recall (Simple)'
```

```
    else:
```

```
        return
```

```
    # Создаем фигуру с subplots
```

```
    fig, ((ax1, ax2)) = plt.subplots(1, 2, figsize=(15, 6))
```



```

# График 1: Гистограмма распределения
n, bins, patches = ax1.hist(scores, bins=20, color='lightcoral',
                             edgecolor='black', alpha=0.7,
density=True)
ax1.axvline(scores.mean(), color='red', linestyle='--',
linewidth=2,
              label=f'Mean: {scores.mean():.3f}')
ax1.axvline(scores.median(), color='blue', linestyle='--',
linewidth=2,
              label=f'Median: {scores.median():.3f}')
ax1.set_xlabel('Context Recall Score')
ax1.set_ylabel('Density')
ax1.set_title(f'Distribution of {metric_name}')
ax1.legend()
ax1.grid(True, alpha=0.3)

# График 2: Категории качества
categories = {
    'Excellent\n(0.8-1.0)': ((scores >= 0.8) & (scores <=
1.0)).sum(),
    'Good\n(0.6-0.8)': ((scores >= 0.6) & (scores < 0.8)).sum(),
    'Fair\n(0.4-0.6)': ((scores >= 0.4) & (scores < 0.6)).sum(),
    'Poor\n(0.0-0.4)': (scores < 0.4).sum()}

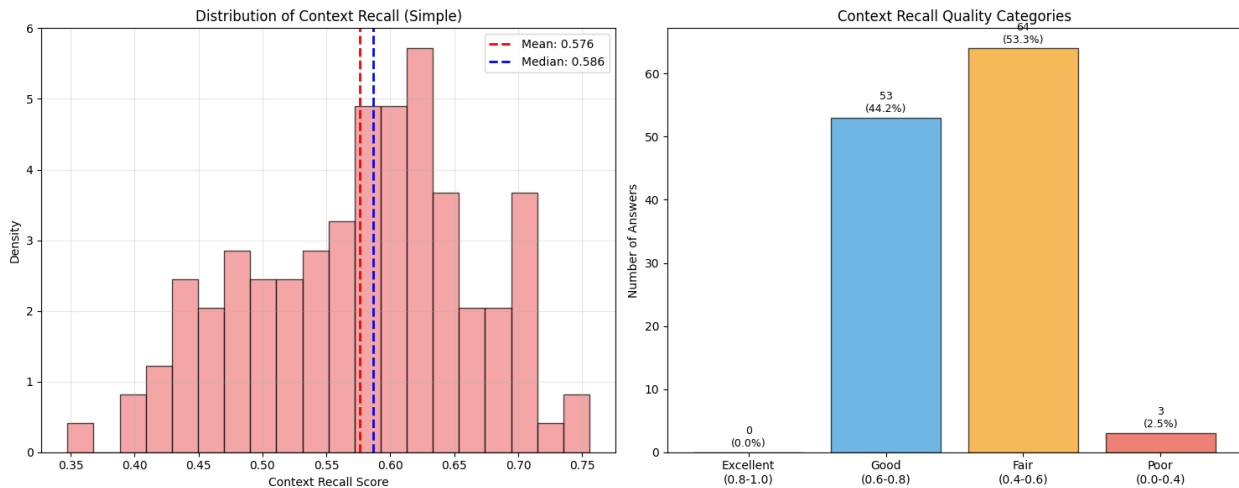
colors = ['#2ecc71', '#3498db', '#f39c12', '#e74c3c']
bars = ax2.bar(categories.keys(), categories.values(),
color=colors,
               alpha=0.7, edgecolor='black')
ax2.set_title('Context Recall Quality Categories')
ax2.set_ylabel('Number of Answers')
ax2.tick_params(axis='x', rotation=0)

# Добавляем значения на столбцы
for bar, count in zip(bars, categories.values()):
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width() / 2., height + 0.5,
             f'{count}\n({count / len(scores) * 100:.1f}%)',
ha='center',
             va='bottom', fontsize=9)

plt.tight_layout()
plt.show()

plot_overall_context_recall_report(dfs)

```



Вывод

Метрика **Context Recall** определяет насколько ответ покрывает ground truth. Модель в данный момент еще имеет солидный запас по настройке. Практически вся метрика лежит в области *удовлетворительно*

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

def plot_overall_context_precision_report(df):
    """Визуализация общей статистики Context Precision для всего
    набора данных"""

    # Используем детальную версию метрики
    scores = df['context_precision_simple']

    # Создаем фигуру с subplots
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # График 1: Гистограмма распределения
    n, bins, patches = ax1.hist(scores, bins=20, color='lightcoral',
                                edgecolor='black', alpha=0.7,
                                density=False)
    ax1.axvline(scores.mean(), color='red', linestyle='--',
                linewidth=2,
                label=f'Mean: {scores.mean():.3f}')
    ax1.axvline(scores.median(), color='blue', linestyle='--',
                linewidth=2,
                label=f'Median: {scores.median():.3f}')
    ax1.set_xlabel('Context Precision Score')
    ax1.set_ylabel('Frequency')
    ax1.set_title('Distribution of Context Precision Scores\n(Overall
```

```

Dataset)')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # Добавляем аннотацию с общей статистикой
    stats_text = f'Total samples: {len(scores)}\nStd:
{scores.std():.3f}\nMin: {scores.min():.3f}\nMax: {scores.max():.3f}'
    ax1.text(0.02, 0.98, stats_text, transform=ax1.transAxes,
            verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='wheat',
alpha=0.8))

    # График 2: Категории качества Context Precision
    categories = {
        'Excellent\n(0.8-1.0)': ((scores >= 0.8) & (scores <=
1.0)).sum(),
        'Good\n(0.6-0.8)': ((scores >= 0.6) & (scores < 0.8)).sum(),
        'Fair\n(0.4-0.6)': ((scores >= 0.4) & (scores < 0.6)).sum(),
        'Poor\n(0.0-0.4)': (scores < 0.4).sum()}

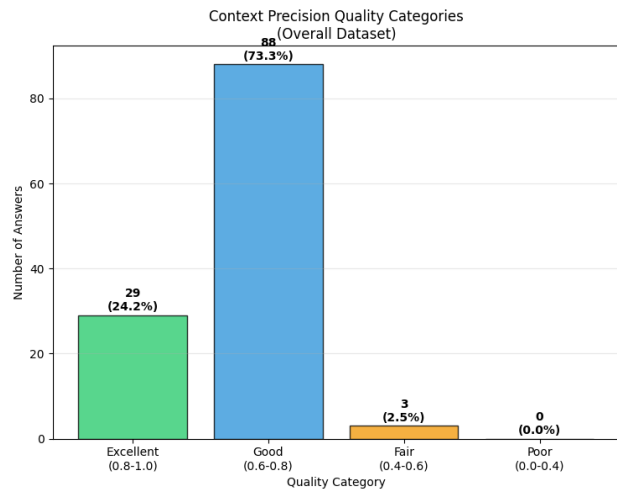
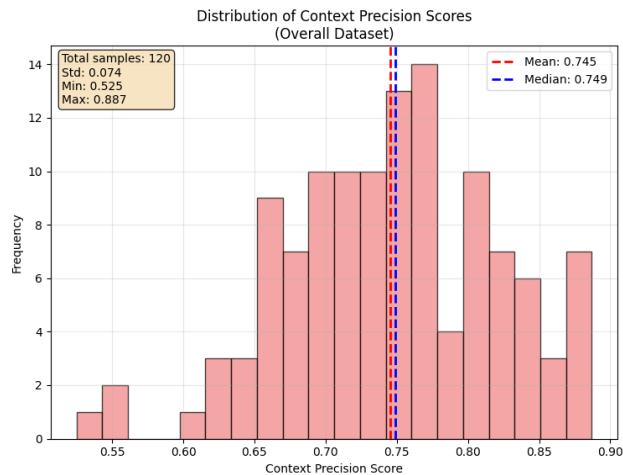
    colors = ['#2ecc71', '#3498db', '#f39c12',
              '#e74c3c'] # Зеленый, синий, оранжевый, красный
    bars = ax2.bar(categories.keys(), categories.values(),
color=colors,
                    alpha=0.8, edgecolor='black')
    ax2.set_title('Context Precision Quality Categories\n(Overall
Dataset)')
    ax2.set_ylabel('Number of Answers')
    ax2.set_xlabel('Quality Category')
    ax2.grid(True, alpha=0.3, axis='y')

    # Добавляем значения и проценты на столбцы
    for bar, count in zip(bars, categories.values()):
        height = bar.get_height()
        percentage = (count / len(scores)) * 100
        ax2.text(bar.get_x() + bar.get_width() / 2., height + 0.5,
                f'{count}\n({percentage:.1f}%)', ha='center',
va='bottom',
                fontweight='bold')

    plt.tight_layout()
    plt.show()

plot_overall_context_precision_report(dfs)

```



Вывод

Метрика **Context Precision** определяет насколько ответ релевантен контексту. Модель в данный момент демонстрирует хорошие результаты. Практически вся метрика лежит в области *хорошо*

Сохранение набора данных

```
dfs.to_pickle('doc_source.pkl')
```